



Université de Sousse
Institut Supérieur d'Informatique et des Technologies de
Communication
(ISITCOM)
Département Informatique

RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du diplôme national d'ingénieur en informatique

Conception et Implémentation d'un Système de Planification de Voyage Intelligent basé sur les Agents LLM et un Moteur de Recommandation Contextuel par Bandits (LinUCB)

Réalisé par :

Ahmed Addali

Encadré par :

Nizar Tenzekhti

Membres du jury

Président M./Mme

Rapporteur M./Mme

Examineur M./Mme

Année universitaire : 2024 – 2025

Dédicace

À mes chers parents, dont les sacrifices silencieux et le soutien indéfectible ont pavé chaque étape de mon parcours. Aucun mot ne saurait exprimer la profondeur de ma reconnaissance envers vous.

À mes enseignants, qui ont su éveiller en moi une passion sincère pour l'intelligence artificielle et les sciences de l'informatique, et qui m'ont transmis la rigueur nécessaire à la recherche.

À mes amis et camarades de promotion, compagnons de longues nuits de travail et de discussions enrichissantes, dont la solidarité a rendu ce parcours plus humain.

À tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce travail : que ces pages soient le témoignage modeste de ma gratitude.

Remerciements

Au terme de ce projet de fin d'études, il m'est agréable d'exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, directement ou indirectement, à l'aboutissement de ce travail.

Mes remerciements les plus sincères s'adressent en premier lieu à mon encadrant, **M. Nizar Tenzekhti**, pour sa disponibilité constante, ses orientations judicieuses et son encadrement technique rigoureux tout au long de ce projet. Sa maîtrise des architectures logicielles et des systèmes d'intelligence artificielle a constitué une source d'inspiration permanente et a significativement enrichi la qualité de ce travail.

Je tiens également à remercier les membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail. Leurs remarques et observations contribueront sans nul doute à son amélioration.

Ma reconnaissance s'étend à l'ensemble du corps enseignant de l'ISITCOM, et en particulier au département d'informatique, pour la qualité de la formation dispensée au cours de ces années, formation qui m'a fourni les fondements théoriques et pratiques nécessaires à la réalisation de ce projet.

Je remercie chaleureusement ma famille pour son soutien moral et matériel inestimable, sa patience et ses encouragements constants, qui ont été un moteur essentiel durant les moments les plus exigeants de ce parcours.

Enfin, j'adresse mes remerciements à mes collègues et amis pour les échanges fructueux, les relectures attentives et la collaboration productive qui ont accompagné chaque étape de ce travail.

Que toutes ces personnes trouvent ici l'expression de ma profonde gratitude.

Résumé

Les touristes visitant l'île de Djerba sont confrontés à l'absence d'outils de planification intelligents capables de combiner la génération automatique d'itinéraires multi-journées avec des recommandations personnalisées en temps réel. Les plateformes existantes se limitent soit à des moteurs de recherche par filtres statiques, soit à des systèmes de recommandation collaboratifs qui ignorent le contexte immédiat du voyageur — son budget, ses contraintes temporelles, sa localisation et ses préférences linguistiques.

Ce travail propose **Guidni**, une plateforme de planification de voyage articulée autour de deux sous-systèmes complémentaires. Le premier est un **agent planificateur conversationnel** construit sur une machine à états LangGraph comportant quatre nœuds (THINK, EXECUTE_TOOL, VALIDATE, RESPOND) et disposant de dix-huit outils appelables par le LLM, couvrant la gestion du profil utilisateur, la recherche d'activités, les données météorologiques via Open-Meteo, la géolocalisation par Nominatim et Haversine, ainsi qu'un pipeline RAG multilingue (français, anglais, arabe) reposant sur les embeddings denses BAAI/bge-m3 et un ré-ordonnancement par cross-encoder BAAI/bge-reranker-v2-m3. Le second sous-système est un **moteur de recommandation hybride LinUCB** exploitant un vecteur de caractéristiques construit par produit de Kronecker ($\phi = x_{\text{user}} \otimes x_{\text{tags}}$, dimension 306), un pipeline de classement en dix étapes intégrant des filtres de diversité et d'exploration, et un apprentissage en ligne à partir de huit signaux comportementaux collectés en temps réel.

Les résultats obtenus attestent d'un planificateur conversationnel de bout en bout opérant en trois langues, d'une latence de scoring inférieure à 100 ms grâce à un cache matriciel en mémoire vive, et d'une ré-indexation RAG incrémentale déclenchée par les événements SQLAlchemy. Ce système contribue à la convergence de la planification de voyage assistée par IA et de l'apprentissage en ligne à partir de signaux comportementaux pour le marché touristique de Djerba.

Mots-clés : Agent LLM, LangGraph, RAG multilingue, Bandit contextuel LinUCB, Recommandation personnalisée.

Abstract

Tourists visiting the island of Djerba lack intelligent planning tools that seamlessly combine multi-day itinerary generation with real-time personalised recommendations. Existing platforms are limited to either static filter-based search engines or collaborative recommendation systems that disregard the traveller’s immediate context — budget constraints, temporal availability, geolocation and language preferences.

This work introduces **Guidni**, a travel planning platform built around two complementary sub-systems. The first is a **conversational planner agent** implemented as a LangGraph state machine with four nodes (THINK, EXECUTE_TOOL, VALIDATE, RESPOND) and eighteen LLM-callable tools spanning user profile management, activity search, weather data retrieval via Open-Meteo, geolocation through Nominatim and Haversine distance computation, and a multilingual RAG pipeline (French, English, Arabic) powered by BAAI/bge-m3 dense embeddings and BAAI/bge-reranker-v2-m3 cross-encoder reranking. The second sub-system is a **Hybrid LinUCB contextual bandit recommender** that leverages a Kronecker-product feature vector ($\phi = x_{\text{user}} \otimes x_{\text{tags}}$, dimension 306), a ten-step ranker pipeline incorporating diversity and exploration filters, and online learning from eight real-time behavioural reward signals.

The experimental results demonstrate an end-to-end conversational planner supporting three languages, sub-100 ms recommendation scoring latency through an in-RAM matrix cache, and incremental RAG re-indexing triggered by SQLAlchemy event listeners. The system contributes a joint approach to AI-assisted trip planning and online learning from real-time behavioural signals for the Djerba tourism market.

Keywords: LLM Agent, LangGraph, Multilingual RAG, LinUCB Contextual Bandit, Personalised Recommendation.

Liste des Abréviations

Abréviation	Signification
2TUP	Two-Track Unified Process
AI	Artificial Intelligence
API	Application Programming Interface
AR	Arabe
BAAI	Beijing Academy of Artificial Intelligence
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DB	Database (Base de données)
EN	Anglais
ER	Entity-Relationship (Entité-Relation)
ESM	ECMAScript Modules
FAQ	Frequently Asked Questions (Foire Aux Questions)
FK	Foreign Key (Clé étrangère)
FR	Français
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
IPS	Inverse Propensity Scoring
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model (Grand modèle de langage)
MAB	Multi-Armed Bandit (Bandit multi-bras)
ML	Machine Learning (Apprentissage automatique)
MoSCoW	Must / Should / Could / Won't
NLP	Natural Language Processing (Traitement du langage naturel)

Abréviation	Signification
NLU	Natural Language Understanding (Compréhension du langage naturel)
ORM	Object-Relational Mapping (Mapping objet-relationnel)
PFE	Projet de Fin d'Études
PK	Primary Key (Clé primaire)
PostgreSQL	Post Structured Query Language
RAG	Retrieval-Augmented Generation (Génération augmentée par la récupération)
RAM	Random Access Memory (Mémoire vive)
REST	Representational State Transfer
SQL	Structured Query Language (Langage de requêtes structurées)
TND	Tunisian Dinar (Dinar tunisien)
TTL	Time To Live (Durée de vie)
UCB	Upper Confidence Bound (Borne de confiance supérieure)
UML	Unified Modeling Language (Langage de modélisation unifié)
UP	Unified Process (Processus unifié)
UUID	Universally Unique Identifier (Identifiant unique universel)

Contents

Remerciements	ii
Résumé	iii
Abstract	iv
Liste des Abréviations	v
Liste des Figures	viii
Liste des Tableaux	ix
Introduction Générale	1
0.1 Contexte et Motivation	1
0.2 Problématique	1
0.3 Objectifs du Projet	2
0.4 Structure du Rapport	3
0.5 Méthodologie Scrum	4
0.5.1 Pourquoi Scrum ?	4
0.5.2 Rôles Scrum	5
0.6 Planification des Sprints	6
0.7 Conclusion de l'Introduction	9
1 Contexte Général et Étude Préliminaire	10
1.1 Introduction	10
1.2 Agents LLM et planification conversationnelle	10
1.2.1 Les grands modèles de langage (LLMs)	10
1.2.2 Frameworks agentiques : ReAct et LangGraph	13
1.2.3 Abstraction multi-fournisseurs LLM	15
1.3 Génération augmentée par récupération (RAG)	16
1.3.1 Motivation et principe	16
1.3.2 Modèle d'embedding BAAI/bge-m3	17
1.3.3 Réindexation automatique via SQLAlchemy Events	18

1.4	Systèmes de recommandation : vue d'ensemble	18
1.5	Bandits multi-bras contextuels et algorithme LinUCB	19
1.5.1	Formulation du problème MAB	19
1.5.2	L'algorithme LinUCB et la formule de score	22
1.5.3	Ingénierie des caractéristiques	22
1.5.4	Pipeline de classement post-LinUCB	24
1.6	Travaux connexes et comparaison	25
1.7	Choix technologiques et justification	26
	Conclusion du chapitre	27
2	Lancement de Projet	28
2.1	Introduction	28
2.2	Identification des acteurs et besoins fonctionnels	28
2.2.1	Identification des acteurs	28
2.2.2	Diagramme des cas d'utilisation global	29
2.2.3	Description détaillée des cas d'utilisation principaux	30
2.3	Besoins non fonctionnels	32
2.4	Architecture globale du système	33
2.4.1	Vue d'ensemble architecturale	33
2.4.2	Architecture en couches	34
2.5	Conception de la base de données	35
2.5.1	Vue d'ensemble du schéma	35
2.5.2	Description des tables clés	37
2.6	Conception de l'API REST	37
2.7	Sécurité et gestion des données	38
	Conclusion du chapitre	39
3	Sprint 1 — Conception et Implémentation de l'Agent Planificateur	41
3.1	Introduction	41
3.2	Le graphe LangGraph : machine à états	41
3.2.1	L'AgentState TypedDict	41
3.2.2	Fonctions de routage conditionnel	43
3.3	Le nœud THINK	44
3.3.1	Construction du prompt système	44
3.3.2	Injection du contexte dynamique	44
3.4	Les 18 outils LLM	45
3.4.1	Vue d'ensemble et registre des outils	45
3.4.2	Outil rag_search_for_plan : quatre recherches parallèles	46
3.4.3	Outil enrich_activity : cache d'intelligence IA	47
3.5	Le nœud EXECUTE_TOOL	47

3.6	Le nœud VALIDATE	49
3.7	Le nœud RESPOND et la persistance	49
3.8	Le pipeline RAG	50
3.9	Gestion de la mémoire et du contexte	51
3.9.1	Chargement du contexte	51
3.9.2	Résumé automatique	52
3.9.3	Profil utilisateur	52
3.10	Pipeline rapide de génération de plans	52
	Conclusion du chapitre	55
4	Sprint 2 — Implémentation du Moteur de Recommandation Hybrid	
	LinUCB	56
4.1	Introduction	56
4.2	Ingénierie des caractéristiques	56
4.2.1	Le vecteur utilisateur x_{user}	56
4.2.2	Le vocabulaire de tags et le vecteur x_{tags}	57
4.2.3	Le produit de Kronecker ϕ	58
4.3	Moteur de scoring LinUCB	59
4.3.1	Score UCB et paramètres	59
4.3.2	Mise à jour des matrices et récompenses	59
4.4	Pipeline de classement post-LinUCB	60
4.5	Extracteur de tags par LLM	63
4.6	Couche de données runtime	63
4.6.1	Cache matriciel en RAM	63
4.6.2	Traitement par lots d'événements	64
4.6.3	Cache des bras	64
4.7	Tracker JavaScript	65
4.7.1	Architecture et initialisation	65
4.7.2	Stratégie de flush et beacon	65
4.8	Extracteur de tags LLM et jobs nocturnes	66
4.8.1	Mise à jour des profils utilisateurs	66
4.8.2	Gestion de la dérive conceptuelle	67
4.8.3	Planificateur de tâches	67
	Conclusion du chapitre	68
5	Sprint 3 — Tests et Évaluation	69
5.1	Introduction	69
5.2	Stratégie de test	69
5.3	Évaluation de l'agent planificateur	70
5.3.1	Scénarios de test	70

5.3.2	Résultats de l'agent	71
5.4	Évaluation du pipeline RAG	72
5.4.1	Précision de récupération	72
5.4.2	Performance de la réindexation automatique	73
5.5	Évaluation du moteur de recommandation	74
5.5.1	Simulation de récompense cumulée	74
5.5.2	Latence de scoring	75
5.6	Performance globale du système	76
5.7	Discussion et limites	77
	Conclusion du chapitre	78
6	Sprint 4 — Intégration et Déploiement	79
6.1	Introduction	79
6.2	Sprint 4 Planning	79
6.2.1	Objectif du Sprint 4	79
6.2.2	Sprint Backlog du Sprint 4	79
6.3	Analyse	80
6.3.1	Diagramme de Cas d'Utilisation du Sprint 4	80
6.3.2	Description Textuelle des Cas d'Utilisation	81
6.4	Conception	81
6.4.1	Diagramme de Séquence — Health Check	81
6.4.2	Diagramme de Séquence — Rebuild Index	83
6.5	Réalisation	83
6.5.1	Endpoint de Santé (GET /health)	83
6.5.2	Tableau de Bord Administrateur	85
6.5.3	Documentation API OpenAPI	85
6.5.4	Journalisation Rotative	87
6.5.5	Scénario de Démonstration Complète	87
6.6	Tests d'Intégration	89
6.7	Conclusion du Sprint 4	90
	Conclusion Générale et Perspectives	91
	Bilan des contributions	91
	Perspectives et travaux futurs	92
	Mot de la fin	93
A	Annexes	94
	Annexes	94
A.1	Vocabulaire des tags (TAG_VOCAB complet)	94

A.2	Invite système de l'agent Guidni (extraits)	96
A.3	Extraits du schéma de base de données	98
A.4	Captures d'écran de l'application	101

List of Figures

1	Cycle du processus Scrum adapté au projet Guidni	5
2	Diagramme de Gantt des quatre sprints du projet Guidni	7
1.1	Architecture simplifiée d'un décodeur Transformer avec mécanisme d'attention	12
1.2	Graphe LangGraph de l'agent planificateur Guidni	14
1.3	Diagramme de séquence d'un cycle complet de l'agent	14
1.4	Architecture du pipeline RAG avec réindexation automatique	17
1.5	Cycle complet du bandit LinUCB dans le système Guidni	21
2.1	Diagramme des cas d'utilisation global du système Guidni	30
2.2	Architecture globale du système Guidni	34
2.3	Schéma ER du sous-système agent planificateur	36
2.4	Schéma ER du sous-système de recommandation LinUCB	36
2.5	Flux de sécurité et de validation dans l'endpoint /chat	39
3.1	Diagramme d'états du graphe LangGraph avec conditions de routage . . .	43
3.2	Flux d'exécution de l'outil create_plan_structure	48
3.3	Exemple de résultats d'une recherche RAG avec scores de similarité	51
3.4	Pipeline de génération rapide de plans (3 phases parallèles)	54
3.5	Exemple de plan de voyage généré par l'agent Guidni	55
4.1	Pipeline de classement en 10 étapes du moteur de recommandation	62
4.2	Flux de collecte d'événements et mise à jour des matrices LinUCB	66
5.1	Affichage des étapes de raisonnement de l'agent dans l'interface utilisateur	72
5.2	Récompense cumulée comparée : LinUCB vs baselines sur 500 sessions simulées	75
5.3	Réponse de l'endpoint /health indiquant l'état de tous les composants . . .	77
6.1	Diagramme de séquence — Endpoint GET /health	82
6.2	Diagramme de séquence — Reconstruction index RAG	83
6.3	Réponse de l'endpoint GET /health	85
6.4	Documentation API OpenAPI générée automatiquement	86

6.5	Interface de chat avec plan généré (démonstration)	88
6.6	Sortie complète du plan avec étapes de réflexion	89
A.1	Interface de chat de l'agent planificateur Guidni	101
A.2	Exemple complet de plan de voyage généré (5 jours, famille)	102
A.3	Réponse complète de l'endpoint /health	103
A.4	Endpoint /admin/rebuild-index après reconstruction complète de l'index RAG	103
A.5	Documentation OpenAPI auto-générée par FastAPI (/docs)	104
A.6	Exemple de réponse de l'API /api/recommendations avec scores exposés (EXPOSE_SCORES=true)	105

List of Tables

2	Rôles Scrum dans le projet Guidni	6
3	Planification des sprints	8
1.1	Comparaison des fournisseurs LLM intégrés dans Guidni	15
1.2	Comparaison des approches de recommandation	19
1.3	Les 9 composantes du vecteur utilisateur x_{user}	23
1.4	Comparaison avec les systèmes existants	26
2.1	UC-01 : Générer un plan de voyage	31
2.2	UC-02 : Obtenir des recommandations personnalisées	31
2.3	UC-03 : Tracker un événement comportemental	32
2.4	Besoins non fonctionnels classifiés selon la norme ISO 25010	33
2.5	Description des tables AI-owned	37
2.6	Contrat API — Sous-système Agent Planificateur	38
2.7	Contrat API — Sous-système de Recommandation	38
3.1	Champs de l'AgentState et leur comportement	42
3.2	Catalogue des 18 outils LLM	46
4.1	Exemple de construction du vecteur ϕ pour un cas concret	58
4.2	Planning des jobs nocturnes APScheduler	68
5.1	Matrice de couverture des tests	70
5.2	Scénarios de test de l'agent planificateur	71
5.3	Résultats d'évaluation de l'agent planificateur	72
5.4	Résultats de précision de récupération RAG par type et langue	73
5.5	Latence de scoring LinUCB selon le nombre de bras	75
5.6	Métriques de performance système mesurées	76
6.1	Sprint Backlog — Sprint 4	80
6.2	UC-4.1 : Consulter l'état de santé	81
6.3	UC-4.2 : Reconstruire l'index RAG	81
6.4	Tests d'intégration — Sprint 4	90

A.1 Vocabulaire complet des 40 tags (TAG_VOCAB)	94
---	----

Introduction Générale

0.1 Contexte et Motivation

Le tourisme représente un pilier essentiel de l'économie tunisienne, contribuant à hauteur de 14% du PIB national et générant plus de 2 millions d'emplois directs et indirects. L'île de Djerba, avec ses plages paradisiaques, son patrimoine culturel unique et son climat méditerranéen, attire plus de 3 millions de visiteurs internationaux. Cependant, malgré ce potentiel exceptionnel, les touristes font face à des défis majeurs lors de la planification de leur séjour : la dispersion des informations, la barrière linguistique (français, anglais, arabe), et l'absence de recommandations véritablement personnalisées adaptées à leur profil et contexte de voyage.

Les plateformes de voyage existantes se divisent en deux catégories principales. D'une part, les moteurs de recherche génériques (Booking.com, TripAdvisor, Expedia) offrent un catalogue exhaustif mais imposent une navigation fastidieuse à travers des filtres multiples, sans compréhension sémantique des besoins implicites de l'utilisateur. D'autre part, les agents conversationnels récents basés sur les grands modèles de langage (LLM) produisent des réponses fluides mais souffrent d'hallucinations factuelles critiques : ils inventent des prix, des adresses inexistantes ou des horaires erronés, détruisant ainsi la confiance utilisateur dès la première interaction.

Le projet Guidni émerge de ce constat avec une ambition double : concevoir un planificateur de voyage conversationnel multilingue qui combine l'intelligence naturelle des LLM avec l'ancrage factuel d'une base de données locale vérifiée, et développer un moteur de recommandation adaptatif qui apprend en temps réel des préférences contextuelles des utilisateurs pour affiner ses suggestions au fil des interactions.

0.2 Problématique

La question centrale qui guide ce travail de fin d'études est la suivante :

« Comment concevoir et implémenter une plateforme intelligente de planification et de recommandation touristique pour Djerba qui combine un agent conversationnel multilingue basé sur les LLM avec un moteur de recommandation adaptatif »

par apprentissage en ligne, tout en garantissant la fiabilité des informations fournies et l'adaptation aux préférences contextuelles des utilisateurs ? »

Cette problématique soulève plusieurs défis techniques majeurs. Premièrement, l'intégration d'un LLM dans un système de production nécessite de résoudre le problème des hallucinations : comment permettre au modèle de raisonner et de générer des plans cohérents tout en s'assurant que toutes les informations factuelles (prix, adresses, horaires) proviennent de sources vérifiées ? Deuxièmement, la personnalisation en temps réel exige un algorithme capable d'apprendre continuellement à partir de signaux comportementaux faibles (clics, temps de consultation, ajouts aux favoris) sans nécessiter de grandes quantités de données historiques. Troisièmement, le support multilingue (français, anglais, arabe) impose une architecture capable de détecter automatiquement la langue de l'utilisateur et de maintenir la cohérence sémantique across les trois langues dans la récupération d'informations et la génération de réponses.

0.3 Objectifs du Projet

Les objectifs de ce projet se déclinent en quatre axes principaux :

Objectif 1 : Agent Planificateur Conversationnel. Développer un agent basé sur LangGraph avec une machine à états explicite orchestrant le cycle de raisonnement ReAct (Reason + Act) en quatre nœuds : THINK (analyse de la situation), EXECUTE_TOOL (appel des 18 outils spécialisés), VALIDATE (vérification de la cohérence structurelle du plan), et RESPOND (formulation de la réponse finale). L'agent doit supporter le multilinguisme complet et la gestion de conversations longues via un système de mémoire avec résumé automatique.

Objectif 2 : Pipeline RAG Multilingue. Implémenter un système de génération augmentée par récupération (RAG) indexant six types d'entités touristiques (activités, hébergements, restaurants, attractions, locations, transferts) avec des embeddings denses BAAI/bge-m3 supportant le français, l'anglais et l'arabe dans un espace sémantique unifié. Le pipeline doit inclure une réindexation incrémentale automatique lors des mises à jour de la base de données.

Objectif 3 : Moteur de Recommandation LinUCB. Concevoir un bandit multi-bras contextuel utilisant un vecteur de caractéristiques de Kronecker de 360 dimensions ($\phi = x_{user}(9) \otimes x_{tags}(40)$) capturant les interactions entre signaux utilisateur (heure, mois, comportement, historique) et tags des annonces. Le moteur doit inclure un pipeline de classement post-LinUCB en 10 étapes et un mécanisme de récompenses delta par session empêchant l'inflation des signaux.

Objectif 4 : Intégration Système et Déploiement. Assurer l'intégration transparente des deux sous-systèmes avec la base de données PostgreSQL existante de la plateforme

Guidni, sans conflit avec le frontend Next.js/Prisma. Déployer le système sur un serveur unique avec des endpoints REST documentés, un tableau de bord administrateur pour la reconstruction manuelle de l'index RAG, et un monitoring des performances en temps réel.

0.4 Structure du Rapport

Ce rapport est organisé en huit chapitres suivant la méthodologie Scrum adoptée pour le développement du projet :

Chapitre 1 — Introduction Générale. Ce chapitre présente le contexte, la problématique, les objectifs et la méthodologie Scrum choisie pour mener à bien ce projet de fin d'études.

Chapitre 2 — Contexte Général et Étude Préliminaire. Ce chapitre décrit l'organisme d'accueil (ISITCOM), présente une étude approfondie de l'existant dans les domaines des chatbots et systèmes de recommandation, établit le cahier des charges fonctionnel et technique, et détaille la méthodologie Scrum avec ses rôles, artefacts et cérémonies.

Chapitre 3 — Lancement de Projet. Ce chapitre couvre l'analyse des besoins, l'architecture globale du système, la construction du Product Backlog avec priorisation MoSCoW, la planification des quatre sprints, et la définition des critères de qualité (Definition of Done).

Chapitre 4 — Sprint 1 : Conception et Implémentation de l'Agent Planificateur. Ce chapitre présente les livrables du premier sprint : la machine à états LangGraph, les 18 outils appelables, le pipeline RAG multilingue, et le système de mémoire conversationnelle.

Chapitre 5 — Sprint 2 : Implémentation du Moteur de Recommandation Hybrid LinUCB. Ce chapitre détaille le deuxième sprint consacré au moteur de recommandation : ingénierie des caractéristiques avec vecteur de Kronecker, algorithme LinUCB, pipeline de classement en 10 étapes, et tracker JavaScript pour la collecte des événements.

Chapitre 6 — Sprint 3 : Tests et Évaluation. Ce chapitre expose la stratégie de tests (unitaires, intégration, évaluation empirique), les scénarios de test pour l'agent planificateur et le moteur LinUCB, et les résultats obtenus en termes de précision, latence et récompense cumulée.

Chapitre 7 — Sprint 4 : Intégration et Déploiement. Ce chapitre final décrit l'intégration des deux sous-systèmes, le déploiement en environnement de production, les endpoints de santé et de monitoring, et la démonstration finale de la plateforme complète.

Conclusion Générale et Perspectives. La conclusion synthétise les contributions majeures du projet et propose cinq pistes de travaux futurs : fine-tuning d'un LLM local, remplacement du LinUCB linéaire par un bandit neuronal, infrastructure d'expérimentation A/B, extension multi-régions, et interface vocale multimodale.

0.5 Méthodologie Scrum

0.5.1 Pourquoi Scrum ?

Le choix de la méthodologie Scrum pour ce projet de fin d'études s'impose pour plusieurs raisons fondamentales liées à la nature même du domaine de l'intelligence artificielle et aux contraintes académiques du PFE.

Incertitude inhérente à l'IA. Les projets d'IA comportent une incertitude technique importante : les performances réelles des modèles LLM, la qualité des embeddings multilingues, et la convergence de l'algorithme LinUCB ne peuvent être prédites avec certitude avant l'expérimentation. Scrum permet d'embrasser cette incertitude en livrant des incréments fonctionnels à chaque sprint, permettant ainsi d'ajuster la trajectoire du projet en fonction des résultats observés plutôt que de suivre un plan rigide établi a priori.

Livraison itérative de valeur. La décomposition en quatre sprints de trois semaines chacun permet de livrer progressivement des fonctionnalités opérationnelles : l'agent planificateur (Sprint 1), le moteur de recommandation (Sprint 2), les tests complets (Sprint 3), et l'intégration-déploiement (Sprint 4). Chaque livraison peut être démontrée au superviseur et potentiellement utilisée par des testeurs précoces, générant des retours exploitables pour les sprints suivants.

Adéquation avec le calendrier académique. Le format itératif de Scrum s'aligne naturellement avec le rythme universitaire : chaque sprint correspond approximativement à une période de vacances intermédiaire, permettant des points d'étape formels avec l'encadrant. La documentation requise par Scrum (Product Backlog, Sprint Backlog, Definition of Done) constitue également une ossature idéale pour la rédaction du rapport de PFE.

Flexibilité face aux imprévus. Un projet de fin d'études comporte inévitablement des aléas : difficultés techniques imprévues, évolution des exigences, contraintes de temps. Scrum offre un cadre structuré pour gérer ces imprévus : si une user story ne peut être terminée dans un sprint, elle est replanifiée au sprint suivant sans compromettre l'ensemble du projet.



Figure 1: Cycle du processus Scrum adapté au projet Guidni

La figure 1 illustre le cycle Scrum adopté : chaque sprint commence par une phase de **Sprint Planning** où les user stories sont sélectionnées depuis le Product Backlog et décomposées en tâches techniques. Le développement suit ensuite avec des **Daily Scrums** quotidiens (auto-évaluation individuelle) pour synchroniser l'avancement. Le sprint se termine par une **Sprint Review** avec le superviseur pour démontrer les fonctionnalités implémentées, suivie d'une **Sprint Retrospective** pour identifier les améliorations de processus applicables au sprint suivant.

0.5.2 Rôles Scrum

Le tableau 2 définit les rôles Scrum tels qu'attribués dans le cadre de ce projet de fin d'études.

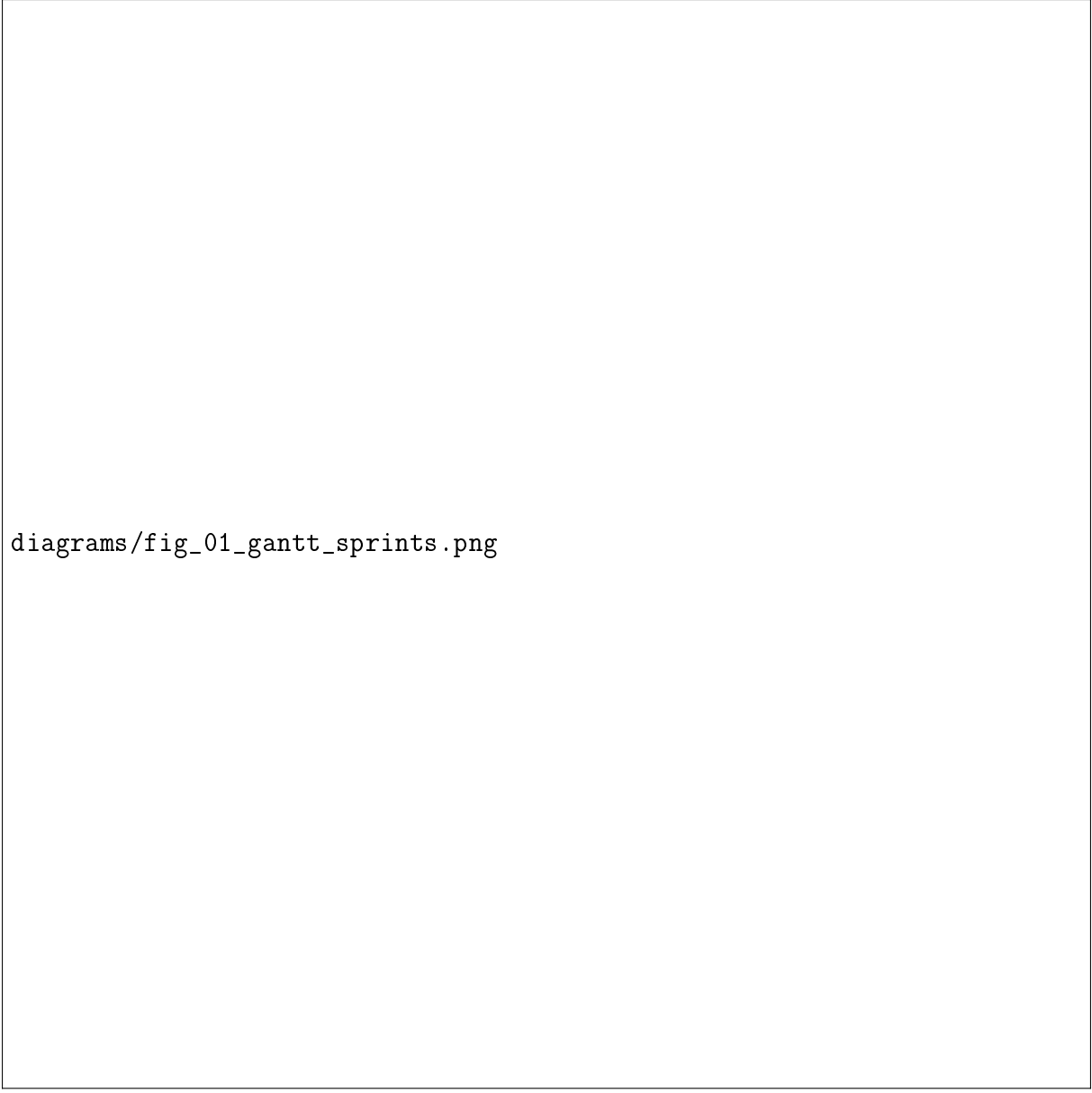
Table 2: Rôles Scrum dans le projet Guidni

Rôle		Titulaire		Responsabilités
Product (PO)	Owner	Superviseur académique		Définir la vision produit, prioriser le Product Backlog, valider les livrables en Sprint Review, arbitrer les compromis fonctionnels.
Scrum (SM)	Master	Étudiant (Ahmed Addali)		Faciliter les cérémonies Scrum, lever les obstacles, garantir le respect du cadre Scrum, assurer la vélocité de l'équipe.
Équipe Développement	de	Étudiant (Ahmed Addali)		Concevoir, implémenter et tester les fonctionnalités, estimer les charges, s'engager sur le Sprint Backlog, produire des incréments potentiellement livrables.

Dans le contexte spécifique d'un PFE individuel, l'étudiant cumule les rôles de Scrum Master et de développeur unique, tandis que le superviseur académique assume le rôle de Product Owner. Cette configuration simplifiée conserve néanmoins l'essentiel des bénéfices de Scrum : la séparation des responsabilités entre la définition du *quoi* (PO) et la réalisation du *comment* (équipe), et l'amélioration continue via les rétrospectives.

0.6 Planification des Sprints

Le projet Guidni est découpé en quatre sprints de trois semaines chacun, couvrant la période totale de douze semaines allouée au développement. La figure 2 présente le diagramme de Gantt global du projet.



diagrams/fig_01_gantt_sprints.png

Figure 2: Diagramme de Gantt des quatre sprints du projet Guidni

Le tableau 3 résume les objectifs, périodes et livrables clés de chaque sprint.

Table 3: Planification des sprints

Sprint	Objectif Principal	Période	Livrables
Sprint 1	Agent planificateur IA avec LangGraph, RAG et mémoire	Semaines 1–3	Graphe 4 nœuds, 18 outils, pipeline RAG bge-m3, mémoire conversationnelle, endpoint /chat
Sprint 2	Moteur de recommandation Hybrid LinUCB	Semaines 4–6	Ingénierie des caractéristiques (360D), algorithme LinUCB, ranker 10 étapes, tracker JS, jobs nocturnes
Sprint 3	Tests complets et évaluation empirique	Semaines 7–9	Tests unitaires (context, linucb, ranker), tests d'intégration (/chat, /plan, /events), évaluation manuelle, métriques de performance
Sprint 4	Intégration, déploiement et démo finale	Semaines 10–12	Intégration des deux sous-systèmes, endpoints de santé, dashboard admin, documentation API, démonstration complète

Chaque sprint suit le même déroulement type :

- **Jour 1** : Sprint Planning (2–4 heures) – Sélection des user stories, estimation T-Shirt (XS à XL), création du Sprint Backlog.
- **Jours 2–18** : Développement avec Daily Scrum quotidien (15 minutes d'auto-évaluation) et avancement selon les tâches définies.
- **Jour 19** : Sprint Review (1–2 heures avec le superviseur) – Démonstration des fonctionnalités implémentées, validation des critères d'acceptation.
- **Jour 20** : Sprint Retrospective (1 heure) – Analyse des points forts, faiblesses et actions d'amélioration pour le sprint suivant.

La vélocité cible est de 8 à 10 points de story par sprint, basée sur une échelle de complexité relative où une story XS vaut 1 point, S vaut 2 points, M vaut 4 points, L vaut 8 points et XL vaut 13 points. Cette granularité permet un suivi précis de la charge réelle versus la charge estimée.

0.7 Conclusion de l'Introduction

Ce chapitre introductif a posé les fondations du projet Guidni en présentant le contexte touristique de Djerba, la problématique centrale combinant fiabilité des LLM et personnalisation en temps réel, les quatre objectifs techniques majeurs, et la méthodologie Scrum structurée en quatre sprints thématiques. Les chapitres suivants détailleront successivement le cadre général et l'étude préliminaire (Chapitre 2), le lancement de projet avec l'architecture et le Product Backlog (Chapitre 3), puis les réalisations concrètes de chaque sprint (Chapitres 4 à 7).

Chapter 1

Contexte Général et Étude Préliminaire

1.1 Introduction

Ce chapitre établit les fondements théoriques et bibliographiques sur lesquels repose la conception du système Guidni. Il couvre cinq domaines majeurs : les agents conversationnels basés sur les grands modèles de langage (LLM), la génération augmentée par récupération (RAG), les systèmes de recommandation classiques, les bandits multi-bras contextuels et en particulier l'algorithme LinUCB, ainsi qu'une revue comparative des travaux connexes. Pour chaque domaine, la présentation progresse des concepts généraux vers les choix spécifiques retenus pour le projet Guidni.

L'objectif de ce chapitre est double : d'une part, fournir au lecteur les clés de compréhension nécessaires pour appréhender les décisions architecturales présentées dans les chapitres suivants ; d'autre part, situer la contribution de ce projet par rapport à l'état actuel de la recherche et des systèmes industriels dans les domaines de la planification touristique intelligente et de la recommandation personnalisée en temps réel.

1.2 Agents LLM et planification conversationnelle

1.2.1 Les grands modèles de langage (LLMs)

L'architecture Transformer, introduite par Vaswani et al. en 2017 [7], constitue le socle de tous les grands modèles de langage modernes. Son innovation centrale réside dans le mécanisme d'attention multi-têtes (*multi-head attention*), qui permet au modèle de pondérer l'importance relative de chaque token d'entrée par rapport à tous les autres, quelle que soit leur distance dans la séquence. Contrairement aux architectures récurrentes qui traitent les tokens séquentiellement, le Transformer traite l'ensemble de la séquence en parallèle, ce qui permet un entraînement massivement distribué sur des corpus de texte de plusieurs téraoctets.

L'évolution des LLMs a connu une étape décisive avec l'introduction de l'ajustement par instructions (*instruction tuning*) et de l'apprentissage par renforcement à partir de retours humains (RLHF — *Reinforcement Learning from Human Feedback*). Ces techniques transforment un modèle de prédiction de tokens en un assistant capable de comprendre et d'exécuter des instructions formulées en langage naturel. Le modèle n'apprend plus seulement à compléter une séquence statistiquement probable, mais à produire des réponses utiles, structurées et conformes aux attentes de l'utilisateur.

L'émergence de l'appel d'outils (*tool use* ou *function calling*) constitue une avancée fondamentale pour la construction d'agents autonomes. Dans ce paradigme, le LLM n'essaie pas de générer la réponse directement depuis sa mémoire paramétrique ; il émet plutôt un objet JSON structuré décrivant quelle fonction externe appeler et avec quels arguments. Par exemple, au lieu d'inventer les conditions météorologiques à Djerba, le modèle peut appeler explicitement `get_weather(location="Djerba", date="2025-01-15")` et recevoir en retour des données réelles de l'API Open-Meteo.

Cette capacité d'appel d'outils est fondamentale pour la construction d'agents fiables qui accèdent à des données en temps réel, des bases de données et des APIs externes. Un LLM sans outils est contraint de répondre uniquement à partir de ses données d'entraînement, ce qui entraîne inévitablement des hallucinations factuelles — des informations plausibles mais fausses. Pour un planificateur de voyage, ce problème est particulièrement critique : recommander un hôtel inexistant, un prix erroné ou une activité fermée détruit immédiatement la confiance de l'utilisateur. L'accès aux outils transforme le LLM d'un générateur de texte en un orchestrateur de données vérifiées.

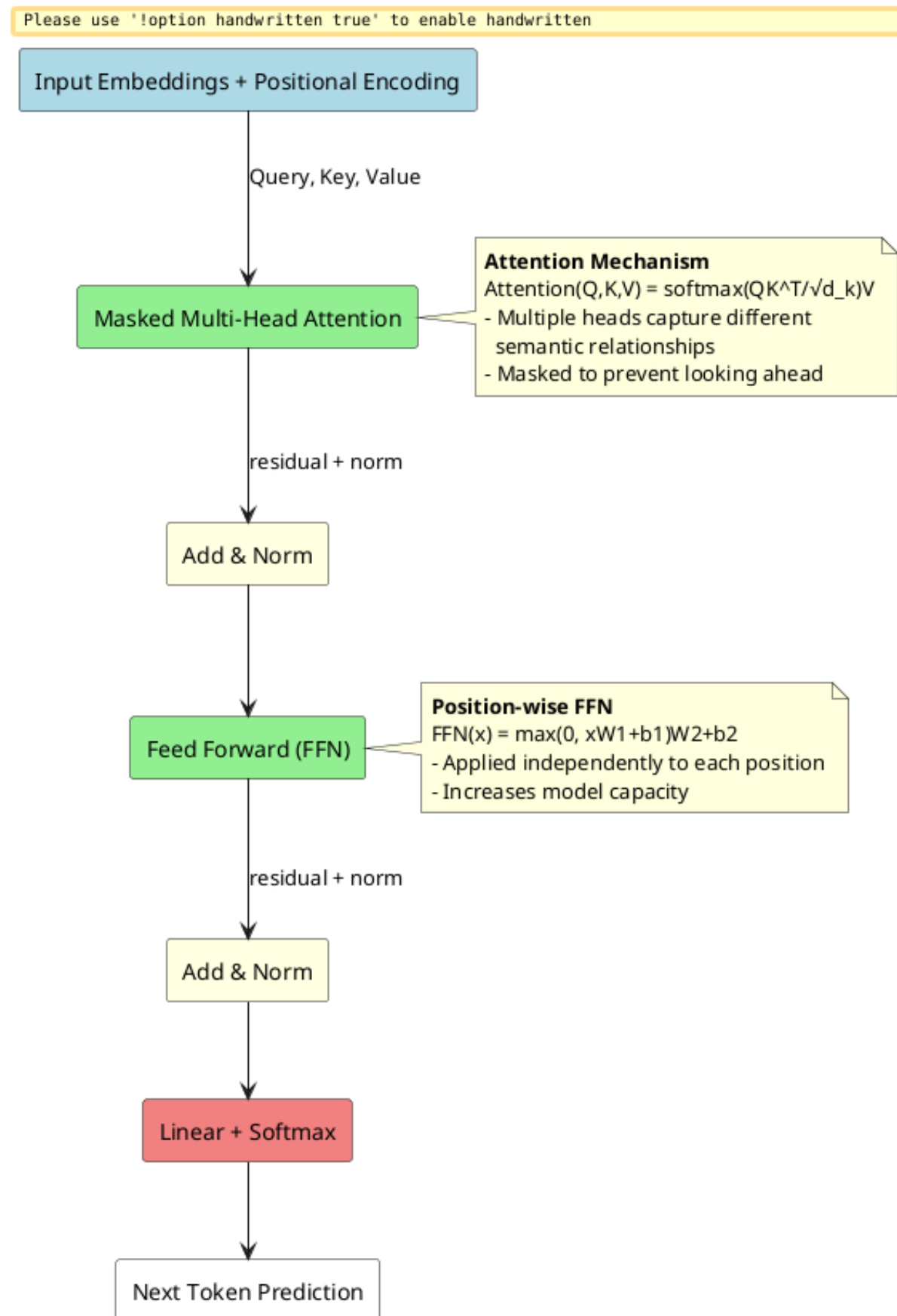


Figure 1.1: Architecture simplifiée d'un décodeur Transformer avec mécanisme d'attention

1.2.2 Frameworks agentiques : ReAct et LangGraph

Le patron ReAct (*Reason + Act*), proposé par Yao et al. [8], formalise le cycle de raisonnement agentique : à chaque étape, le modèle raisonne sur la situation courante, décide d'une action à entreprendre (typiquement un appel d'outil), observe le résultat retourné, puis raisonne à nouveau sur la base de cette nouvelle information. Ce cycle simple mais puissant permet de résoudre des tâches complexes en plusieurs étapes, où chaque étape affine la compréhension du problème et rapproche l'agent de la solution finale.

LangGraph, développé par LangChain Inc., étend le concept ReAct avec une machine à états typée explicite. L'état de l'agent est défini par un `TypedDict` Python appelé `AgentState` qui accumule les messages via l'opérateur `operator.add`, et suit l'ensemble des données collectées : profil utilisateur, activités trouvées, météo, hébergements, restaurants, contexte RAG, plan en cours de construction, étapes de raisonnement, outils utilisés, compteur d'itérations, et indicateurs de décision (besoin d'information supplémentaire, plan prêt pour validation). Des fonctions de routage conditionnel déterminent quel nœud s'exécute ensuite en fonction de l'état courant, et le mode streaming permet l'affichage en temps réel des étapes de réflexion dans l'interface utilisateur.

Le graphe spécifique construit pour l'agent Guidni comporte quatre nœuds : **THINK** (le LLM analyse la situation et décide de l'action), **EXECUTE_TOOL** (l'outil appelé s'exécute et retourne un résultat), **VALIDATE** (vérification de la cohérence d'un plan généré), et **RESPOND** (formulation de la réponse finale). La topologie du graphe est la suivante : le point d'entrée est THINK ; après THINK, la fonction `_should_continue()` route vers EXECUTE_TOOL si le LLM a émis un appel d'outil, vers RESPOND si l'agent a besoin d'informations supplémentaires de l'utilisateur, vers VALIDATE si un plan structuré est prêt (`is_plan_ready=True`), ou vers RESPOND par défaut (réponse textuelle ou sortie forcée après `MAX_AGENT_ITERATIONS=15`). Après EXECUTE_TOOL, la fonction `_after_tool_execution()` renvoie à THINK pour réévaluation, sauf si l'outil `ask_user` a été utilisé, auquel cas l'agent passe directement à RESPOND. Après VALIDATE, la fonction `_after_validation()` renvoie à THINK si des erreurs ont été détectées et qu'il reste des itérations, ou à RESPOND si la validation a réussi.

LangGraph a été préféré aux alternatives plus simples (chaînes LangChain Expression Language, CrewAI) pour plusieurs raisons : l'état explicite facilite le débogage en rendant visible l'ensemble des données accumulées à chaque étape, la boucle de validation garantit la cohérence des plans générés avant leur présentation à l'utilisateur, la sortie forcée à 15 itérations empêche les boucles infinies en cas de raisonnement circulaire, et le support du streaming permet d'afficher les étapes de réflexion (*thinking steps*) de manière progressive dans l'interface frontend, offrant une expérience utilisateur transparente.

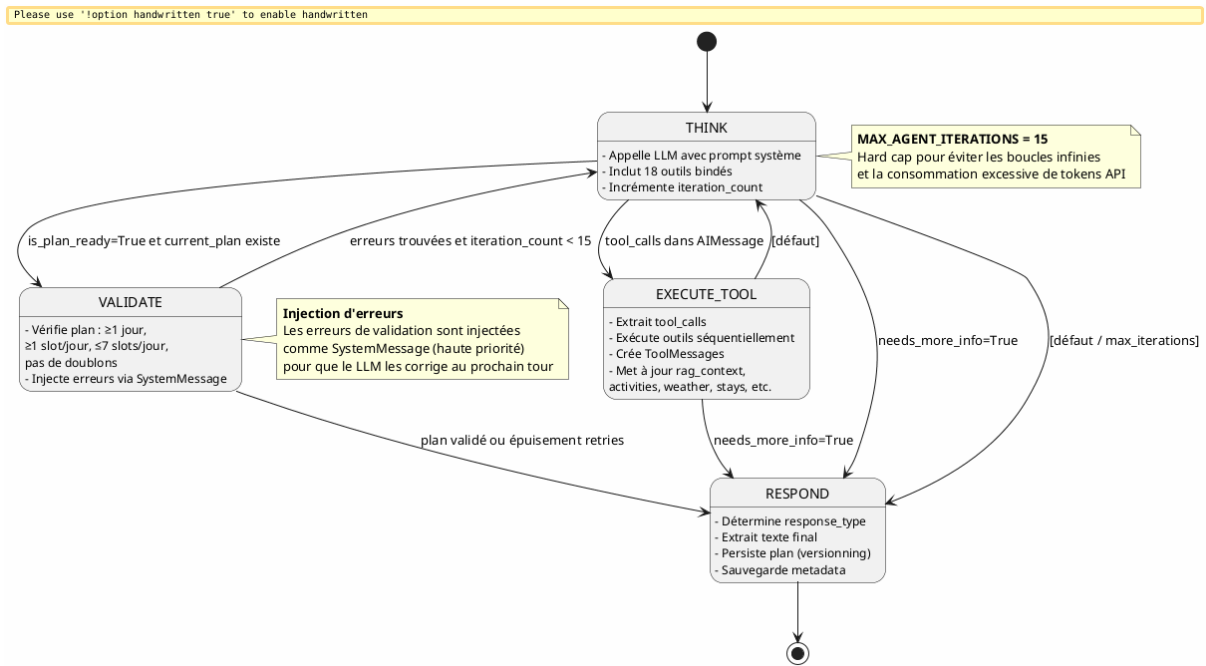


Figure 1.2: Graphe LangGraph de l'agent planificateur Guidni

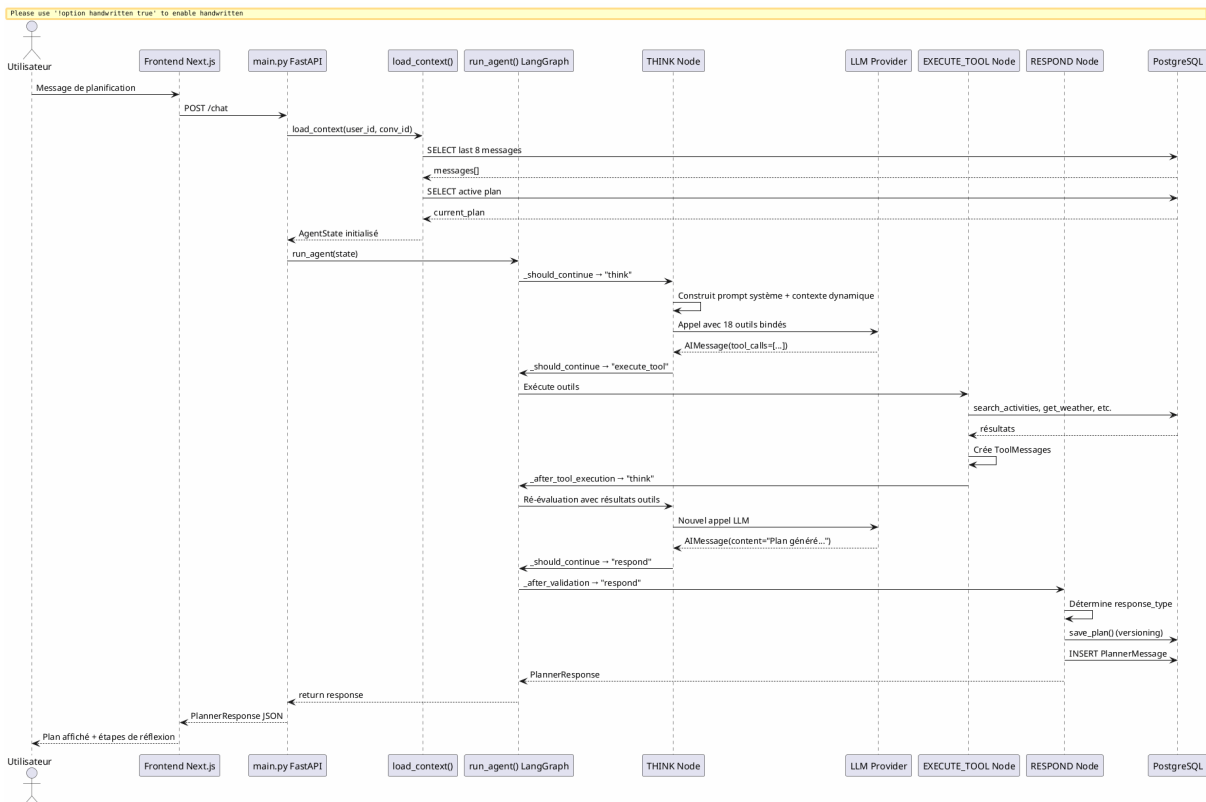


Figure 1.3: Diagramme de séquence d'un cycle complet de l'agent

1.2.3 Abstraction multi-fournisseurs LLM

L'abstraction de l'accès aux fournisseurs LLM constitue un choix architectural essentiel pour tout système agentique destiné à un usage en production. Les coûts varient considérablement d'un fournisseur à l'autre — Ollama est gratuit mais plus lent car il s'exécute localement, Groq offre une inférence ultra-rapide mais impose des quotas d'utilisation, Gemini propose la fenêtre de contexte la plus large (jusqu'à un million de tokens) mais engendre des coûts API. La latence diffère d'un facteur de dix entre un modèle local à 7 milliards de paramètres et un modèle cloud optimisé. De plus, certains environnements de déploiement exigent un fonctionnement entièrement hors ligne, rendant indispensable la possibilité de basculer entre fournisseurs sans modification du code applicatif.

Le système Guidni implémente un patron fabrique (*factory pattern*) dans le module `provider.py`. La fonction centrale `get_llm()` accepte quatre paramètres : `temperature` (contrôle de la créativité), `json_mode` (forçage du format JSON en sortie), `provider` (fournisseur à utiliser) et `model` (nom du modèle). La fonction `parse_model_id()` décompose un identifiant de la forme "gemini/gemini-2.5-flash" en un couple (fournisseur, nom du modèle). Chaque fournisseur dispose de son propre fichier client : `ollama_client.py` instancie `ChatOllama`, `groq_client.py` instancie `ChatGroq`, `gemini_client.py` instancie `ChatGoogleGenerativeAI`, et `lightning_client.py` instancie le client Lightning AI. Le code appelant ne change jamais lorsque le fournisseur change.

Le schéma `ChatRequest` accepte un champ optionnel `model` permettant au frontend de sélectionner dynamiquement un modèle spécifique pour chaque requête, par exemple "groq/llama-3.3-70b-versatile" ou "ollama/qwen2.5:7b", sans redémarrage du serveur. Cette flexibilité permet aux utilisateurs avancés de comparer les performances des différents modèles sur la même requête de planification.

Table 1.1: Comparaison des fournisseurs LLM intégrés dans Guidni

Fournisseur	Modèle	Type	Contexte	Latence	Points forts	Contraintes
Ollama	qwen2.5:7b	Local	8 192 tok.	2–5 s	Gratuit, confidentialité, hors-ligne	Lent, RAM requise
Groq	llama-3.3-70b-versatile	Cloud	128k tok.	0.5–1 s	Très rapide, grande fenêtre	Quota réseau
Gemini	gemini-2.5-flash	Cloud	1M tok.	1–2 s	Multimodal, contexte énorme	Coût API, format list
Lightning	openai/gpt-5	Cloud	128k tok.	1–3 s	Dernière génération	Réseau, coût

La section suivante aborde le mécanisme de génération augmentée par récupération (RAG), qui permet à l'agent d'accéder à des données locales vérifiées plutôt que de s'appuyer sur la mémoire paramétrique du LLM.

1.3 Génération augmentée par récupération (RAG)

1.3.1 Motivation et principe

Le problème fondamental que résout la technique RAG est l’incapacité des LLMs à connaître des informations spécifiques à un domaine local. Les modèles de langage sont entraînés sur des corpus internet généraux et ne peuvent pas connaître les caractéristiques précises d’un hôtel à Djerba, le menu d’un restaurant local, ou le prix actuel d’une activité de plongée. Interroger un LLM sur “l’Hôtel Djerba Plaza” ou “le Restaurant Haroun” sans contexte produira des descriptions hallucinées — des informations plausibles mais factuellement fausses. Pour un planificateur de voyage, cette situation est catastrophique car les utilisateurs font confiance aux adresses, prix et horaires recommandés.

Le principe RAG (*Retrieval-Augmented Generation*), formalisé par Lewis et al. [4], repose sur une idée simple mais efficace : au lieu de s’appuyer sur la mémoire paramétrique du LLM, le système récupère des documents pertinents depuis une base de connaissances structurée et les injecte comme contexte dans le prompt. Le LLM raisonne alors sur des données réelles et vérifiées plutôt que d’inventer des informations. Cette approche combine la capacité de raisonnement et de génération fluide du LLM avec la précision factuelle d’une base de données interrogée en temps réel.

L’architecture RAG se décompose en deux phases distinctes. La **phase d’indexation**, exécutée au démarrage de l’application, convertit l’ensemble des enregistrements de la base de données en documents textuels, calcule les vecteurs d’embedding pour chaque document, et construit un index vectoriel interrogeable. La **phase de requête**, exécutée à chaque demande utilisateur, prend la requête en entrée, trouve les documents sémantiquement similaires dans l’index, et retourne les résultats les plus pertinents avec leurs scores de similarité.

L’implémentation Guidni indexe six types d’entités : les activités (**Activity**), les hébergements (**Stay**), les restaurants (**Restaurant**), les attractions (**Attraction**), les locations (**Rental**) et les transferts (**Transfer**). Chaque entité est convertie en un document textuel riche incluant l’ensemble des champs pertinents — titre, description, catégorie, région, ville, prix, durée, horaires d’ouverture, équipements — accompagné d’un dictionnaire de métadonnées structurées utilisé pour le filtrage. Le système supporte deux moteurs vectoriels interchangeables : Qdrant (index vectoriel dédié) et pgvector (extension PostgreSQL), sélectionnés via la variable de configuration `VECTOR_ENGINE`.

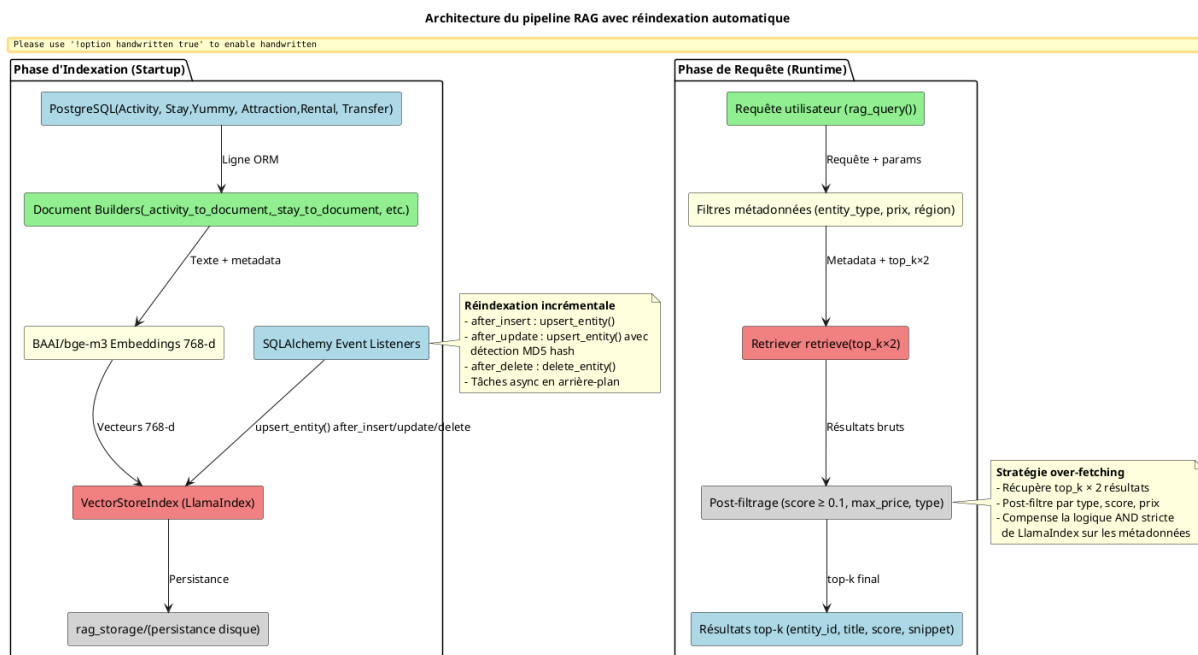


Figure 1.4: Architecture du pipeline RAG avec réindexation automatique

1.3.2 Modèle d'embedding BAAI/bge-m3

Le modèle BAAI/bge-m3, publié par la Beijing Academy of Artificial Intelligence, est un modèle d'embedding multilingue open source supportant plus de 100 langues, dont le français, l'anglais et l'arabe. Il produit des vecteurs denses de 1024 dimensions qui capturent la signification sémantique des textes indépendamment de la langue : une requête en français sur “plage familiale” retrouvera des documents mentionnant “family beach” en anglais ou des termes équivalents en arabe. Cette propriété multilingue est essentielle pour le marché touristique de Djerba, intrinsèquement trilingue.

La configuration retenue dans le module `settings.py` définit les paramètres suivants : `chunk_size=512` tokens pour le découpage des documents, `chunk_overlap=50` tokens pour assurer la continuité sémantique entre les fragments, exécution sur CPU (`device="cpu"`) sans nécessité de GPU, et `trust_remote_code=True` pour le chargeur HuggingFace. Le modèle est initialisé une seule fois au démarrage de l'application via la fonction `initialize_rag_settings()` et stocké comme paramètre global `LlamaIndex`. Le LLM intégré de `LlamaIndex` est explicitement désactivé (`LlamaSettings.llm = None`) car tout le raisonnement est délégué à l'agent `LangGraph`.

Le choix de bge-m3 par rapport aux alternatives a été motivé par plusieurs facteurs : il est gratuit (aucune clé API nécessaire), respectueux de la vie privée (exécution entièrement locale), multilingue de manière native — ce qui est critique pour le marché touristique FR/EN/AR — et surpasse les modèles plus petits sur les benchmarks de récupération pour du contenu multi-domaine. L'alternative OpenAI `text-embedding-3` offre des performances similaires mais introduit une dépendance réseau, un coût par requête et une transmission

des données hors du serveur.

1.3.3 Réindexation automatique via SQLAlchemy Events

Le problème résolu par la réindexation automatique est le suivant : si un propriétaire d’hôtel met à jour le prix de son annonce ou si une nouvelle activité est ajoutée à la base de données, l’index vectoriel devient obsolète et l’agent recommandera des informations périmées. Une reconstruction manuelle de l’index nécessiterait un redémarrage du serveur, ce qui est inacceptable pour un système en production.

La solution implémentée dans le module `db_events.py` repose sur les écouteurs d’événements SQLAlchemy. Au démarrage de l’application, la fonction `register_db_events()` attache des écouteurs sur les événements `after_insert`, `after_update` et `after_delete` des six modèles suivis : `Activity`, `Stay`, `Restaurant`, `Attraction`, `Rental` et `Transfer`. Lorsqu’un de ces événements se produit, une tâche asynchrone en arrière-plan est créée via `loop.create_task()` pour appeler `upsert_entity()` (insertion ou mise à jour) ou `delete_entity()` (suppression) sur l’entité concernée.

Les bénéfices de cette approche sont triples : l’index reste toujours à jour sans aucune intervention manuelle, la réindexation est non bloquante car elle s’exécute en arrière-plan via des tâches asynchrones, et le traitement est ciblé — seule l’entité modifiée est réindexée, et non l’ensemble du corpus. Cette architecture événementielle garantit que l’agent planificateur travaille systématiquement sur des données actualisées.

La section suivante élargit la perspective aux systèmes de recommandation, domaine théorique qui sous-tend le second sous-système du projet Guidni.

1.4 Systèmes de recommandation : vue d’ensemble

Le **filtrage collaboratif** repose sur la matrice d’interactions utilisateur-item. En identifiant des utilisateurs aux comportements similaires, le système recommande à un utilisateur les items appréciés par ses “voisins” comportementaux. Les techniques de factorisation matricielle (SVD, ALS) permettent de réduire la dimensionnalité de cette matrice creuse. Cette approche excelle dans la découverte d’items inattendus, mais souffre du problème de démarrage à froid (*cold start*) : un nouvel utilisateur ou un nouvel item n’ayant aucune interaction ne peut être recommandé, et le système nécessite un historique d’interactions conséquent pour formuler des recommandations pertinentes.

Le **filtrage basé sur le contenu** représente chaque item par un vecteur de caractéristiques (catégorie, tags, prix, localisation) et recommande les items dont le profil est similaire à ceux déjà appréciés par l’utilisateur. Cette approche fonctionne dès la première interaction car elle ne dépend pas d’autres utilisateurs. Cependant, elle crée des bulles de filtres (*filter bubbles*) qui enferment l’utilisateur dans un ensemble restreint de catégories déjà

explorées, éliminant toute sérendipité — la découverte fortuite d’items intéressants en dehors du profil habituel.

Les **systèmes hybrides hors ligne** combinent les deux approches précédentes et sont entraînés sur des données historiques par lots (*batch training*). Ils offrent une meilleure couverture du catalogue et atténuent partiellement le démarrage à froid. Cependant, ils sont incapables de réagir aux signaux émis par l’utilisateur en temps réel durant sa session : le temps passé sur une page, la profondeur de défilement ou l’ajout à une liste de souhaits ne sont exploités qu’au prochain cycle de réentraînement, qui peut survenir des heures ou des jours plus tard. De plus, la dérive conceptuelle (*concept drift*) — l’évolution des préférences au fil des saisons — dégrade les performances entre deux cycles.

L’**apprentissage en ligne par bandits contextuels** apprend en temps réel à partir de chaque interaction. À chaque requête, le système observe un vecteur de contexte (heure, saison, appareil, comportement de session) et sélectionne les items qui maximisent la récompense attendue tout en explorant des alternatives moins connues. Cette approche équilibre naturellement l’exploration de nouveaux items avec l’exploitation des items connus, s’adapte automatiquement aux tendances saisonnières, et ne nécessite aucun réentraînement par lots. C’est cette approche qui a été retenue pour le moteur de recommandation Guidni.

Table 1.2: Comparaison des approches de recommandation

Approche	Cold Start	Dérive temp.	Données	Complexité	Guidni ?
Filtrage collaboratif	Problématique	Mauvaise	Grand historique	Moyenne	Non
Basé contenu	Acceptable	Mauvaise	Attributs items	Faible	Partiel
Hybride offline	Partiel	Mauvaise	Les deux	Élevée	Non
LinUCB contextuel	Bonus exploration	Reset decay	λ - Contexte temps réel	Faible– Moy.	Oui

1.5 Bandits multi-bras contextuels et algorithme LinUCB

1.5.1 Formulation du problème MAB

Le problème des bandits multi-bras (*Multi-Armed Bandit*, MAB) tire son nom de la métaphore d’un joueur face à une rangée de machines à sous : chaque machine (appelée “bras”) offre une distribution de gains différente et inconnue. À chaque tour, le joueur doit choisir une machine et observe la récompense obtenue. Son objectif est de maximiser la récompense cumulée au fil du temps. Le défi fondamental réside dans le dilemme exploration-exploitation : faut-il actionner les bras dont on sait qu’ils rapportent (exploitation)

ou essayer des bras inconnus qui pourraient rapporter davantage (exploration) ?

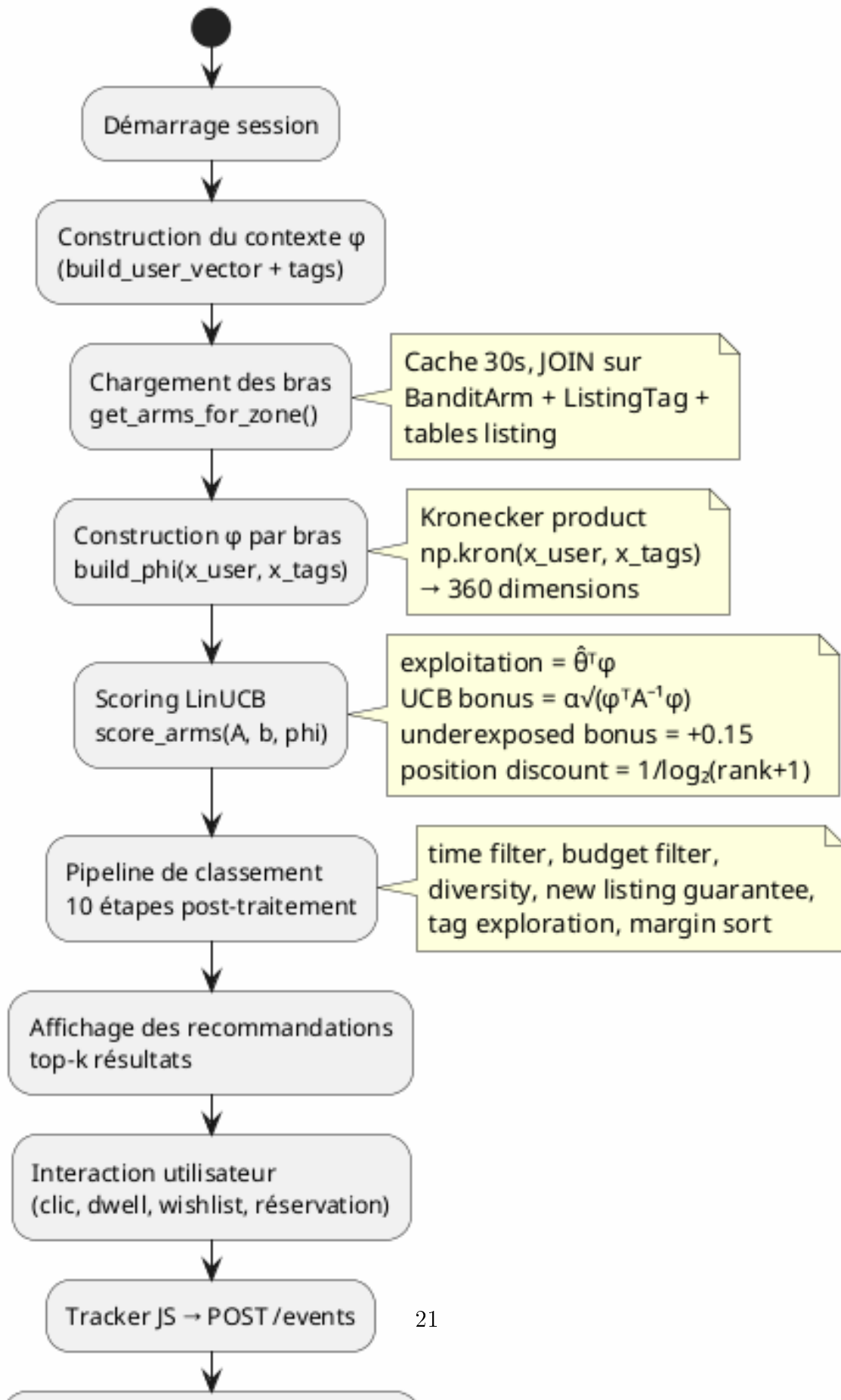
Formellement, à chaque pas de temps t , un agent choisit un bras a_t parmi K bras disponibles et reçoit une récompense r_t tirée d'une distribution inconnue. L'objectif est de minimiser le regret cumulé, défini comme la différence entre la récompense totale obtenue et la récompense maximale qu'aurait obtenue un oracle ayant toujours choisi le meilleur bras : $R_T = \sum_{t=1}^T r^* - r_t$, où r^* est la récompense espérée du bras optimal.

Dans la variante **contextuelle** du problème, l'agent observe un vecteur de contexte ϕ_t avant de choisir un bras. La récompense espérée dépend à la fois du bras choisi et du contexte observé. Cette extension est cruciale pour la recommandation touristique : la meilleure recommandation dépend du contexte — une famille à 10h en été (tags : `beach_access`, `pool`, `family`) n'a pas les mêmes attentes qu'un voyageur solo à 20h en hiver (tags : `romantic`, `bar`, `activity_indoor`).

Dans le système Guidni, neuf types d'événements sont suivis avec des valeurs de récompense allant de -0.2 (`quick_exit` : l'utilisateur a quitté la page en moins de 5 secondes) à 1.0 (`reservation` : signal de valeur maximale), en passant par 0.3 (`click`), $0.5 / 0.7$ (`dwell_30s / dwell_60s`), $0.4 / 0.6$ (`gallery_swipe / scroll_reviews`), et 0.85 (`wishlist`). Un mécanisme de delta par session empêche la même annonce d'accumuler des récompenses multiples lors d'une même visite : seule la différence entre la récompense courante et le maximum précédemment émis pour cette annonce est comptabilisée.

Cycle complet du bandit LinUCB dans le système Guidni

Please use '!option handwritten true' to enable handwritten



1.5.2 L'algorithme LinUCB et la formule de score

L'algorithme LinUCB repose sur l'hypothèse que la récompense espérée est une fonction linéaire du vecteur de contexte : $\mathbb{E}[r|\phi] = \theta^\top \phi$, où θ est un vecteur de paramètres inconnu que le système doit estimer à partir des récompenses observées. Cette hypothèse de linéarité est raisonnable pour un espace de caractéristiques bien conçu, car le produit de Kronecker utilisé pour construire ϕ capture déjà les interactions non linéaires entre caractéristiques utilisateur et tags d'annonces.

La règle de mise à jour LinUCB est la suivante : après avoir observé une récompense r pour un contexte ϕ , le système met à jour les matrices globales selon $A \leftarrow A + \phi\phi^\top$ et $b \leftarrow b + r \cdot \phi$. L'estimateur du paramètre θ est alors $\hat{\theta} = A^{-1}b$, calculé dans le code via `np.linalg.solve(A, b)` — jamais par inversion matricielle directe (`np.linalg.inv`) pour des raisons de stabilité numérique. En cas de matrice singulière, un repli vers `np.linalg.lstsq` est prévu.

La formule de score UCB combine exploitation et exploration :

$$\text{score} = \underbrace{\hat{\theta}^\top \phi}_{\text{exploitation}} + \underbrace{\alpha \sqrt{\phi^\top A^{-1} \phi}}_{\text{bonus UCB (exploration)}} \quad (1.1)$$

Le premier terme est la récompense prédite (exploitation) ; le second est le bonus d'exploration, proportionnel à l'incertitude — élevé lorsque la matrice A a peu d'observations dans la direction de ϕ . Le paramètre $\alpha = 0.3$ contrôle l'équilibre exploration-exploitation ; une valeur plus élevée favorise l'exploration de bras moins connus.

Les annonces ayant moins de 50 impressions reçoivent un bonus supplémentaire de +0.15 ajouté au score UCB, assurant que les nouvelles annonces sont présentées à un nombre minimum d'utilisateurs avant que le système puisse correctement estimer leur valeur. Ce bonus d'exposition (`UNDEREXPOSED_BONUS`) est contrôlé par le drapeau `is_underexposed` défini par la fonction `apply_popularity_correction()`.

La santé numérique de la matrice A est surveillée par la fonction `is_matrix_healthy()`, qui vérifie que A est définie positive (toutes les valeurs propres sont strictement positives) et que toutes les valeurs sont finies. Tous les 90 jours, une réinitialisation partielle est appliquée : $A_{\text{new}} = 0.7 \times A + 0.3 \times I$ et $b_{\text{new}} = 0.7 \times b$, avec $\lambda_{\text{decay}} = 0.7$. Cette opération oublie progressivement les données anciennes tout en conservant la structure apprise, adressant la dérive conceptuelle lorsque les préférences touristiques évoluent au fil des saisons.

1.5.3 Ingénierie des caractéristiques

Le vecteur utilisateur x_{user} est de dimension 9. L'heure et le mois sont encodés avec des fonctions sinus et cosinus pour préserver leur nature cyclique : minuit et 23h doivent

être proches, décembre et janvier doivent être voisins. La profondeur de défilement (`scroll_depth`) est écrêtée à $[0, 1]$. Le temps de consultation est normalisé par une transformation logarithmique : $\log_{1p}(\text{dwell})/\log_{1p}(300)$, écrêté à $[0, 1]$. L’affinité de prix est le prix moyen de réservation de l’utilisateur divisé par le maximum de la plateforme (2000 TND). Le score d’affinité de tag maximal provient du profil utilisateur. Le type de voyage est encodé numériquement : famille= 1.0, couple= 0.67, solo= 0.33, inconnu= 0.0.

Le vocabulaire de tags comprend **40 tags** en ordre alphabétique fixe. L’ordre ne doit jamais changer après le déploiement car toute modification invaliderait la matrice A . Les tags couvrent les équipements (`beach_access`, `pool`, `spa`, `wifi`, `parking`, `gym`, `hammam`, `air_conditioning`), les types d’activité (`adventure`, `cultural`, `nature`, `activity_outdoor`, `activity_indoor`), les caractéristiques (`budget`, `luxury`, `romantic`, `family`, `business`, `solo`), la restauration (`fine_dining`, `casual_dining`, `street_food`, `bar`), les types d’hébergement (`hotel`, `hostel`, `riad`, `villa`), et la saisonnalité (`best_morning`, `best_evening`, `best_summer`, `best_winter`, `all_seasons`).

Le produit de Kronecker $\phi = \text{np.kron}(x_{\text{user}}, x_{\text{tags}})$ combine le vecteur utilisateur de dimension 9 avec un vecteur binaire de tags de longueur 40 (1 si l’annonce possède le tag, 0 sinon). Ce produit crée toutes les combinaisons par paires entre caractéristiques utilisateur et tags d’annonces, produisant un vecteur ϕ de longueur $9 \times 40 = 360$ qui capture les effets d’interaction — par exemple, “utilisateur famille + tag pool” constitue une dimension unique du vecteur.

Un mécanisme de sécurité par hachage du vocabulaire garantit la cohérence entre le vocabulaire et les matrices. Au démarrage, un hash SHA-256 du vocabulaire de tags trié est calculé par la fonction `compute_vocab_hash()` et comparé au hash stocké dans la table `SystemConfig`. Si les hash divergent, le système refuse de démarrer, empêchant une corruption silencieuse de la matrice A .

Table 1.3: Les 9 composantes du vecteur utilisateur x_{user}

Idx	Caractéristique	Formule	Exemple	<i>Exemple :</i>
0	sin(heure)	$\sin(2\pi \times h/24)$	0.866	
1	cos(heure)	$\cos(2\pi \times h/24)$	0.500	
2	sin(mois)	$\sin(2\pi \times m/12)$	0.866	
3	cos(mois)	$\cos(2\pi \times m/12)$	-0.500	
4	Profondeur défilement	$\text{clip}(\text{scroll}, 0, 1)$	0.70	
5	Temps consultation	$\text{clip}(\log_e(1 + \text{dwell})/\log_e(301), 0, 1)$	0.72	
6	Affinité prix	$\text{avg_price}/2000$	0.075	
7	Affinité tag max	$\text{max}(\text{tag_affinity})$	0.82	
8	Type voyage	famille= 1.0, couple= 0.67, solo= 0.33	1.0	

utilisateur famille, 10h du matin, mois d’août, 60s de consultation.

1.5.4 Pipeline de classement post-LinUCB

Le score brut LinUCB ne suffit pas pour produire une liste de recommandations optimale. Plusieurs contraintes métier doivent être appliquées en post-traitement : diversité des types d’annonces, garantie d’exposition pour les nouvelles annonces, exploration de catégories inconnues de l’utilisateur, et optimisation des revenus de la plateforme. Le pipeline de classement (`ranker.py`) applique une chaîne de neuf filtres et ajustements dans un ordre précis après le scoring LinUCB initial.

Les trois premiers filtres sont des filtres d’éligibilité qui réduisent l’ensemble des candidats avant le scoring : la correction de popularité marque les annonces sous-exposées, le filtre temporel élimine les annonces inadaptées à l’heure, et le filtre budgétaire supprime les annonces hors budget. Les étapes suivantes construisent les vecteurs de caractéristiques et appliquent le scoring LinUCB. Les quatre derniers filtres sont des ajustements post-scoring qui optimisent la diversité, l’équité et les revenus.

Les neuf étapes du pipeline sont les suivantes :

1. **Correction de popularité** : marquer comme sous-exposées les annonces ayant moins de 50 impressions, déclenchant le bonus $+0.15$ lors du scoring.
2. **Filtre temporel** : retirer les annonces tagées `best_evening` ou `bar le matin` (6h–12h), et `best_morning` le soir (20h–23h).
3. **Filtre budgétaire** : retirer les annonces dont le prix dépasse $\text{budget} \times 1.25$ lorsqu’une estimation de budget est disponible.
4. **Construction des vecteurs ϕ** : calculer le produit de Kronecker pour chaque annonce restante à partir du contexte utilisateur courant.
5. **Scoring LinUCB** : calculer le score UCB pour toutes les annonces restantes.
6. **Filtre de diversité** : parcourir la liste classée de haut en bas, n’accepter au maximum que 2 annonces par type (`STAY`, `ACTIVITY`, `RESTAURANT`).
7. **Garantie nouvelles annonces** : si aucune annonce du résultat top- k n’a moins de 50 impressions, remplacer le dernier créneau par l’annonce sous-exposée la mieux classée.
8. **Exploration de tags** : remplacer 10% des créneaux par les annonces les mieux classées parmi les catégories de tags que l’utilisateur n’a pas encore explorées (affinité < 0.2).
9. **Tri par marge** : dans les paliers de score dont l’écart est inférieur à 0.05, trier par `commission_rate` décroissant pour maximiser les revenus de la plateforme.

Le résultat final est tronqué à `top_k` éléments (par défaut 10) avant d’être retourné.

1.6 Travaux connexes et comparaison

Li et al. [5] ont introduit l'algorithme LinUCB dans le contexte de la recommandation d'articles de presse sur le portail Yahoo! Today. Leur travail a démontré que les bandits contextuels linéaires surpassent les méthodes de recommandation classiques lorsque le contenu évolue rapidement et que les préférences utilisateur dépendent du contexte temporel. Bouneffouf et al. [2] ont étendu cette approche aux recommandations mobiles avec des bandits hybrides intégrant des signaux contextuels similaires aux nôtres (localisation, heure). Le *Thompson Sampling* contextuel constitue une alternative bayésienne, mais LinUCB a été préféré pour sa nature déterministe, sa facilité de débogage (pas d'échantillonnage postérieur), et son caractère éprouvé en production à grande échelle.

TripAdvisor, la plateforme de référence pour les avis touristiques, utilise un système de recommandation combinant filtrage collaboratif et signaux de contenu. Le classement est entraîné hors ligne sur des données historiques massives et ne se met pas à jour en temps réel à partir des signaux de session en cours. De plus, TripAdvisor recommande des attractions individuelles sans composer d'itinéraire multi-journées intégré — l'utilisateur doit assembler manuellement son programme de voyage.

Booking.com a développé un système de classement hybride sophistiqué, intégrant des signaux en temps réel et une infrastructure de tests A/B à grande échelle [1]. Cependant, ce système est entièrement fermé et non reproductible, conçu pour un volume de trafic de plusieurs ordres de grandeur supérieur à celui d'une place de marché touristique locale. Les techniques employées (réseaux de neurones profonds, infrastructure de feature store distribuée) ne sont pas adaptées au contexte d'un projet académique avec des contraintes de ressources limitées.

Dans le domaine de la planification de voyage automatisée, Lim et al. [6] et Jiang et al. [3] proposent des approches par optimisation combinatoire sur des graphes de points d'intérêt. Ces travaux modélisent la planification comme un problème d'optimisation sous contraintes (temps, budget, distance) mais n'intègrent pas d'interaction conversationnelle, de données météorologiques en temps réel, ni d'apprentissage en ligne à partir du comportement utilisateur.

Table 1.4: Comparaison avec les systèmes existants

Système	Planif.	Reco.	TR	Multilingue	Apprent.	OL	Open src.
Guidni (ce projet)	✓	✓		✓	✓		✓
TripAdvisor	×	×		✓	×		×
Booking.com	×	✓		✓	✓		×
ChatGPT (sans outils)	Partiel	×		✓	×		×
Lim et al. (2018)	✓	×		×	×		✓
Bouneffouf et al.	×	✓		×	✓		✓

Planif. = planification multi-jours, *Reco.* = recommandation temps réel, *Apprent.* = apprentissage en ligne.

1.7 Choix technologiques et justification

FastAPI a été préféré à Django et Flask pour sa nature nativement asynchrone, sa génération automatique de documentation OpenAPI, sa validation des données par Pydantic, et ses performances supérieures pour les tâches d'entrée-sortie intensives typiques des applications IA. L'agent LLM effectue de multiples appels réseau concurrents (LLM, base de données, APIs météo, géocodage), rendant l'asynchronicité essentielle.

LangGraph a été choisi plutôt que LangChain Expression Language (LCEL) ou CrewAI pour son état explicitement typé qui facilite le débogage, son graphe orienté qui permet des boucles de validation impossibles avec des chaînes linéaires, son support natif du streaming, et sa flexibilité pour définir des fonctions de routage conditionnel complexes.

LlamaIndex a été préféré à FAISS ou Chroma pour sa gestion intégrée du magasin de documents (*document store*), son filtrage par métadonnées nativement supporté, sa persistance automatique sur disque, et son système d'événements facilitant l'intégration avec SQLAlchemy pour la réindexation incrémentale.

BAAI/bge-m3 a été sélectionné plutôt que OpenAI `text-embedding-3` car il est gratuit, s'exécute localement (aucune donnée transmise à des tiers), supporte nativement le français, l'anglais et l'arabe, et n'introduit aucune dépendance réseau pour la recherche sémantique.

LinUCB a été préféré au Thompson Sampling et aux bandits neuronaux pour sa mathématique plus simple, sa nature déterministe (pas d'échantillonnage MCMC), sa facilité de débogage (le score est une formule explicite), et son efficacité prouvée en production à grande échelle chez Yahoo! [5] et dans les systèmes de recommandation d'actualités.

APScheduler a été choisi plutôt que Celery pour sa simplicité de configuration (pas de broker Redis nécessaire), sa légèreté (une seule dépendance Python), et sa capacité suffisante pour les tâches nocturnes planifiées à l'échelle de ce projet.

Le **tracker JavaScript vanilla** a été développé sans dépendance externe (ni Google Analytics, ni Segment) pour garantir un contrôle total sur les données collectées, éliminer les cookies tiers, assurer la conformité RGPD, et offrir un poids de chargement minimal.

Conclusion du chapitre

Ce chapitre a établi les fondements théoriques nécessaires à la compréhension du système Guidni. Les agents LLM et le patron ReAct ont été présentés comme base de l'architecture agentique, le framework LangGraph a été justifié comme choix de machine à états pour sa debuggabilité et ses boucles de validation, et l'abstraction multi-fournisseurs LLM a été exposée pour sa flexibilité. Le pipeline RAG avec le modèle d'embedding multilingue BAAI/bge-m3 et la réindexation automatique par événements SQLAlchemy ont été décrits comme mécanisme de grounding factuel. Les systèmes de recommandation ont été comparés selon quatre axes, établissant la supériorité de l'apprentissage en ligne par bandits contextuels pour le cas d'usage touristique. L'algorithme LinUCB a été formalisé mathématiquement, l'ingénierie des caractéristiques par produit de Kronecker a été détaillée avec ses 40 tags et son vecteur ϕ de dimension 360, et le pipeline de classement post-scoring en neuf étapes a été décrit.

Le chapitre suivant traduira ces fondements théoriques en exigences concrètes, modèles UML et conceptions architecturales détaillées pour les deux sous-systèmes du projet Guidni.

Chapter 2

Lancement de Projet

2.1 Introduction

Ce chapitre suit la branche fonctionnelle de la méthodologie 2TUP en capturant l'ensemble des besoins du système à travers des modèles UML, avant de converger avec la branche technique pour produire l'architecture globale et la conception de la base de données. L'objectif est de fournir une spécification complète et non ambiguë à partir de laquelle les chapitres suivants pourront implémenter les deux sous-systèmes sans interprétation subjective.

La démarche adoptée progresse en six étapes : l'identification des acteurs et la modélisation des cas d'utilisation (section 3.2), l'expression des besoins non fonctionnels (section 3.3), la conception de l'architecture globale en couches (section 3.4), la spécification du schéma de base de données (section 3.5), la définition des contrats API REST (section 3.6), et enfin les considérations de sécurité et de gestion des données (section 3.7).

2.2 Identification des acteurs et besoins fonctionnels

2.2.1 Identification des acteurs

Le **Touriste** constitue l'acteur principal du système. Il peut être anonyme (session sans authentification) ou identifié (connecté via son compte utilisateur). Le touriste interagit avec l'interface de chat pour formuler des demandes de planification de voyage, recevoir des itinéraires structurés, les modifier par des messages de suivi, et consulter l'historique de ses conversations. Son comportement de navigation — clics, temps de consultation, profondeur de défilement, ajouts aux favoris — est automatiquement capté par le tracker JavaScript pour alimenter le moteur de recommandation LinUCB.

L'**Administrateur de la plateforme** est responsable de la supervision technique du système. Il peut déclencher une reconstruction complète de l'index RAG via l'endpoint `POST /admin/rebuild-index` lorsque l'indexation incrémentale se désynchronise, consulter

l'état de santé du système via `GET /health` (qui vérifie la connexion à la base de données, la disponibilité d'Ollama si configuré, et l'état de l'index RAG), et sélectionner le fournisseur LLM par défaut via la configuration du serveur. L'administrateur n'interagit pas directement avec l'agent conversationnel mais supervise son bon fonctionnement.

Le **Système** représente l'ensemble des processus automatisés qui s'exécutent sans intervention humaine. Il comprend les écouteurs d'événements SQLAlchemy qui déclenchent la réindexation RAG incrémentale lors de modifications en base de données, le tracker JavaScript qui collecte et transmet les événements comportementaux vers l'endpoint `POST /api/recommendations/events`, le planificateur APScheduler qui exécute les trois jobs nocturnes (mise à jour des profils utilisateur, vérification de la dérive conceptuelle, rafraîchissement des tags), et la tâche de persistance des matrices LinUCB qui sauvegarde les matrices A et b en base de données toutes les deux minutes.

2.2.2 Diagramme des cas d'utilisation global

Le diagramme des cas d'utilisation ci-dessous présente une vue d'ensemble des interactions entre les trois acteurs identifiés et les fonctionnalités principales du système Guidni. Les cas d'utilisation sont organisés en trois groupes : les interactions conversationnelles du touriste avec l'agent planificateur, les opérations d'administration du système, et les processus automatisés du sous-système de recommandation.

Deux types de relations structurelles enrichissent le diagramme : la relation d'extension «**extend**» entre “Modifier un plan existant” et “Envoyer un message de planification” indique que la modification est un cas particulier de l'envoi de message ; la relation d'inclusion «**include**» entre “Recevoir un plan de voyage” et “Valider le plan” indique que la validation est systématiquement exécutée avant la présentation du plan au touriste.

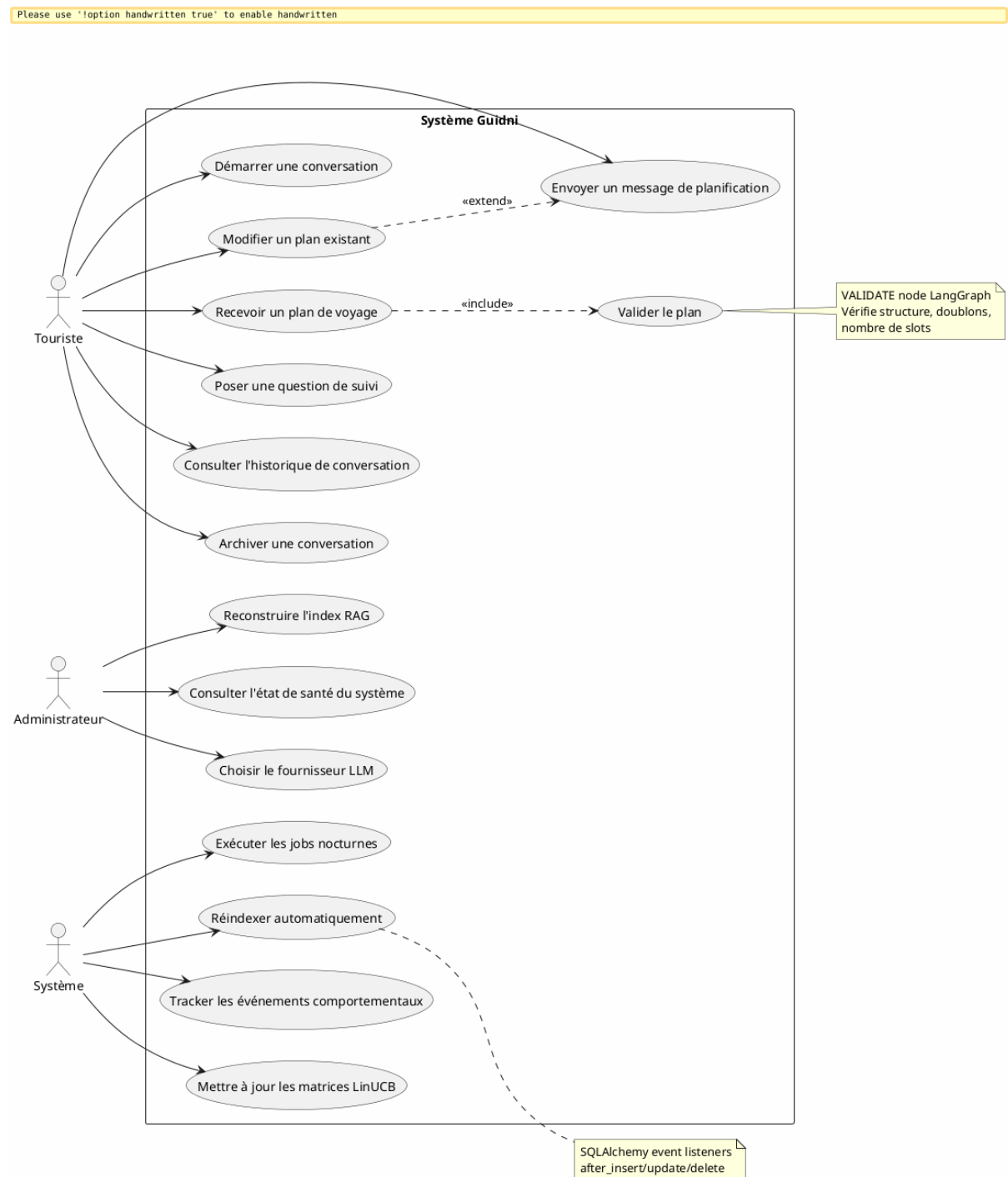


Figure 2.1: Diagramme des cas d'utilisation global du système Guidni

2.2.3 Description détaillée des cas d'utilisation principaux

Les trois cas d'utilisation les plus critiques du système sont décrits en détail dans les tableaux suivants. Chaque description spécifie l'acteur déclencheur, les préconditions nécessaires, le flux principal d'exécution, les flux alternatifs possibles, et les postconditions garanties.

Table 2.1: UC-01 : Générer un plan de voyage

Champ	Description
Acteur	Touriste
Précondition	L'utilisateur existe en base, une conversation est créée ou existante
Déclencheur	L'utilisateur envoie un message de planification via <code>POST /chat</code>
Flux principal	(1) Réception de la requête <code>ChatRequest</code> avec <code>user_id</code> et <code>message</code> ; (2) Validation de l'existence de l'utilisateur en base (HTTP 404 sinon) ; (3) Chargement du contexte (8 derniers messages + plan courant + résumé) ; (4) Appel de <code>run_agent()</code> avec le graphe <code>LangGraph</code> ; (5) Nœud <code>THINK</code> : le LLM raisonne avec les 18 outils disponibles ; (6) Exécution des outils (recherche RAG, météo, activités, géolocalisation) ; (7) <code>create_plan_structure()</code> valide le plan contre la base de données ; (8) Nœud <code>VALIDATE</code> : vérification des doublons et du nombre de créneaux ; (9) Nœud <code>RESPOND</code> : sauvegarde du plan dans <code>GeneratedPlan</code> ; (10) Retour du <code>PlannerResponse</code> au frontend.
Flux alternatif	Le LLM a besoin d'informations $\rightarrow$ outil <code>ask_user</code> $\rightarrow$ <code>needs_more_info=True</code> $\rightarrow$ <code>RESPOND</code> retourne une question
Postcondition	Ligne <code>GeneratedPlan</code> créée, <code>PlannerMessage</code> sauvegardé

Table 2.2: UC-02 : Obtenir des recommandations personnalisées

Champ	Description
Acteur	Touriste / Système
Précondition	Des lignes <code>BanditArm</code> existent pour la zone demandée
Déclencheur	Requête <code>GET /api/recommendations</code> avec paramètres de contexte
Flux principal	(1) Récupération des bras depuis le cache RAM ou la base de données ; (2) Chargement du profil utilisateur si <code>user_id</code> fourni ; (3) Construction du <code>UserContext</code> à partir des paramètres de requête ; (4) Chargement des matrices A et b depuis le cache RAM ; (5) Calcul du vecteur ϕ par produit de Kronecker pour chaque bras ; (6) Exécution du pipeline de classement en 9 étapes ; (7) Retour du <code>RecommendationResponse</code> avec items classés.
Postcondition	<code>RecommendationEvent</code> de type <code>impression</code> enregistré

Table 2.3: UC-03 : Tracker un événement comportemental

Champ	Description
Acteur	Système (tracker JavaScript)
Précondition	Session active avec un <code>session_id</code> valide
Déclencheur	L'utilisateur clique, consulte, ou réserve une annonce
Flux principal	(1) Le tracker JS met l'événement en file d'attente ; (2) Toutes les 15 secondes ou tous les 10 événements : <code>flush()</code> ; (3) <code>POST /api/recommendations/events</code> avec le lot d'événements ; (4) <code>compute_reward()</code> calcule la récompense delta pour chaque événement ; (5) Le vecteur ϕ est calculé pour chaque bras concerné ; (6) Les matrices A et b sont mises à jour en RAM ; (7) <code>log_event()</code> et <code>update_arm_counters()</code> enregistrent les compteurs ; (8) Réponse avec les récompenses de session cumulées.
Postcondition	Compteurs <code>BanditArm</code> mis à jour, <code>RecommendationEvent</code> créé

La section suivante complète l'analyse en spécifiant les besoins non fonctionnels du système.

2.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité que le système doit respecter indépendamment de ses fonctionnalités. Ils sont classifiés selon les catégories de la norme ISO 25010, qui couvre la performance, la disponibilité, la scalabilité, la maintenabilité, la sécurité, le support multilingue et la confidentialité des données.

Le tableau suivant synthétise ces besoins avec pour chaque catégorie une mesure cible vérifiable, permettant de valider le respect de ces exigences lors de la phase de tests présentée au chapitre 5.

Table 2.4: Besoins non fonctionnels classifiés selon la norme ISO 25010

Catégorie	Besoin	Mesure / Cible
Performance	Latence de l'endpoint <code>/chat</code>	< 5 s (P95)
	Latence <code>/api/recommendations</code>	< 100 ms (P95)
	Scoring LinUCB pour $N = 1000$ bras	< 50 ms
Disponibilité	Uptime du service	99,5%
	Démarrage sans index RAG	Non fatal (mode dégradé)
Scalabilité	Pool de connexions <code>psycopg2</code>	<code>minconn= 2, maxconn= 10</code>
	Pool <code>AsyncPG</code>	<code>pool_size= 10, max_overflow= 20</code>
Maintenabilité	Ajout d'un nouveau fournisseur LLM	1 fichier client + config
	Journalisation rotative	10 MB info, 50 MB debug
Sécurité	Validation utilisateur	Chaque requête vérifie <code>user_id</code>
	Limitation de débit événements	10/listing/session, 100/session
Multilingue	Langues supportées	FR, EN, AR (auto-détection)
	Embeddings multilingues	BAAI/bge-m3 (100+ langues)
Confidentialité	Données vers LLM externe	Pas de PII sans consentement
	Option locale	Ollama pour souveraineté complète

La section suivante présente l'architecture globale conçue pour satisfaire ces besoins.

2.4 Architecture globale du système

2.4.1 Vue d'ensemble architecturale

Le **frontend** du système Guidni est une application Next.js couplée à l'ORM Prisma, qui gère l'interface utilisateur, l'authentification, et les opérations CRUD sur les entités touristiques (activités, hébergements, restaurants, locations, transferts). Le frontend lit et écrit dans la base de données PostgreSQL partagée via Prisma, et communique avec le backend IA via les endpoints REST `/chat`, `/plan` et `/api/recommendations`.

Le **backend IA** est une application FastAPI qui héberge les deux sous-systèmes intelligents : l'agent planificateur LangGraph (9 endpoints REST) et le moteur de recommandation LinUCB (3 endpoints REST). Il possède en écriture 4 tables planificateur et 7 tables recommandation, et lit en lecture seule les tables Prisma du frontend. L'application utilise SQLAlchemy en mode asynchrone avec le pilote `asyncpg` pour les opérations du planificateur, et `psycopg2` avec un pool de connexions threadé pour les opérations de recommandation en temps réel.

Les **services externes** comprennent l'API Open-Meteo pour les données météorologiques en temps réel, le service Nominatim pour le géocodage et le calcul de distances par la

formule de Haversine, le serveur Ollama local pour l'inférence LLM sans réseau, l'API cloud Groq pour l'inférence rapide, l'API Google Gemini pour les contextes longs, et l'API Lightning AI comme quatrième fournisseur. Le stockage LlamaIndex sur disque (`rag_storage/`) persiste l'index vectoriel entre les redémarrages.

Le **tracker JavaScript** est un module vanilla JS sans dépendance externe, intégré dans les pages du frontend. Il collecte les événements comportementaux (clics, temps de consultation, défilement, ajout aux favoris, réservations) et les transmet par lots vers l'endpoint `POST /api/recommendations/events`. Il utilise l'API `navigator.sendBeacon()` pour garantir la transmission des événements même lors de la fermeture de la page.

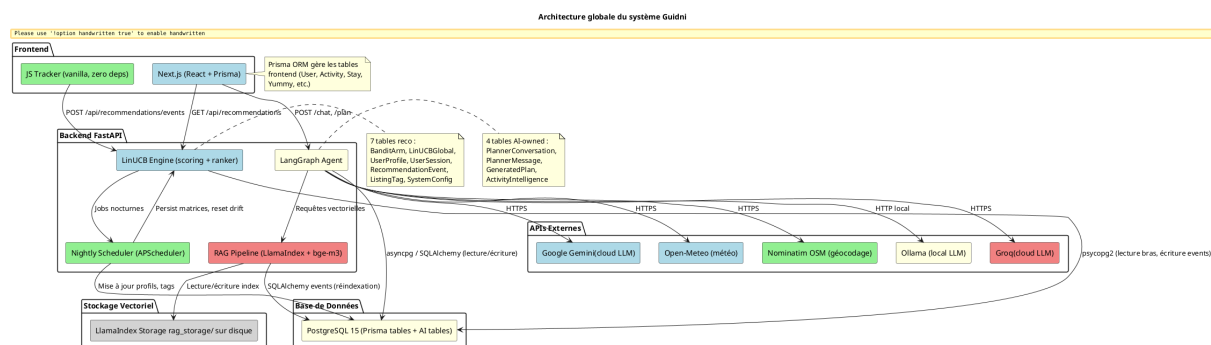


Figure 2.2: Architecture globale du système Guidni

2.4.2 Architecture en couches

La **couche de présentation** comprend les endpoints FastAPI définis dans `main.py` et `recommendation_router.py`, les schémas Pydantic de requête et de réponse (`ChatRequest`, `PlannerResponse`, `UserPreferences`), le middleware CORS configuré pour accepter toutes les origines en développement, et la génération automatique de documentation OpenAPI accessible via l'endpoint `/docs`.

La **couche de logique métier** contient le cœur intelligent du système : l'agent LangGraph avec sa machine à états à quatre nœuds et ses 18 outils appelables, le moteur de classement LinUCB avec son pipeline en 9 étapes, le moteur de requête RAG avec recherche sémantique et reranking par cross-encoder, et le système de mémoire conversationnelle (chargement des 8 derniers messages, résumé automatique au seuil de 15 messages, construction du profil utilisateur).

La **couche d'accès aux données** regroupe les modèles SQLAlchemy ORM et les requêtes asynchrones pour le sous-système planificateur, le constructeur d'index LlamaIndex avec persistance sur disque et réindexation incrémentale par événements, et le pool de connexions `psycopg2` (`ThreadedConnectionPool`, `minconn= 2`, `maxconn= 10`) pour les opérations de recommandation en temps réel. La **couche d'infrastructure** fournit le moteur AsyncPG avec pool de connexions (`pool_size= 10`, `max_overflow= 20`), le planificateur APScheduler configuré sur le fuseau horaire Africa/Tunis, la journalisation

rotative à deux niveaux (10 MB info, 50 MB debug), et la gestion de la configuration par variables d'environnement via `dotenv`.

La section suivante détaille la conception du schéma de base de données partagé.

2.5 Conception de la base de données

2.5.1 Vue d'ensemble du schéma

Le système Guidni adopte un patron de base de données partagée : la même instance PostgreSQL 15 est utilisée simultanément par le frontend Next.js/Prisma et par le backend IA FastAPI. Le backend IA lit 14 tables gérées par Prisma (modèles en lecture seule : `User`, `Activity`, `Stay`, `Restaurant`, `Attraction`, `Rental`, `Transfer`, etc.) et possède en écriture 4 tables planificateur et 7 tables recommandation. Cette architecture évite la duplication des données tout en maintenant des frontières de propriété claires entre les deux systèmes.

La convention de nommage reflète l'origine de chaque table : les tables Prisma utilisent le PascalCase avec guillemets doubles ("`User`", "`Activity`"), les tables AI-owned du planificateur utilisent également le PascalCase ("`PlannerConversation`", "`GeneratedPlan`"), tandis que les tables de recommandation sont nommées selon les conventions définies dans le module `db_init.py`. Cette distinction visuelle permet d'identifier immédiatement l'origine et le propriétaire de chaque table.

La stratégie de versionnement des plans repose sur le champ `isActive` de la table `GeneratedPlan`. Lorsqu'un utilisateur modifie un plan existant via un message de suivi, l'ancien plan est désactivé (`isActive=False`) et une nouvelle ligne est insérée avec le champ `version` incrémenté. Cette approche conserve l'historique complet des plans générés et permet la récupération de versions antérieures si nécessaire.

Schéma ER du sous-système agent planificateur

Please use '!option handwritten true' to enable handwritten



Figure 2.3: Schéma ER du sous-système agent planificateur

Schéma ER du sous-système de recommandation LinUCB

Please use '!option handwritten true' to enable handwritten

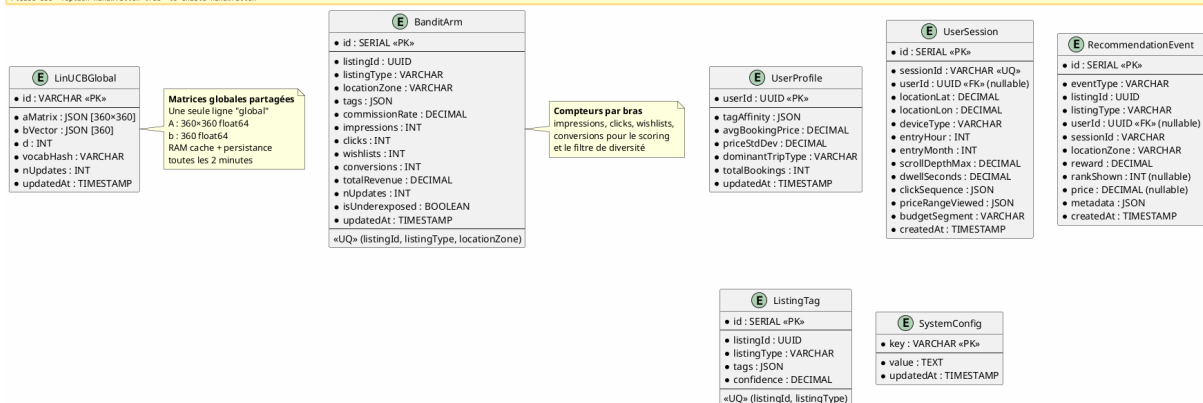


Figure 2.4: Schéma ER du sous-système de recommandation LinUCB

2.5.2 Description des tables clés

Le tableau suivant récapitule les onze tables détenues par le backend IA, en précisant leur rôle fonctionnel, les composants qui y écrivent et ceux qui les lisent. Cette matrice de responsabilité permet de comprendre les flux de données entre les différents composants du système.

Table 2.5: Description des tables AI-owned

Table	Rôle	Écrit par	Lu par
PlannerConversation	Suivi des conversations agent	Agent (main.py)	Agent, Frontend
PlannerMessage	Messages individuels	Agent (main.py)	Agent (contexte)
GeneratedPlan	Plans versionnés	Agent (RESPOND)	Frontend, Agent
ActivityIntelligence	Cache d'enrichissement IA	Agent (outils)	Agent (outils)
BanditArm	Compteurs par annonce	Reco router, Nightly	Ranker
LinUCBGlobal	Matrices A et b globales	Cache RAM (2 min)	Ranker (scoring)
UserProfile	Affinités de tags utilisateur	Job nocturne	Ranker (contexte)
UserSession	Données de session navigation	Reco router	Ranker (contexte)
RecommendationEvent	Journal des interactions	Reco router	Job nocturne
ListingTag	Tags extraits par LLM	Tag extractor	Ranker (phi)
SystemConfig	Configuration système	Admin, Nightly	Startup checks

La section suivante définit les contrats API REST pour les deux sous-systèmes.

2.6 Conception de l'API REST

L'API REST du système Guidni suit les conventions RESTful standard : les méthodes HTTP (**GET**, **POST**, **DELETE**) correspondent aux opérations de lecture, création et suppression, les corps de requête et de réponse utilisent le format JSON, et la validation des entrées est assurée par les schémas Pydantic qui génèrent automatiquement des erreurs 422 en cas de données invalides.

La documentation OpenAPI est générée automatiquement par FastAPI et accessible via l'endpoint `/docs`. Elle décrit l'ensemble des endpoints, leurs paramètres, les schémas de requête et de réponse, et les codes d'erreur possibles. Les deux tableaux suivants spécifient les contrats API pour chacun des deux sous-systèmes.

Table 2.6: Contrat API — Sous-système Agent Planificateur

Mét.	Endpoint	Description	Requête	Réponse
POST	/chat	Chat principal	ChatRequest	PlannerResponse
POST	/plan	Pipeline plan	UserPreferences	{plan_data}
GET	/conversations/ {user_id}	Lister conversations	—	{conversations}
GET	/conversations/ detail/{conv_id}	Détail conversation	—	{messages}
DELETE	/conversations/ {conv_id}	Archiver	—	{archived}
GET	/users	Lister utilisateurs	—	{users}
GET	/models	Modèles LLM	—	{models}
POST	/admin/ rebuild-index	Reconstruire RAG	—	{documents_indexed}
GET	/health	État de santé	—	{status, db, rag}

Table 2.7: Contrat API — Sous-système de Recommandation

Mét.	Endpoint	Description	Requête	Réponse
GET	/api/ recommendations	Recommandations	Query params (zone, types, session, hour, month, top_k)	RecommendationResponse
POST	/api/ recommendations/ events	Lot d'événements	BatchRequest (session_id, events[])	BatchResponse
POST	/api/ recommendations/ merge-session	Fusionner session	{session_id, user_id}	{merged_events}

2.7 Sécurité et gestion des données

La configuration CORS (*Cross-Origin Resource Sharing*) est définie dans `main.py` avec `allow_origins=["*"]` pour la phase de développement, permettant au frontend Next.js de communiquer avec le backend FastAPI sans restriction d'origine. En production, cette configuration devra être restreinte au domaine spécifique du frontend pour prévenir les requêtes cross-origin non autorisées.

La validation de l'utilisateur est effectuée sur chaque requête entrante. Avant tout traitement, le `user_id` fourni dans la requête est vérifié par une requête `SELECT` sur la table "User". Si l'utilisateur n'est pas trouvé, une erreur HTTP 404 est retournée immédiatement avec un message explicite. Cette vérification systématique empêche tout traitement pour des identifiants inexistants ou forgés.

Le moteur de recommandation implémente une limitation de débit en mémoire pour prévenir les abus : un maximum de 10 événements par annonce et par session, et 100 événements au total par session. Les sessions inactives depuis plus de 2 heures sont automatiquement

nettoyées du cache mémoire pour éviter les fuites de mémoire. Ces seuils sont configurés dans le module `recommendation_router.py` et appliqués avant tout calcul de récompense.

La confidentialité des données est assurée par plusieurs mécanismes : les résultats des outils transmis au LLM externe ne contiennent aucune donnée personnelle identifiable (PII) — seuls les identifiants, catégories et caractéristiques anonymisées sont envoyés. Le fournisseur Ollama local offre une option de souveraineté complète pour les déploiements sensibles, car aucune donnée ne quitte le serveur. Les vecteurs de caractéristiques du moteur LinUCB sont construits à partir de signaux comportementaux agrégés (profondeur de défilement, temps de consultation) et non à partir de données personnelles brutes.

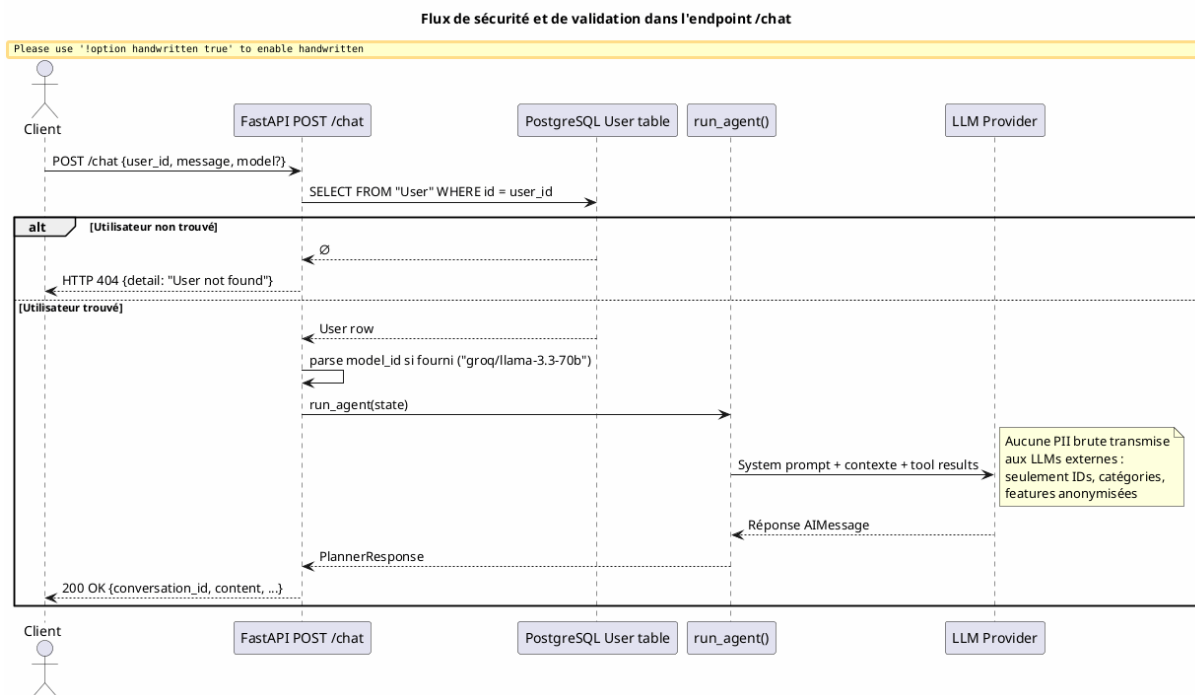


Figure 2.5: Flux de sécurité et de validation dans l'endpoint /chat

Conclusion du chapitre

Ce chapitre a présenté l'analyse complète des besoins et la conception du système Guidni. Les trois acteurs du système ont été identifiés (Touriste, Administrateur, Système) et leurs interactions modélisées par un diagramme de cas d'utilisation global. Les trois cas d'utilisation principaux ont été décrits en détail : la génération de plan de voyage, l'obtention de recommandations personnalisées, et le suivi des événements comportementaux. Les besoins non fonctionnels ont été classifiés selon la norme ISO 25010, l'architecture globale a été conçue en couches avec une séparation claire des responsabilités, le schéma de base de données partagé a été spécifié avec ses 11 tables AI-owned, et les contrats API REST ont été formalisés pour les 12 endpoints (9 planificateur + 3 recommandation).

Le chapitre suivant présentera l'implémentation détaillée de l'agent planificateur IA,

en décrivant le graphe LangGraph, les 18 outils appelables, le pipeline RAG, et le système de mémoire conversationnelle.

Chapter 3

Sprint 1 — Conception et Implémentation de l’Agent Planificateur

3.1 Introduction

Ce chapitre présente l’implémentation complète de l’agent planificateur IA du système Guidni, conformément à l’architecture conçue au chapitre 3. L’implémentation est organisée autour de quatre composants principaux : la machine à états LangGraph qui orchestre le raisonnement de l’agent, les 18 outils appelables par le LLM qui permettent l’accès aux données en temps réel, le pipeline RAG qui ancre les réponses dans des données factuelles vérifiées, et le système de gestion de la mémoire conversationnelle qui assure la cohérence multi-tour.

La présentation suit le flux d’exécution naturel d’une requête utilisateur : le message entre dans le graphe via le nœud THINK, les outils sont exécutés par le nœud EXECUTE_TOOL, le plan est validé par le nœud VALIDATE, et la réponse finale est formatée et persistée par le nœud RESPOND. Pour chaque composant, les choix d’implémentation sont justifiés par les contraintes du domaine touristique et les spécificités du marché de Djerba.

3.2 Le graphe LangGraph : machine à états

3.2.1 L’AgentState TypedDict

Le `TypedDict` Python est une structure de dictionnaire avec des clés et des types déclarés statiquement, offrant les bénéfices d’un objet fortement typé tout en restant compatible avec l’écosystème de sérialisation JSON. LangGraph utilise ce patron pour définir l’état qui circule entre tous les nœuds du graphe : chaque nœud reçoit l’état courant, effectue son

traitement, et retourne un dictionnaire partiel contenant uniquement les champs modifiés. Le framework fusionne automatiquement ces mises à jour partielles dans l'état global.

Les champs de l'**AgentState** sont divisés en deux catégories de comportement. Les champs **accumulateurs** utilisent l'annotation `Annotated[list, operator.add]`, ce qui signifie que chaque nœud *ajoute* ses éléments à la liste existante plutôt que de la remplacer. Deux champs utilisent ce mécanisme : **messages** (la liste complète des messages LangChain) et **thinking_steps** (les étapes de réflexion affichées dans l'interface utilisateur). Les champs à **écrasement** (tous les autres) sont remplacés par la dernière valeur fournie par un nœud, selon le comportement par défaut de LangGraph.

Les champs sont regroupés par fonction : les champs d'identité conversationnelle (**user_id**, **conversation_id**, **is_modification**) identifient le contexte de la requête ; les champs de données peuplés par les outils (**user_profile**, **activities**, **enriched_activities**, **weather**, **stays**, **restaurants**, **rag_context**, **current_plan**) stockent les résultats intermédiaires ; les drapeaux de contrôle de flux (**needs_more_info** déclenche le routage vers RESPOND, **is_plan_ready** déclenche le routage vers VALIDATE, **iteration_count** est incrémenté par THINK et vérifié contre **MAX_AGENT_ITERATIONS=15**) ; les champs de sélection LLM (**llm_provider**, **llm_model**) permettent le choix du fournisseur par requête ; et les champs de sortie (**response_type**, **final_response**, **final_plan**, **questions**) sont définis par RESPOND.

Table 3.1: Champs de l'AgentState et leur comportement

Champ	Type	Comport.	Rôle
messages	list[BaseMessage]	Accumule	Messages (conversation) LangChain
thinking_steps	list[dict]	Accumule	Étapes de réflexion pour l'UI
user_id	str	Écrase	Identifiant utilisateur
conversation_id	str	Écrase	Identifiant conversation
user_profile	dict None	Écrase	Profil utilisateur (outils)
activities	list[dict] None	Écrase	Activités trouvées
enriched_activities	list[dict] None	Écrase	Activités enrichies par IA
weather	dict None	Écrase	Données météo
stays	list[dict] None	Écrase	Hébergements trouvés
restaurants	list[dict] None	Écrase	Restaurants trouvés
rag_context	list[dict] None	Écrase	Résultats RAG
current_plan	dict None	Écrase	Plan en cours de construction
tools_used	list[str]	Écrase	Outils utilisés ce tour
iteration_count	int	Écrase	Compteur de sécurité
needs_more_info	bool	Écrase	Question à poser ?
is_plan_ready	bool	Écrase	Plan finalisé ?
is_modification	bool	Écrase	Modification d'un plan ?
response_type	str	Écrase	Type de réponse finale
final_response	str	Écrase	Texte de réponse
final_plan	dict None	Écrase	Plan structuré final
questions	list[dict] None	Écrase	Questions avec suggestions
llm_provider	str None	Écrase	Fournisseur LLM choisi
llm_model	str None	Écrase	Modèle LLM choisi

3.2.2 Fonctions de routage conditionnel

La fonction `_should_continue(state)` détermine le nœud suivant après THINK. Elle évalue cinq conditions dans un ordre de priorité strict. Premièrement, si `iteration_count` $\geq$ `MAX_AGENT_ITERATIONS` (15), le routage force la réponse ("respond") pour empêcher les boucles infinies : sans ce garde-fou, un outil défaillant ou un LLM confus pourrait boucler indéfiniment, consommant des tokens et des crédits API. Deuxièmement, si le dernier message possède un attribut `tool_calls` non vide, le routage envoie vers "execute_tool" pour exécuter les outils demandés.

Troisièmement, si `needs_more_info` est `True` (l'outil `ask_user` a été invoqué), le routage envoie vers "respond" pour délivrer la question à l'utilisateur immédiatement. Quatrièmement, si `is_plan_ready` est `True` et `current_plan` est non nul, le routage envoie vers "validate" pour vérifier la cohérence du plan. Cinquièmement, par défaut (réponse textuelle), le routage envoie vers "respond".

La fonction `_after_tool_execution(state)` détermine le nœud suivant après EXECUTE_TOOL. Si `needs_more_info` est `True` (le tool `ask_user` a été utilisé), le routage envoie vers "respond" pour éviter un retour inutile au LLM. Sinon, le routage renvoie vers "think" pour que le LLM réévalue la situation avec les résultats d'outils reçus.

La fonction `_after_validation(state)` détermine le nœud suivant après VALIDATE. Si le plan a été invalidé (`is_plan_ready=False`) et qu'il reste des itérations (`iteration_count < 15`), le routage renvoie vers "think" pour permettre au LLM de corriger les erreurs identifiées. Sinon, le routage envoie vers "respond" (soit la validation a réussi, soit les itérations sont épuisées).

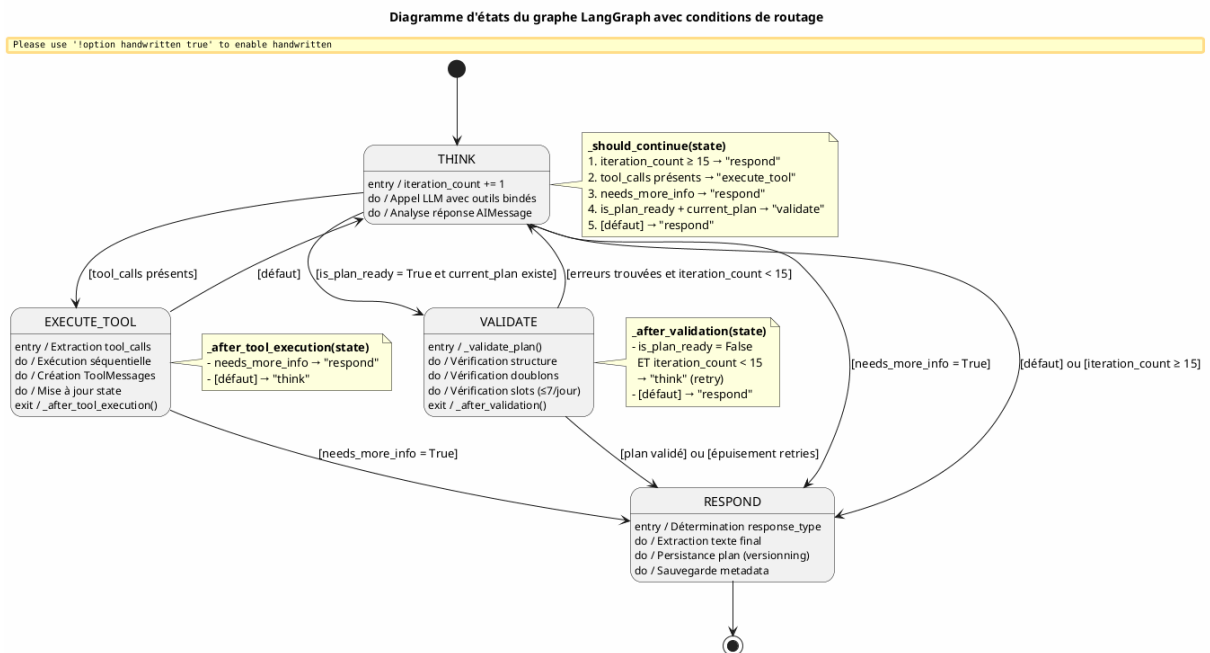


Figure 3.1: Diagramme d'états du graphe LangGraph avec conditions de routage

3.3 Le nœud THINK

3.3.1 Construction du prompt système

Le prompt système du module `personality.py` (122 lignes) définit l'identité de l'agent Guidni comme un ami local chaleureux et expert du tourisme à Djerba, qui parle français, anglais et arabe avec détection automatique de la langue de l'utilisateur. Le prompt interdit explicitement à l'agent d'inventer des données : il ne doit recommander que des lieux, activités et hébergements effectivement récupérés par les outils de recherche. Les recommandations doivent aller au-delà des clichés touristiques pour offrir des conseils authentiques d'initié.

Les règles comportementales encodées dans le prompt définissent les contraintes de planification réaliste : un maximum de 4 à 5 activités par jour, inclusion systématique de repas et de périodes de repos, prise en compte des distances et de la logistique entre les lieux (en utilisant l'outil `get_distance`), variation des types d'activités pour éviter la monotonie, et explication transparente du raisonnement derrière chaque recommandation. L'agent doit réfléchir avant de répondre (principe ReAct), être honnête sur ses limitations, et ne jamais forcer une recommandation si les données disponibles sont insuffisantes.

Le prompt contient également des règles cruciales de désambiguïsation des types d'entités : `activity` désigne une expérience payante (catégorie "DO"), `stay` désigne un hébergement uniquement (ne *jamais* placer un hébergement dans un créneau d'activité diurne), `restaurant/yummy` désigne un lieu de restauration (catégorie "EAT"), et `attraction` désigne un site touristique gratuit. Cette désambiguïsation est essentielle car sans elle, le LLM peut confondre un hôtel avec une attraction à visiter dans la journée.

3.3.2 Injection du contexte dynamique

Avant chaque appel au LLM, le nœud THINK construit un message de contexte à partir de l'état courant. Si `user_profile` existe, un résumé du profil est ajouté. Si `current_plan` existe, l'information "version X avec N jours" est ajoutée. Si `weather` existe, les prévisions météorologiques sont incluses. Si `rag_context` est non vide, les cinq premiers résultats sont formatés avec titre, type d'entité et score de similarité. Ce contexte est injecté comme un `SystemMessage` distinct du prompt de personnalité, permettant au LLM de voir clairement les données déjà collectées.

L'heuristique d'arrêt forcé constitue un mécanisme de sécurité critique : si `iteration_count` ≥ 2 et que `rag_context` est non vide, un `SystemMessage` final est injecté avec l'instruction : "IMPORTANT : Vous avez déjà suffisamment de données. Générez votre réponse FINALE maintenant. N'appellez PLUS d'outils de recherche." Cette heuristique empêche l'agent de rechercher indéfiniment lorsqu'il dispose déjà d'un contexte suffisant, réduisant la latence

et la consommation de tokens.

La normalisation du contenu spécifique à Gemini est gérée par les fonctions utilitaires `_get_text_content()` et `_normalize_content()`. Google Gemini retourne le contenu des messages sous forme de liste de dictionnaires (`[{"type": "text", "text": "..."}]`) au lieu d'une chaîne de caractères simple comme Ollama et Groq. Ces fonctions détectent le format et extraient le texte de manière transparente, assurant la compatibilité multi-fournisseurs sans modification du code des nœuds.

3.4 Les 18 outils LLM

3.4.1 Vue d'ensemble et registre des outils

Le décorateur `@tool` de LangChain transforme une fonction Python asynchrone en un outil appelable par le LLM. La décoration analyse la signature de la fonction, sa docstring et ses annotations de type pour générer automatiquement un schéma JSON décrivant le nom de l'outil, sa description et les paramètres attendus. La méthode `bind_tools()` de LangGraph attache ensuite ces schémas au LLM pour que le modèle puisse émettre des appels d'outils structurés.

Le registre central défini dans `registry.py` exporte la fonction `get_all_tools()` qui retourne les 18 outils dans un ordre fixe. Lors de l'exécution, le nœud `EXECUTE_TOOL` construit un dictionnaire de correspondance `{tool.name: tool}` pour dispatcher les appels par nom. Les outils couvrent sept domaines fonctionnels : le profil utilisateur (1 outil), la recherche d'entités (5 outils), la météo et la géolocalisation (3 outils), la disponibilité et le budget (3 outils), la construction de plans (3 outils), la communication (1 outil) et la recherche sémantique RAG (3 outils).

La gestion des erreurs suit un patron uniforme : chaque outil encapsule ses requêtes de base de données et ses appels API dans un bloc `try/except` et retourne un dictionnaire `{"error": "message"}` plutôt que de lever une exception. Le LLM reçoit l'erreur dans un `ToolMessage` et peut décider de réessayer avec des paramètres différents ou d'informer l'utilisateur du problème. Ce patron garantit que le graphe LangGraph ne s'arrête jamais brutalement à cause d'un outil défaillant.

Table 3.2: Catalogue des 18 outils LLM

Outil	Module	Tables DB	API ext.	Retourne
get_user_profile	user_tools	User, Reservation, Review, Wishlist, Preference	—	Profil dict
search_activities	activity_tools	Activity, Images	—	Liste activités
get_activity_details	activity_tools	Activity, Images	—	Activité complète
enrich_activity	activity_tools	ActivityIntelligence	LLM	Enrichissement JSON
get_weather	weather_tools	—	Open-Meteo	Météo dict
get_distance	geo_tools	—	— (Haversine)	{km, minutes}
geocode_address	geo_tools	—	Nominatim	{lat, lon}
search_stays	stay_tools	Stay, Images	—	Liste hébergements
search_restaurants	restaurant_tools	Restaurant, Images, Hours	—	Liste restaurants
check_availability	availability_tools	Activity, Reservation	—	{available, spots}
estimate_daily_budget	budget_tools	—	—	{total, ventilation}
estimate_trip_budget	budget_tools	—	—	{total, moy. jour}
create_plan_structure	plan_tools	Activity (valid.)	—	{valid, plan}
modify_plan	plan_tools	—	—	{modified, plan}
save_plan_tool	plan_tools	GeneratedPlan	—	{saved, id}
ask_user	communication	—	—	{question, suggestions}
rag_search	rag_tools	VectorStoreIndex	—	Résultats scorés
rag_search_for_plan	rag_tools	VectorStoreIndex	—	4 listes thématiques
rag_get_similar	rag_tools	VectorStoreIndex	—	Entités similaires

3.4.2 Outil rag_search_for_plan : quatre recherches parallèles

L'outil `rag_search_for_plan` est le plus complexe du registre car il adresse un problème fondamental : une requête RAG unique ne peut pas distinguer entre “que faire” (activités, attractions) et “où manger” (restaurants) et “où dormir” (hébergements). Mélanger ces types dans une seule requête pollue les résultats car les vecteurs d'embedding de “hôtel 3 étoiles avec piscine” sont sémantiquement proches de “activité de natation”, produisant des recommandations incohérentes.

L'outil exécute quatre recherches parallèles avec des requêtes spécialisées : la recherche d'activités utilise la requête complète de l'utilisateur enrichie du thème de la journée ; la recherche de restaurants utilise une requête alimentaire spécifique (“restaurant dining food cuisine local {region}”) indépendamment du thème du voyage, car des thèmes comme “journée aventure” ne correspondent pas aux vecteurs de description des restaurants ; la

recherche d'hébergements utilise la requête complète avec un filtre de prix ; la recherche d'attractions utilise une requête patrimoniale (“sightseeing landmark museum heritage culture {region}”) pour trouver des sites gratuits.

La logique de dimensionnement adapte le nombre de résultats demandés à la durée du voyage. Les formules utilisées sont : $\min(\text{num_days} \times 4, 20)$ pour les activités, $\min(\text{num_days} \times 2, 10)$ pour les restaurants, $\min(\text{num_days}, 5)$ pour les hébergements, et $\min(\text{num_days} \times 2, 10)$ pour les attractions. Cette adaptation empêche le sur-échantillonnage pour les voyages courts et le sous-échantillonnage pour les voyages longs.

Le résultat retourné contient un champ **WARNING** proéminent : “stays are ACCOMMODATION — suggest as overnight stay ONLY, NEVER place them as daytime activities”. Ce garde-fou textuel injecté dans la réponse de l'outil empêche le LLM de commettre l'erreur de placer un hôtel dans un créneau d'activité diurne — un problème récurrent lors des phases de test qui corrompait les plans générés.

3.4.3 Outil `enrich_activity` : cache d'intelligence IA

L'outil `enrich_activity` produit des métadonnées structurées que la base de données ne contient pas nativement : `noise_level` (calme/modéré/bruyant), `ambiance` (romantique/famille/aventure), `physical_intensity` (faible/modérée/élevée), `best_time_of_day` (matin/après-midi/soir/indifférent), `ideal_weather`, `ideal_group_type`, `age_friendly`, `indoor_outdoor`, `tags`, et un résumé en une phrase. Ces métadonnées permettent à l'agent de prendre des décisions de planification plus intelligentes que le simple filtrage par catégorie.

Le mécanisme de cache repose sur la table `ActivityIntelligence`. Avant tout appel au LLM, l'outil vérifie l'existence d'un enrichissement préexistant pour l'activité demandée. Si un enrichissement est trouvé, il est retourné immédiatement sans appel LLM (coût nul, réponse instantanée). Si aucun enrichissement n'existe, le LLM est appelé, la réponse JSON est analysée, et le résultat est inséré dans la table via un `upsert`. Les appels ultérieurs pour la même activité sont gratuits et instantanés.

En cas d'échec de l'appel LLM (erreur réseau, quota dépassé, JSON malformé), un enrichissement de base est généré à partir du champ `categories` de l'activité. Par exemple, une activité de catégorie “water sports” reçoit automatiquement `physical_intensity="high"` et `indoor_outdoor="outdoor"`. Ce mécanisme de repli garantit que l'outil ne retourne jamais une erreur à l'agent, maintenant la fluidité du cycle de raisonnement.

3.5 Le nœud `EXECUTE_TOOL`

Le nœud d'exécution des outils, défini dans `tool_executor.py`, constitue le pont entre les décisions du LLM et les systèmes de données. Il extrait tous les `tool_calls` du dernier `AIMessage`, construit un dictionnaire de correspondance `{name: tool}` à partir du

registre, et exécute chaque outil séquentiellement via `await tool_fn.ainvoke(tool_args)`. Chaque résultat est encapsulé dans un `ToolMessage` avec le contenu sérialisé en JSON compact et l'identifiant `tool_call_id` pour la corrélation avec l'appel d'origine.

Les mises à jour critiques de l'état sont déclenchées par les résultats de certains outils. Lorsque `create_plan_structure` retourne `result["valid"]=True`, l'exécuteur met à jour `is_plan_ready=True` et `current_plan=result["plan"]`, déclenchant le routage vers `VALIDATE`. Lorsque `ask_user` est invoqué, `needs_more_info=True` et `questions` sont mis à jour, provoquant un routage direct vers `RESPOND`. Les trois outils RAG (`rag_search`, `rag_search_for_plan`, `rag_get_similar`) mettent à jour le champ `rag_context` ; dans le cas spécifique de `rag_search_for_plan`, les quatre sous-listes (`activities_for_time_slots`, `restaurants_for_meals`, `stays_for_overnight_only`, `attractions_for_time_slots`) sont aplaties en une liste unifiée.

Pour chaque outil exécuté, une étape de réflexion (*thinking step*) est créée avec une description lisible par l'humain (par exemple “✓ search_activities completed — Found 8 items”) et ajoutée à la liste accumulée `thinking_steps`. En cas d'échec d'un outil, un message d'erreur est créé (“× tool_name failed: message”) et l'erreur est encapsulée dans un `ToolMessage` pour que le LLM puisse décider de la marche à suivre. L'exécuteur effectue également une journalisation détaillée au niveau `DEBUG`, incluant les arguments d'entrée complets et les résultats complets de chaque outil, facilitant le débogage des chaînes de raisonnement complexes.

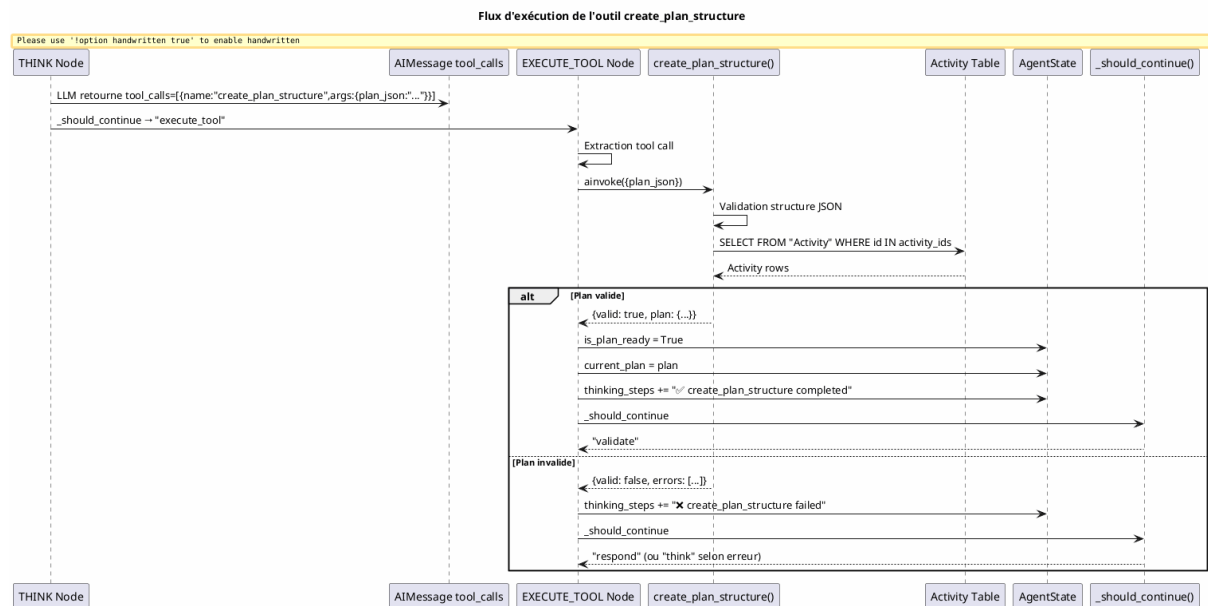


Figure 3.2: Flux d'exécution de l'outil `create_plan_structure`

3.6 Le nœud VALIDATE

Le nœud de validation, défini dans `validator.py`, implémente les garde-fous qui empêchent la livraison de plans incohérents à l'utilisateur. La fonction `_validate_plan()` vérifie quatre règles : le plan doit contenir au moins un jour, chaque jour doit comporter au moins un créneau, aucun jour ne doit dépasser 7 créneaux (pour éviter les plannings irréalistes surchargeant la journée), et aucun `activity_id` ne doit apparaître plus d'une fois dans l'ensemble du plan (détection des doublons).

Le mécanisme d'injection d'erreurs repose sur les `SystemMessage` de LangChain. Si des erreurs de validation sont détectées, un message système est ajouté à la liste des messages avec le contenu : “[VALIDATION] Validation issues: Day 2 has duplicate activity ID abc123; Day 3 has too many activities (9). Please fix these issues.” Le champ `is_plan_ready` est remis à `False`, et la fonction de routage `_after_validation()` renvoie l'agent au nœud THINK. Le LLM lit le message d'erreur et corrige le plan lors de son prochain appel d'outil.

Le choix d'injecter les erreurs comme `SystemMessage` plutôt que comme résultat d'outil est un choix de conception délibéré. Les messages système sont traités avec une priorité élevée par les LLMs (instruction de niveau système), tandis que les résultats d'outils peuvent être enfouis dans un historique de messages long. Cette hiérarchie de priorité garantit que le LLM prend les erreurs de validation au sérieux et les corrige effectivement.

3.7 Le nœud RESPOND et la persistance

Le nœud de réponse, défini dans `responder.py`, détermine le type de réponse selon l'état courant : si `needs_more_info` est `True` et `questions` est non nul, le type est "question" ; si `is_plan_ready` est `True` et `is_modification` est `True`, le type est "modification" ; si `is_plan_ready` est `True`, le type est "plan" ; sinon, le type est "text" (réponse conversationnelle simple).

L'extraction du texte final parcourt la liste des messages en ordre inverse pour trouver le dernier `AIMessage` avec un contenu non vide. Le contenu est normalisé via `_normalize_content()` pour gérer le format liste-de-dictionnaires de Gemini. Cette approche est nécessaire car dans un cycle multi-outils, plusieurs `ToolMessage` peuvent suivre le dernier `AIMessage` contenant du texte, et il faut retrouver la bonne source de texte.

La persistance des plans exploite le versionnement. Lorsque le type de réponse est "plan" ou "modification", la fonction `save_plan()` est appelée. Elle désactive d'abord tous les plans actifs existants pour cette conversation (`UPDATE SET isActive=False`), puis détermine le numéro de version en comptant les plans existants, et enfin insère une nouvelle ligne avec `isActive=True` et `version=N+1`. Ce mécanisme de versionnement

permet de récupérer les itérations précédentes d'un plan si l'utilisateur souhaite revenir à une version antérieure.

Les métadonnées sauvegardées avec chaque message assistant comprennent la liste `thinking_steps`, la liste `tools_used`, et le booléen `has_plan`. Ces métadonnées sont stockées dans la colonne JSON `metadata` de la table `PlannerMessage`, permettant au frontend de reconstruire l'affichage des étapes de réflexion à partir de l'historique de conversation sans nécessiter de nouveaux appels à l'agent.

3.8 Le pipeline RAG

L'architecture RAG a été présentée en détail au chapitre 2. Cette section se concentre sur les aspects spécifiques de l'implémentation. Chaque type d'entité dispose d'une fonction dédiée de construction de documents (`_activity_to_document`, `_stay_to_document`, `_yummy_to_document`, `_attraction_to_document`) qui convertit une instance de modèle ORM en un `Document` LlamaIndex. Le document comprend un corps textuel riche incluant tous les champs pertinents (titre, description, catégorie, région, prix, durée, équipements) et un dictionnaire de métadonnées structurées contenant `entity_type`, `entity_id`, `title`, `category`, `region`, `city` et `price`. Ces métadonnées sont utilisées pour le filtrage lors des requêtes.

La détection de changements par hachage optimise la réindexation incrémentale. Lorsqu'une entité est modifiée en base de données, la fonction `upsert_entity()` recalcule le hash MD5 du texte du nouveau document et le compare au hash stocké dans le dictionnaire `_entity_hashes`. Si le hash est identique, le recalcul coûteux des embeddings est évité. Cette optimisation est particulièrement importante car le calcul d'un embedding par le modèle BAAI/bge-m3 prend environ 200 ms par document sur CPU.

La fonction `rag_query()` implémente une stratégie de sur-échantillonnage : le récupérateur demande `top_k × 2` résultats à l'index, puis filtre les résultats par type d'entité, seuil de score et prix maximum, et enfin limite à `top_k`. Ce sur-échantillonnage compense le fait que les filtres de métadonnées dans LlamaIndex utilisent une logique AND qui peut être trop restrictive pour les requêtes multi-types, éliminant des résultats pertinents avant le post-filtrage applicatif.

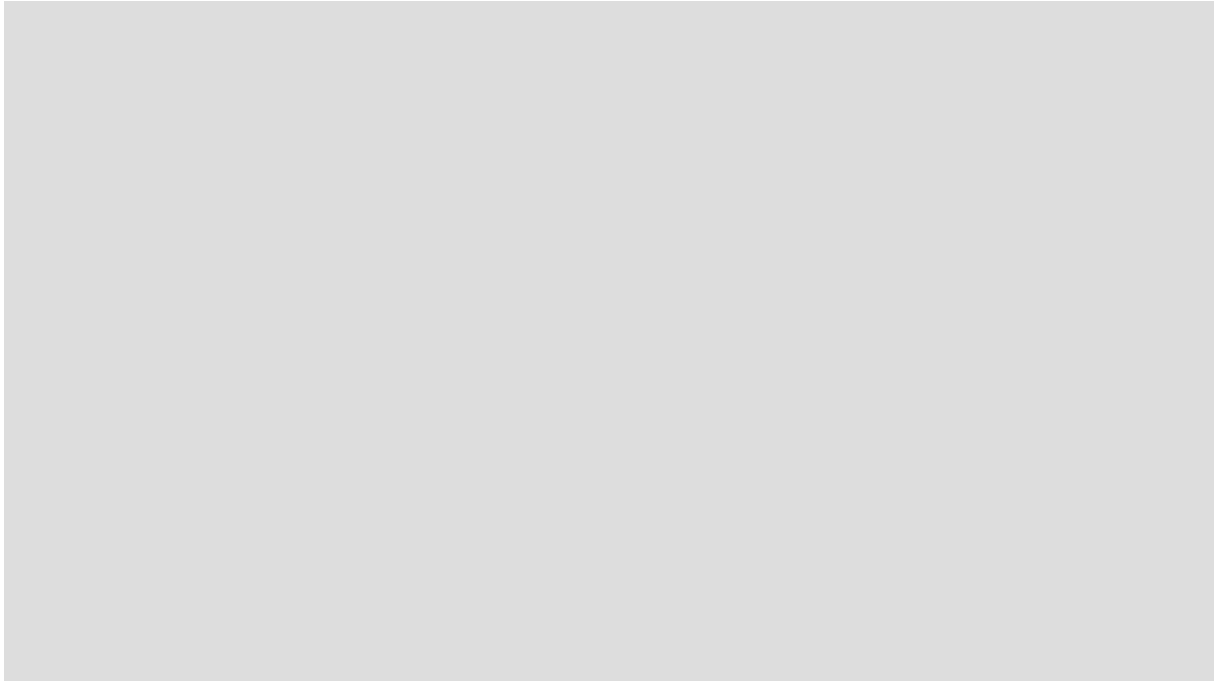


Figure 3.3: Exemple de résultats d'une recherche RAG avec scores de similarité

3.9 Gestion de la mémoire et du contexte

3.9.1 Chargement du contexte

La fonction `load_context()`, définie dans `conversation.py`, ouvre une session de base de données et récupère la `PlannerConversation` avec ses 8 derniers `PlannerMessage`. Chaque message est converti en son type `LangChain` correspondant : les messages de rôle "user" deviennent des `HumanMessage`, les messages "assistant" des `AIMessage`, et les messages "system" des `SystemMessage`. Le plan actif est ensuite récupéré via `get_current_plan()`.

Le choix d'une fenêtre de 8 messages résulte d'un compromis empirique. Charger moins de messages risque de perdre un contexte pertinent (l'utilisateur a dit "supprimez la journée plage" mais cette journée a été mentionnée 12 messages plus tôt et n'est plus visible). Charger plus de messages augmente la consommation de tokens et la latence de chaque appel LLM. Huit messages représentent environ 2 à 4 tours de conversation, couvrant la plupart des scénarios de modification de plans.

Si le champ `contextSummary` de la conversation est non nul (c'est-à-dire qu'un résumé a été généré par le résumeur automatique), un `SystemMessage` est inséré en position 0 de la liste de messages contenant le résumé structuré. Cette injection donne au LLM une vue compressée de l'historique complet de la conversation, même si seulement 8 messages bruts sont chargés. Le résumé étant structuré en JSON (préférences confirmées, contraintes, modifications), il est plus exploitable qu'une simple concaténation de messages anciens.

3.9.2 Résumé automatique

La fonction `summarize_if_needed()`, définie dans `summarizer.py`, vérifie deux conditions avant de déclencher un résumé : le nombre de messages doit dépasser `CONVERSATION_SUMMARY_THRESHOLD` (15 par défaut), et au moins 10 nouveaux messages doivent avoir été ajoutés depuis le dernier résumé. Si un résumé récent existe et que moins de 10 messages se sont accumulés, le résumé existant est conservé tel quel.

Le résumé est produit par un appel LLM dédié (`temperature=0.3`, `json_mode=True`) avec pour instruction de générer un JSON structuré contenant : `trip_overview` (description générale du voyage), `confirmed_preferences` (préférences confirmées par l'utilisateur), `confirmed_constraints` (nombre de jours, budget, taille du groupe, dates), `modifications_made` (liste des modifications demandées), `important_notes` (notes importantes), et `message_count` (nombre de messages résumés). Ce format structuré est plus exploitable par l'agent qu'un résumé en texte libre.

En cas d'échec de l'appel LLM (réseau, quota, JSON invalide), un résumé de base est construit à partir des métadonnées de la conversation (nombre de messages, outils utilisés, types de réponses) sans nécessiter d'appel LLM. Ce mécanisme de repli garantit que la fonction ne lève jamais d'exception et ne bloque jamais le flux principal de l'agent.

3.9.3 Profil utilisateur

L'outil `get_user_profile` collecte l'ensemble des données historiques de l'utilisateur depuis la base de données : les 30 dernières réservations d'activités, les avis publiés, les éléments de la liste de souhaits, et les préférences déclarées. Il calcule la fréquence par catégorie (`favorite_categories` triées par nombre de réservations), les dépenses totales et moyennes, la note moyenne donnée dans les avis, et la liste des activités en favoris. Ces informations composent un dictionnaire de profil riche que le nœud THINK injecte dans le contexte du LLM.

Ce profil influence directement la qualité des recommandations de l'agent : le LLM sait avant de générer un plan que cet utilisateur a réservé des sports nautiques 5 fois, dépense en moyenne 80 TND par jour, possède 3 activités spa dans sa liste de souhaits, et attribue des notes de 5 étoiles aux tours culturels. Cette connaissance du profil permet des recommandations véritablement personnalisées sans recours au moteur LinUCB — l'agent adapte son raisonnement à l'historique comportemental de chaque utilisateur.

3.10 Pipeline rapide de génération de plans

Le pipeline rapide existe en complément de la boucle agentique complète pour répondre à un besoin spécifique : la boucle complète de l'agent nécessite 15 à 40 secondes (appels LLM multiples, exécutions d'outils, boucle de validation), ce qui est acceptable pour une

conversation exploratoire mais excessif lorsque l'utilisateur fournit toutes les informations nécessaires en un seul message structuré (dates, taille du groupe, budget, préférences).

La détection automatique est assurée par la fonction `parse_chat_to_plan_request()`, qui analyse le message de l'utilisateur à la recherche d'informations structurées : dates ou durée en jours, mention de budget, composition du groupe (famille, couple, solo). Si ces éléments sont détectés, un objet `UserPreferences` est construit et le pipeline rapide est invoqué à la place de la boucle agentique, sans que l'utilisateur ne perçoive la différence dans l'interface.

Le pipeline s'exécute en trois phases. La **phase 1** collecte toutes les données en parallèle via `asyncio.gather()` : recherche RAG thématique (`rag_search_for_plan`), prévisions météorologiques (`get_weather`), calcul des distances entre les lieux, et estimation du budget total (`estimate_trip_budget`). La **phase 2** pré-traite les contraintes : filtrage par budget, suppression des activités incompatibles avec la météo, regroupement par proximité géographique. La **phase 3** effectue un unique appel LLM avec toutes les données pré-chargées en contexte pour produire le plan JSON final.

Le compromis est explicite : le pipeline rapide sacrifie la profondeur conversationnelle et la boucle de validation de l'agent complet au profit de la vitesse (5 à 10 secondes au lieu de 15 à 40). Il est approprié pour les utilisateurs expérimentés qui fournissent des informations complètes dès le départ, tandis que l'agent complet est préférable pour les conversations exploratoires où les besoins de l'utilisateur se précisent progressivement.

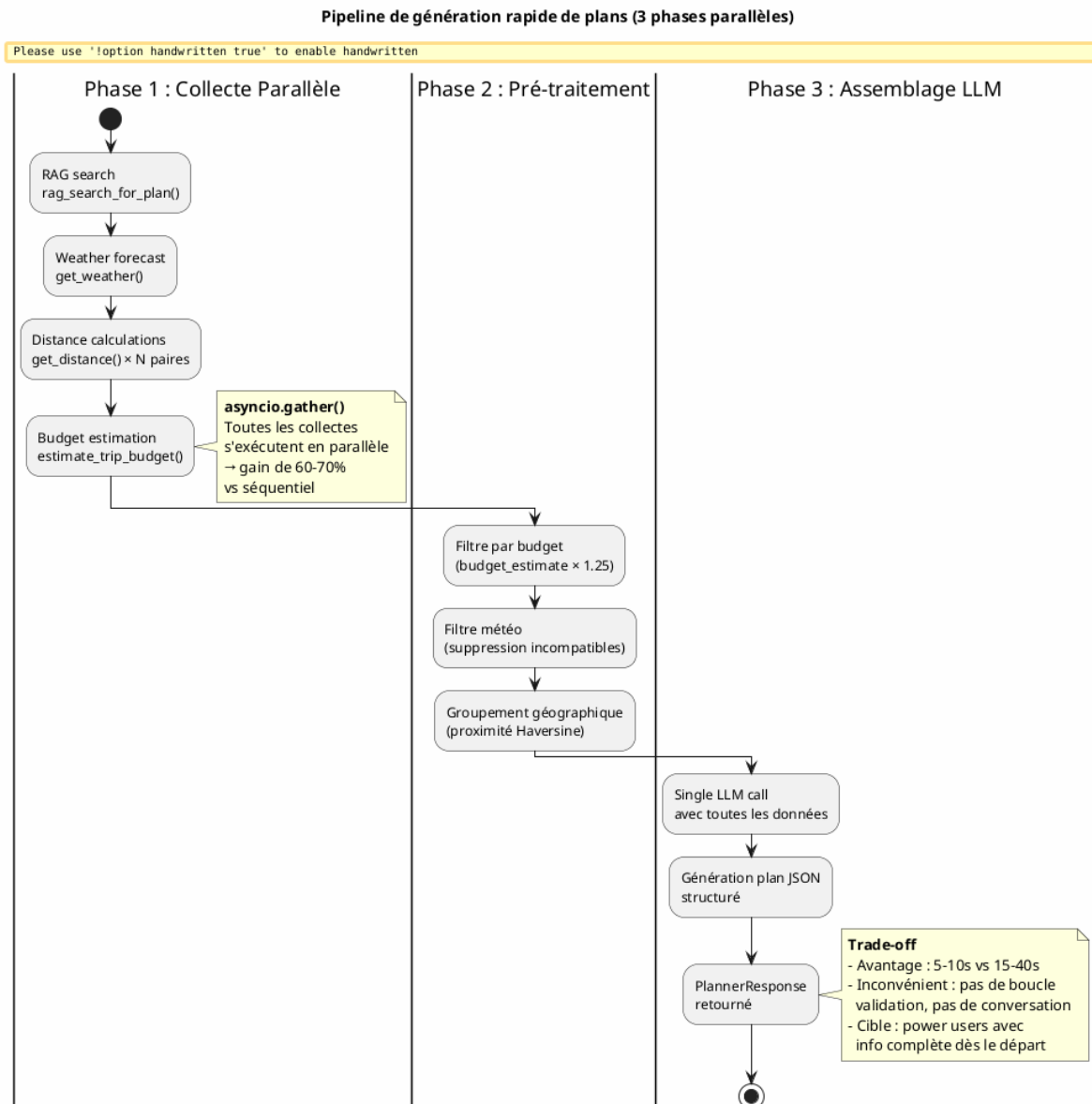


Figure 3.4: Pipeline de génération rapide de plans (3 phases parallèles)

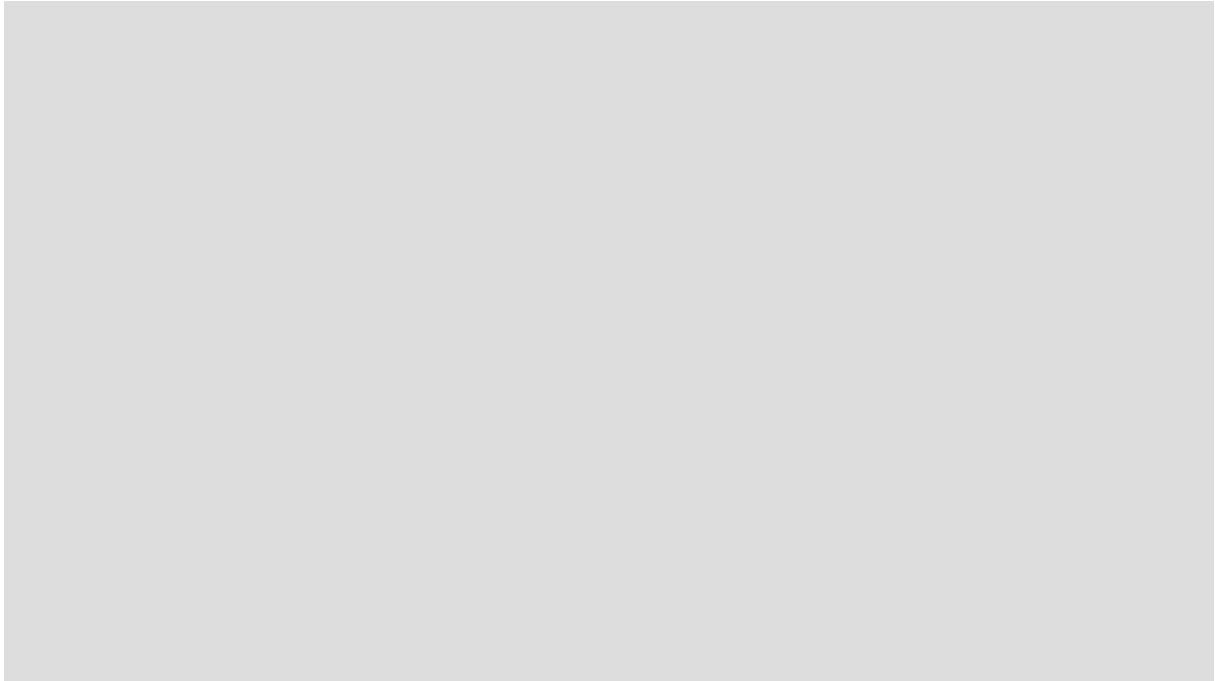


Figure 3.5: Exemple de plan de voyage généré par l'agent Guidni

Conclusion du chapitre

Ce chapitre a présenté l'implémentation complète de l'agent planificateur IA du système Guidni. La machine à états LangGraph avec ses quatre nœuds (THINK, EXECUTE_TOOL, VALIDATE, RESPOND) et ses trois fonctions de routage conditionnel a été détaillée, incluant les mécanismes de sécurité (arrêt forcé à 15 itérations, heuristique de force-stop à 2 itérations avec contexte RAG). Les 18 outils ont été catalogués et les plus complexes décrits en profondeur : **rag_search_for_plan** avec ses quatre recherches parallèles et son garde-fou de désambiguïsation des types d'entités, et **enrich_activity** avec son cache d'intelligence IA. Le pipeline RAG, le système de mémoire conversationnelle (fenêtre de 8 messages, résumé automatique au seuil de 15 messages, profil utilisateur), et le pipeline rapide de génération de plans en trois phases ont complété la présentation.

Le chapitre suivant présente l'implémentation du second sous-système : le moteur de recommandation Hybrid LinUCB, avec son ingénierie des caractéristiques, son cycle d'apprentissage en ligne, et son pipeline de classement en neuf étapes.

Chapter 4

Sprint 2 — Implémentation du Moteur de Recommandation Hybrid LinUCB

4.1 Introduction

Ce chapitre présente l'implémentation complète du moteur de recommandation Hybrid LinUCB du système Guidni — un système de bandits multi-bras contextuel qui apprend en temps réel à partir du comportement des utilisateurs. Contrairement aux systèmes de recommandation collaboratifs classiques qui nécessitent une phase d'entraînement hors ligne sur un corpus historique, le moteur LinUCB met à jour ses paramètres à chaque interaction utilisateur, s'adaptant continuellement aux préférences émergentes et aux tendances saisonnières du marché touristique de Djerba.

L'implémentation est organisée en cinq couches fonctionnelles : l'ingénierie des caractéristiques (`context.py`) qui transforme les signaux bruts de session en vecteurs de caractéristiques normalisés, le moteur de scoring LinUCB (`linucb.py`) qui calcule les scores d'exploitation et d'exploration, le pipeline de classement post-LinUCB (`ranker.py`) qui applique dix étapes de filtrage et d'ajustement, la couche de données runtime (`db.py` et `recommendation_router.py`) qui gère le cache matriciel en RAM et le traitement par lots des événements, et enfin le tracker JavaScript client (`tracker.js`) couplé aux jobs nocturnes (`nightly_job.py` et `scheduler.py`).

4.2 Ingénierie des caractéristiques

4.2.1 Le vecteur utilisateur x_{user}

La fonction `build_user_vector()` reçoit un objet `UserContext` (dataclass contenant les signaux de session et les données de profil historique) et produit un vecteur numpy de forme (9,) en float64. Chaque dimension capture un aspect distinct du contexte

utilisateur, combinant des signaux temporels, comportementaux et historiques dans un espace de caractéristiques uniformément normalisé à $[0, 1]$.

L’encodage cyclique du temps constitue les quatre premières dimensions. L’heure et le mois sont encodés par paires sinus/cosinus : $x[0] = \sin(2\pi \cdot h/24)$, $x[1] = \cos(2\pi \cdot h/24)$, $x[2] = \sin(2\pi \cdot m/12)$, $x[3] = \cos(2\pi \cdot m/12)$. Ce choix est fondamental car un encodage linéaire brut rendrait l’heure 23 et l’heure 0 maximalement distantes alors qu’elles sont toutes deux “nocturnes”, et décembre et janvier maximalement distants alors qu’ils sont tous deux “hivernaux”. L’encodage sinus/cosinus préserve la proximité cyclique : $\sin(2\pi \cdot 23/24) \approx \sin(2\pi \cdot 1/24)$.

Les caractéristiques comportementales occupent les dimensions 4 à 7. La profondeur de défilement (`scroll_depth`) est directement bornée à $[0, 1]$ par `np.clip()`. Le temps de séjour est normalisé par transformation logarithmique : $x[5] = \text{clip}(\ln(1 + t)/\ln(1 + 300), 0, 1)$, où la borne de 300 secondes assure que la différence entre 5 et 30 secondes (signal fort de désintérêt vs intérêt modéré) est beaucoup plus grande que la différence entre 300 et 600 secondes (les deux indiquent un intérêt élevé). L’affinité de prix est calculée comme $x[6] = \text{clip}(\text{avg_booking_price}/2000, 0, 1)$, où 2000 TND est le prix maximum de la plateforme. L’affinité maximale de tags est $x[7] = \max(\text{tag_affinity.values}())$ si un profil existe, 0 sinon.

La dimension 8 encode le type de voyage : `family`= 1.0, `couple`= 0.67, `solo`= 0.33, `unknown`= 0.0. Cet ordinal reflète la taille croissante du groupe social et les patrons de consommation associés (les familles dépensent plus en activités de groupe, les solos en expériences individuelles). Tous les vecteurs sont nettoyés par `np.nan_to_num()` qui remplace les NaN et inf par 0.0 avant retour, garantissant qu’aucune valeur non finie ne corrompt les calculs matriciels.

4.2.2 Le vocabulaire de tags et le vecteur x_{tags}

Le vocabulaire `TAG_VOCAB` est une liste fixe de **exactement 40 tags** en ordre alphabétique strict, de `activity_indoor` à `wifi`. L’assertion `assert D_TAGS == 40` est vérifiée à l’importation du module, empêchant tout démarrage de l’application si le vocabulaire a été altéré. Un dictionnaire de correspondance inverse `_TAG_INDEX` pré-calculé à l’importation (`{tag: i}`) permet le lookup en $O(1)$ lors de la construction du vecteur.

L’immutabilité du vocabulaire est critique : la position de chaque tag dans cette liste détermine son index dans le vecteur x_{tags} et par conséquent ses lignes dans la matrice A . Si le vocabulaire change après le premier déploiement (même un simple réordonnancement), la structure entière de la matrice A (360×360) est invalidée car ses coefficients encodent des associations apprises pour des paires spécifiques (`caractéristique_utilisateur`, `tag`) à des positions spécifiques. La vérification de sécurité `vocab_hash` (SHA-256 du vocabulaire trié) dans `SystemConfig` empêche ce scénario : au chargement des matrices depuis la base

de données, le hash stocké est comparé au hash courant, et une `RuntimeError` est levée en cas de discordance.

L’encodage binaire est direct : x_{tags} est un vecteur de longueur 40 où $x_{tags}[i] = 1.0$ si `TAG_VOCAB[i]` est présent dans la liste de tags de l’annonce, 0.0 sinon. Les tags d’une annonce proviennent de la table `ListingTag` (peuplée par l’extracteur LLM) ou du champ `BanditArm.tags` en repli.

4.2.3 Le produit de Kronecker ϕ

Le produit de Kronecker $\phi = \text{np.kron}(x_{user}, x_{tags})$ produit un vecteur de longueur $9 \times 40 = 360$ où chaque élément $\phi[i \cdot 40 + j] = x_{user}[i] \times x_{tags}[j]$ encode l’interaction entre la i^e caractéristique utilisateur et le j^e tag de l’annonce. La dimension de ϕ est définie par la constante `D_PHI = 360`, vérifiée par assertion à l’importation.

Cette expansion capture les effets d’interaction que ni les caractéristiques utilisateur seules ni les tags seuls ne peuvent modéliser. La dimension correspondant à $(\text{type_famille} \times \text{tag_pool})$ aura un coefficient θ élevé si les familles qui voient des annonces piscine tendent à cliquer et réserver. La dimension $(\text{heure_matin} \times \text{tag_best_evening})$ aura un coefficient faible ou négatif si les utilisateurs matinaux cliquent rarement sur les activités de soirée. Ces effets d’interaction sont la raison fondamentale du choix du modèle Hybrid LinUCB : un modèle linéaire simple sur la concaténation $[x_{user}; x_{tags}]$ ne capturerait que les effets marginaux.

La fonction `validate_phi()` vérifie que la forme est $(360,)$, remplace tout NaN ou inf par 0.0 et journalise un WARNING. Ce nettoyage garantit qu’un vecteur de caractéristiques malformé ne corrompt jamais la mise à jour de la matrice A , car $A += \text{outer}(\phi, \phi)$ propagerait les NaN à toute la matrice.

Table 4.1: Exemple de construction du vecteur ϕ pour un cas concret

Composant	Valeurs
Utilisateur	Famille, 10h, août, budget ≈ 300 TND, tag_affinity = {pool: 0.9, beach_access: 0.8}
Annonce	Villa avec piscine, tags = [pool, family, luxury, wifi]
x_{user} (9)	$[\sin(2\pi \cdot 10/24), \cos(2\pi \cdot 10/24), \sin(2\pi \cdot 8/12), \cos(2\pi \cdot 8/12), 0.0, 0.0, 0.15, 0.9, 1.0]$
x_{tags} (40)	$[0, \dots, 0, \underbrace{1.0}_{\text{family}}, 0, \dots, \underbrace{1.0}_{\text{luxury}}, 0, \dots, \underbrace{1.0}_{\text{pool}}, 0, \dots, \underbrace{1.0}_{\text{wifi}}]$
ϕ (360)	$\phi[\text{trip_type} \cdot 40 + \text{pool}] = 1.0 \times 1.0 = 1.0$ (forte interaction famille $\times$ piscine)

4.3 Moteur de scoring LinUCB

4.3.1 Score UCB et paramètres

Le calcul du score UCB dans `score_arms()` procède en six étapes. (1) Le vecteur de paramètres estimé est calculé par $\hat{\theta} = \text{solve}(A, b)$ via la résolution de moindres carrés ; (2) l'exploitation est le produit scalaire $\text{exploitation} = \hat{\theta}^\top \phi$, représentant la récompense prédite pour cette paire contexte-bras ; (3) la variance est calculée par $\text{variance} = \phi^\top A^{-1} \phi$ via un second appel à `solve(A, phi)`, mesurant l'incertitude — élevée lorsque A n'a pas été mis à jour souvent dans la direction de ϕ ; (4) le bonus UCB est $\text{ucb_bonus} = \alpha \cdot \sqrt{\max(\text{variance}, 0)}$ avec $\alpha = 0.3$; (5) si `is_underexposed` est `True`, $\text{ucb_bonus} += 0.15$; (6) le score final est $\text{score} = (\text{exploitation} + \text{ucb_bonus}) \times \text{position_discount}$, où le discount de position est $1/\log_2(\text{rank} + 1)$ si une information de rang antérieur est disponible.

Le choix de `np.linalg.solve()` plutôt que `np.linalg.inv()` est critique pour la stabilité numérique. Le calcul explicite de l'inverse matricielle est numériquement instable pour les matrices quasi-singulières et computationnellement plus coûteux ($O(n^3)$ avec un facteur constant plus élevé). La fonction `solve(A, b)` calcule $A^{-1}b$ directement par décomposition LU, qui est plus stable et plus efficace. En cas d'échec (matrice singulière), un repli vers `np.linalg.lstsq()` est utilisé.

La calibration $\alpha = 0.3$ résulte d'un compromis adapté au domaine du tourisme. Un α plus élevé augmente l'exploration (recommandations plus diversifiées mais potentiellement moins pertinentes à court terme), un α plus faible augmente l'exploitation (meilleures performances à court terme mais risque de manquer de nouvelles annonces prometteuses). La valeur 0.3 a été choisie car les sessions touristiques sont courtes (typiquement 10 à 30 minutes) et les nouvelles annonces nécessitent une exposition rapide pour accumuler suffisamment de données d'apprentissage.

La dataclass `ScoredArm` retournée expose le détail du calcul : `exploitation`, `ucb_bonus`, `position_discount`, `impressions`, `conversions` et `commission_rate`. La décomposition exploitation/bonus est rendue accessible via la variable d'environnement `EXPOSE_SCORES=true` pour le débogage et l'analyse A/B.

4.3.2 Mise à jour des matrices et récompenses

La règle de mise à jour en ligne est implémentée dans la fonction `update()` : après observation d'une récompense r pour un contexte ϕ , la matrice est mise à jour par $A += \phi\phi^\top$ (le produit extérieur ajoute la contribution de cette observation) et le vecteur par $b += r \cdot \phi$. Cette mise à jour correspond à la descente de gradient en ligne pour la régression linéaire avec régularisation L_2 (l'initialisation identitaire $A = I$ agit comme régularisateur).

La hiérarchie de récompenses est définie dans le dictionnaire `REWARDS` : `impression=0.0`, `click=0.3`, `dwell_30s=0.5`, `dwell_60s=0.7`, `gallery_swipe=0.4`, `scroll_reviews=0.6`, `wishlist=0.85`, `reservation=1.0`, `quick_exit=-0.2`. Le type `dwell_exit` est traité spécialement : la fonction vérifie `dwell_seconds` pour retourner 0.1 ($< 30s$), 0.5 (30–59s), ou 0.7 ($\geq 60s$) plutôt qu’une valeur fixe.

Le mécanisme de récompenses delta par session empêche le double comptage. Le dictionnaire `session_rewards` trace la récompense maximale reçue par annonce dans la session courante. Lorsqu’un nouvel événement arrive, `compute_reward()` retourne $\max(r_{new}, r_{max}) - r_{max}$ (le delta). Un utilisateur qui clique (0.3), consulte longuement (0.7), puis ajoute aux favoris (0.85) une annonce contribue un delta total de 0.85 et non $0.3 + 0.7 + 0.85 = 1.85$, évitant l’inflation des récompenses.

La fonction `is_matrix_healthy()` vérifie trois conditions : `np.all(np.isfinite(A))` (pas de NaN/inf), `shape == (360, 360)`, et `np.linalg.eigvalsh(A).min() > 0` (définie positive). Si la matrice est malsaine (possible après des erreurs numériques ou de très nombreuses mises à jour sans réinitialisation), le système journalise un avertissement `CRITICAL` et retourne des matrices fraîches ($A = I$, $b = 0$) pour récupérer automatiquement.

4.4 Pipeline de classement post-LinUCB

Après que le moteur LinUCB a produit les scores bruts, dix étapes séquentielles de post-traitement affinent le classement pour respecter les contraintes métier et améliorer l’expérience utilisateur. Les étapes opèrent sur des objets `ArmState` (avant scoring) et `ScoredArm` (après scoring), et sont orchestrées par la fonction `rank()` définie dans `ranker.py`.

La fonction `rank()` reçoit la liste complète des bras, les matrices A et b , le `UserContext`, et retourne une liste de `ScoredArm` triés. Les dix étapes sont décrites ci-dessous dans leur ordre exact d’exécution.

Étape 1 — Correction de popularité (`apply_popularity_correction`). Tout bras avec `impressions < 50` est marqué `is_underexposed=True`. Ce drapeau est lu par `score_arms()` pour ajouter le bonus +0.15, garantissant que les nouvelles annonces reçoivent suffisamment d’exposition pour que le LinUCB apprenne leur vraie valeur avant d’être enfouies par les annonces bien établies.

Étape 2 — Filtre temporel (`apply_time_filter`). Les annonces taguées `best_evening` ou `bar` sont supprimées le matin (6h–12h), et les annonces `best_morning` sont supprimées le soir (20h–23h). Ce filtre empêche de recommander un cocktail bar à une famille à 9h ou une séance de yoga au lever du soleil à un utilisateur naviguant à minuit, améliorant la pertinence à coût nul en apprentissage automatique.

Étape 3 — Filtre budgétaire (`apply_budget_filter`). Si `budget_estimate` est fourni, les annonces dont le prix dépasse $\text{budget} \times 1.25$ sont supprimées. La tolérance

de 25% évite de rejeter des annonces légèrement au-dessus du budget que l'utilisateur pourrait encore considérer, tout en filtrant les options clairement inabordables.

Étape 4 — Construction de ϕ par bras. Pour chaque bras survivant après les filtres, `build_phi(build_user_vector(ctx), arm.tags)` calcule le vecteur de contexte de 360 dimensions. Cette étape est la plus coûteuse en calcul (opérations matricielles) mais opère uniquement sur les bras survivants après réduction par les filtres.

Étape 5 — Scoring LinUCB (`score_arms`). Calcule les scores UCB pour tous les bras restants en utilisant les matrices A et b cachées en RAM. Retourne une liste de `ScoredArm` triée par score décroissant.

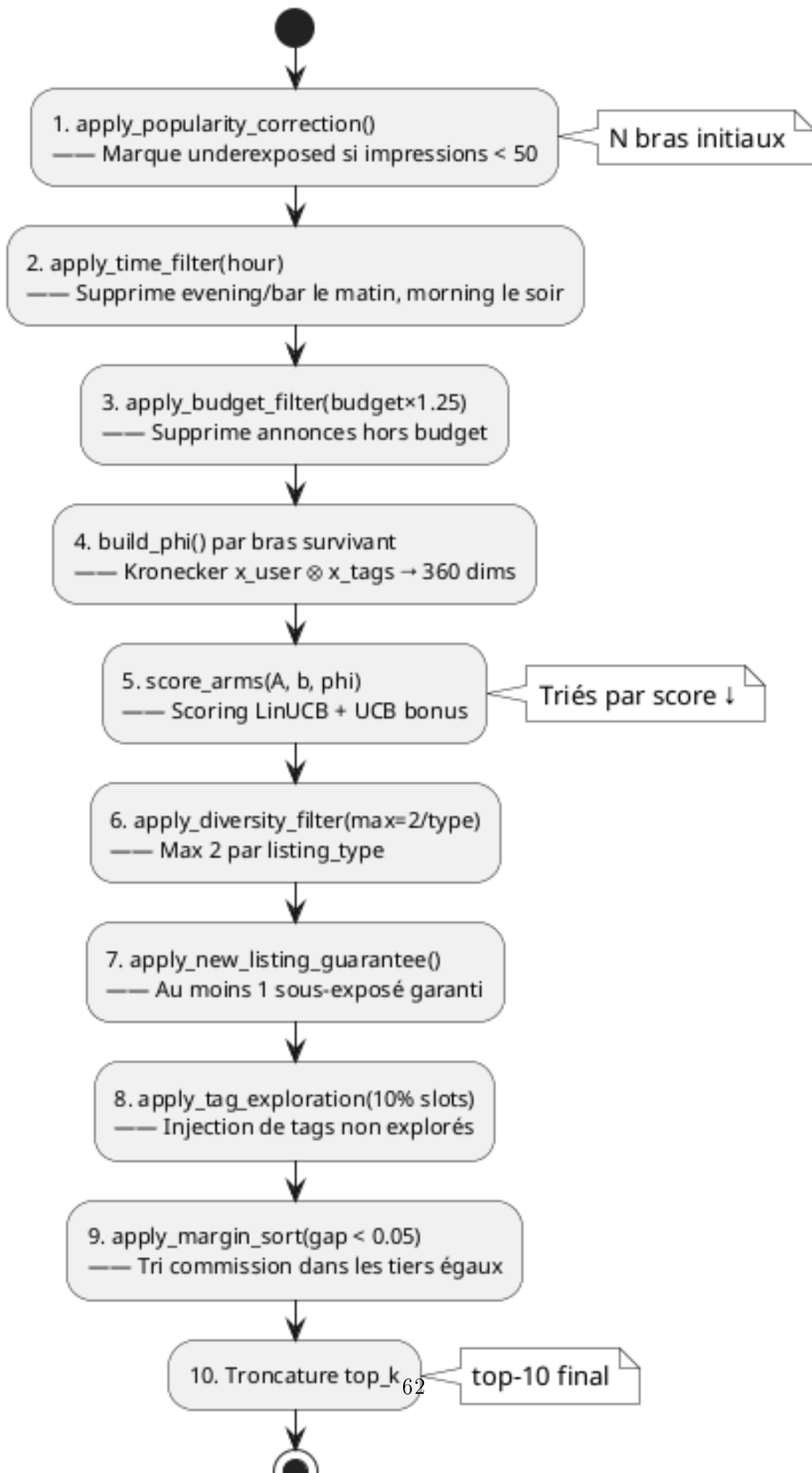
Étape 6 — Filtre de diversité (`apply_diversity_filter`). Parcourt la liste classée du haut vers le bas, comptant les bras acceptés par `listing_type`. Tout bras dont le type a déjà 2 bras acceptés est ignoré. Ce filtre empêche le top-10 d'être entièrement composé d'hôtels (si les hôtels scorent bien dans cette session) et garantit que l'utilisateur voit une variété de types d'annonces.

Étape 7 — Garantie de nouvelle annonce (`apply_new_listing_guarantee`). Si le résultat top- k ne contient aucun bras avec `impressions` < 50 , le bras sous-exposé avec le score le plus élevé de la liste complète est inséré en dernière position. Cette garantie assure que chaque session de recommandation introduit au moins une nouvelle annonce, maintenant l'exploration même lorsque le classeur exploiterait uniquement les annonces connues.

Étape 8 — Exploration de tags (`apply_tag_exploration`). Identifie les catégories de tags où l'utilisateur a une faible affinité (valeur < 0.2 ou absente du profil), trouve les bras les mieux scorés de ces catégories qui ne sont pas déjà dans le résultat, et remplace 10% des créneaux finaux (minimum 1) avec ces bras d'exploration. Ce mécanisme prévient les bulles de filtrage et introduit l'utilisateur à des catégories d'annonces qu'il n'a pas encore essayées.

Étape 9 — Tri par marge (`apply_margin_sort`). Au sein de groupes de bras dont les scores diffèrent de moins de 0.05 (un "tier" de qualité essentiellement équivalente), trie par `commission_rate` décroissant. Ce mécanisme maximise le revenu de la plateforme lorsque plusieurs annonces sont de qualité de recommandation équivalente pour l'utilisateur, sans impact significatif sur l'expérience utilisateur.

Étape 10 — Troncature top- k . La liste finale est tronquée à `top_k` (défaut 10) et retournée au `recommendation_router`.



4.5 Extracteur de tags par LLM

L'extraction de tags par LLM répond à un besoin fondamental : les 40 tags du `TAG_VOCAB` doivent être assignés à chaque annonce (hôtel, activité, restaurant, location, transfert) à partir de sa description textuelle. Le taggage manuel de centaines d'annonces est impraticable, et les descriptions contiennent déjà des informations riches ("suite avec vue sur la mer, piscine privée, idéale pour couples") qui correspondent directement aux tags (`sea_view`, `pool`, `romantic`).

Le prompt système défini dans `tag_extractor.py` instruit le LLM configuré de retourner **uniquement** du JSON valide (pas de markdown, pas de préambule) contenant trois champs : `tags` (liste de chaînes provenant exclusivement du `TAG_VOCAB`, aucun tag personnalisé accepté), `confidence` (flottant 0–1), et `language_detected` (`darija`/`french`/`english`/`arabic`). L'étape de validation filtre strictement les tags : seuls les tags présents dans `set(TAG_VOCAB)` sont conservés, les tags hallucinés sont rejetés. Si aucun tag valide ne subsiste, le repli `["all_seasons"]` est appliqué.

Le traitement par lots opère en tranches de 10 annonces. Les annonces manquantes dans `ListingTag` sont identifiées par `LEFT JOIN` sur les cinq tables d'entités (`Activity`, `Stay`, `Restaurant`, `Rental`, `Transfer`), le LLM extrait les tags pour chacune, les résultats sont insérés dans `ListingTag` par `upsert`, et les tags de `BanditArmUCB` sont mis à jour pour toutes les zones de localisation de cette annonce. Un délai de 1 seconde entre les lots prévient la saturation de l'API LLM.

Les mesures de fiabilité comprennent le décorateur `@retry` de la bibliothèque `tenacity` avec un maximum de 3 tentatives et un backoff exponentiel (`min=2s`, `max=30s`) pour gérer les erreurs transitoires de l'API. En cas d'échec permanent (après 3 tentatives), la fonction `_extract_with_fallback()` retourne le tag de repli `["all_seasons"]` avec une confiance de 0.0, garantissant que chaque annonce possède toujours au moins un tag valide.

4.6 Couche de données runtime

4.6.1 Cache matriciel en RAM

La décision architecturale fondamentale est de stocker la matrice A (360×360 , `float64`) et le vecteur b (360 , `float64`) en RAM au niveau du module Python. La fonction `init_matrix_cache()` charge les matrices depuis la base de données une seule fois au démarrage de l'application et les stocke dans les variables `_GLOBAL_A` et `_GLOBAL_B`. Toutes les lectures ultérieures lors du scoring utilisent uniquement la RAM — jamais la base de données.

Cette stratégie est critique pour la performance : une matrice 360×360 en `float64` occupe environ 1 Mo en mémoire. La désérialisation depuis le JSON PostgreSQL à chaque requête de recommandation ajouterait 10 à 50 ms par requête. Avec le cache en RAM, le

scoring opère à la vitesse mémoire (< 1 ms). Le surcoût mémoire est négligeable comparé au gain de latence.

La sécurité thread est assurée par un `threading.Lock()` (`_matrix_lock`). La fonction `get_cached_matrices()` retourne des copies (`A.copy()`, `b.copy()`) pour que les opérations de scoring en cours utilisent un instantané cohérent tandis que les opérations de mise à jour progressent concurremment.

La persistance est planifiée toutes les 2 minutes via `APScheduler`. La fonction `persist_matrices_to` prend un instantané des matrices courantes sous le verrou, puis écrit de manière asynchrone dans la table `LinUCBGlobal` avec un `upsert` qui incrémente `n_updates`. À l'arrêt de l'application, un flush final est effectué dans le gestionnaire de contexte du `lifespan` FastAPI.

4.6.2 Traitement par lots d'événements

La fonction `process_event_batch()` traite une liste d'événements comportementaux reçus du tracker JavaScript en un seul appel. Pour chaque événement, elle calcule la récompense delta via `compute_reward()`, met à jour les matrices en RAM ($A += \phi\phi^T$, $b += r \cdot \phi$), journalise l'événement dans `RecommendationEventUCB`, et met à jour les compteurs du bras (`impressions`, `clicks`, `wishlists`, `conversions`).

La contrainte fondamentale est que les mises à jour de matrices se font **uniquement en RAM** pendant `process_event_batch()` ; la base de données n'est jamais écrite pour les matrices dans cette fonction. Cette séparation garantit que le chemin de traitement des requêtes de recommandation n'a aucune latence d'écriture en base de données.

La limitation de débit `_check_rate_limit()` utilise un dictionnaire en mémoire pour compter les événements par annonce par session (maximum 10) et le total par session (maximum 100). Les sessions inactives depuis plus de 2 heures sont nettoyées paresseusement. Ce mécanisme empêche les abus de scripts automatisés qui pourraient corrompre les matrices en envoyant des milliers de signaux de récompense artificiels.

4.6.3 Cache des bras

Le cache des bras optimise les requêtes fréquentes. La fonction `get_arms_for_zone()` effectue une requête complexe (JOIN entre `BanditArmUCB`, `ListingTag`, `Stay`, `Activity`, `Restaurant` et autres tables) qui prend 5 à 20 ms. Les résultats sont cachés pendant 30 secondes avec une clé composée de `location_zone + sorted(listing_types)`. Le cache stocke uniquement les données brutes des bras, pas les résultats classés.

Les résultats classés finaux ne sont jamais cachés car ils sont personnalisés (différents pour chaque utilisateur et session). Seules les données brutes des bras (qui changent lentement — les annonces ne sont pas ajoutées toutes les secondes) sont cachées. Lorsqu'un

événement de réservation est reçu, le cache est invalidé immédiatement pour la zone concernée.

4.7 Tracker JavaScript

4.7.1 Architecture et initialisation

Le tracker est une classe `RecommendationTracker` instanciée avec `baseUrl`, `sessionId` et un `userId` optionnel. Elle maintient une file d'événements interne (`_queue`), l'état de session (`_sessionState`), une carte de récompenses de session (`_sessionRewards`), et un minuteur de flush (`setInterval` toutes les 15 secondes). L'architecture est sans dépendance externe (vanilla JavaScript), compatible ESM et CommonJS.

Le suivi de l'état de session capture en temps réel : la profondeur de défilement (mise à jour sur les événements `scroll` avec un listener passif), la géolocalisation (demandée une fois à l'initialisation via `navigator.geolocation`), la gamme de prix consultée (mise à jour à chaque impression pour calculer le segment budgétaire), le type d'appareil (détecté depuis le *user agent* : mobile/tablet/desktop), et l'heure et le mois d'entrée (capturés à l'instanciation).

Les méthodes publiques de suivi couvrent l'ensemble du parcours utilisateur : `trackImpression` (met à jour la gamme de prix, met l'événement en file), `trackClick` (enregistre le temps d'entrée, met à jour la séquence de clics), `trackPageExit` (calcule le temps de séjour depuis le temps d'entrée stocké, produit `quick_exit` ou `dwell_exit`), `trackGallerySwipe` (met en file si `swipeCount` ≥ 3), `trackScrollToReviews`, `trackWishlist`, et `trackReservation` (déclenche un flush immédiat pour les signaux de haute valeur).

4.7.2 Stratégie de flush et beacon

La fonction `_flush()` est appelée toutes les 15 secondes ou lorsque la file atteint 10 événements. Elle envoie un `POST /api/recommendations/events` avec la charge complète incluant l'état de session. En cas de succès, `_sessionRewards` est mis à jour depuis la réponse du serveur (crucial pour le calcul de récompenses delta côté serveur). En cas d'échec réseau, les événements sont remis en file pour le prochain cycle de flush.

La fonction `_flushBeacon()` est utilisée **uniquement** pour les événements de fermeture de page (`beforeunload` et `visibilitychange` en mode `hidden`). Elle utilise `navigator.sendBeacon()` avec un payload `Blob` : le navigateur garantit la livraison même si la page se ferme, contrairement à `fetch` qui serait annulé. Aucune réponse n'est reçue (acceptable puisque la session se termine).

La protection contre la limite de taille du beacon est implémentée : `sendBeacon` a une limite imposée par le navigateur d'environ 64 Ko. Si la file dépasse 20 événements,

seuls les 20 derniers sont envoyés via beacon (`_queue.slice(-20)`), les plus anciens étant abandonnés (acceptable car moins informatifs que les événements récents).

Le flux de fusion d'identité `onLogin()` procède en trois étapes : (1) flush de la file courante via beacon avant tout changement d'identité, (2) `POST /api/recommendations/merge-session` pour lier le `sessionId` anonyme au `userId` désormais connu (mise à jour de `RecommendationEvent.userId` pour les événements anonymes passés), (3) mise à jour de `_userId` indépendamment du succès de la fusion.

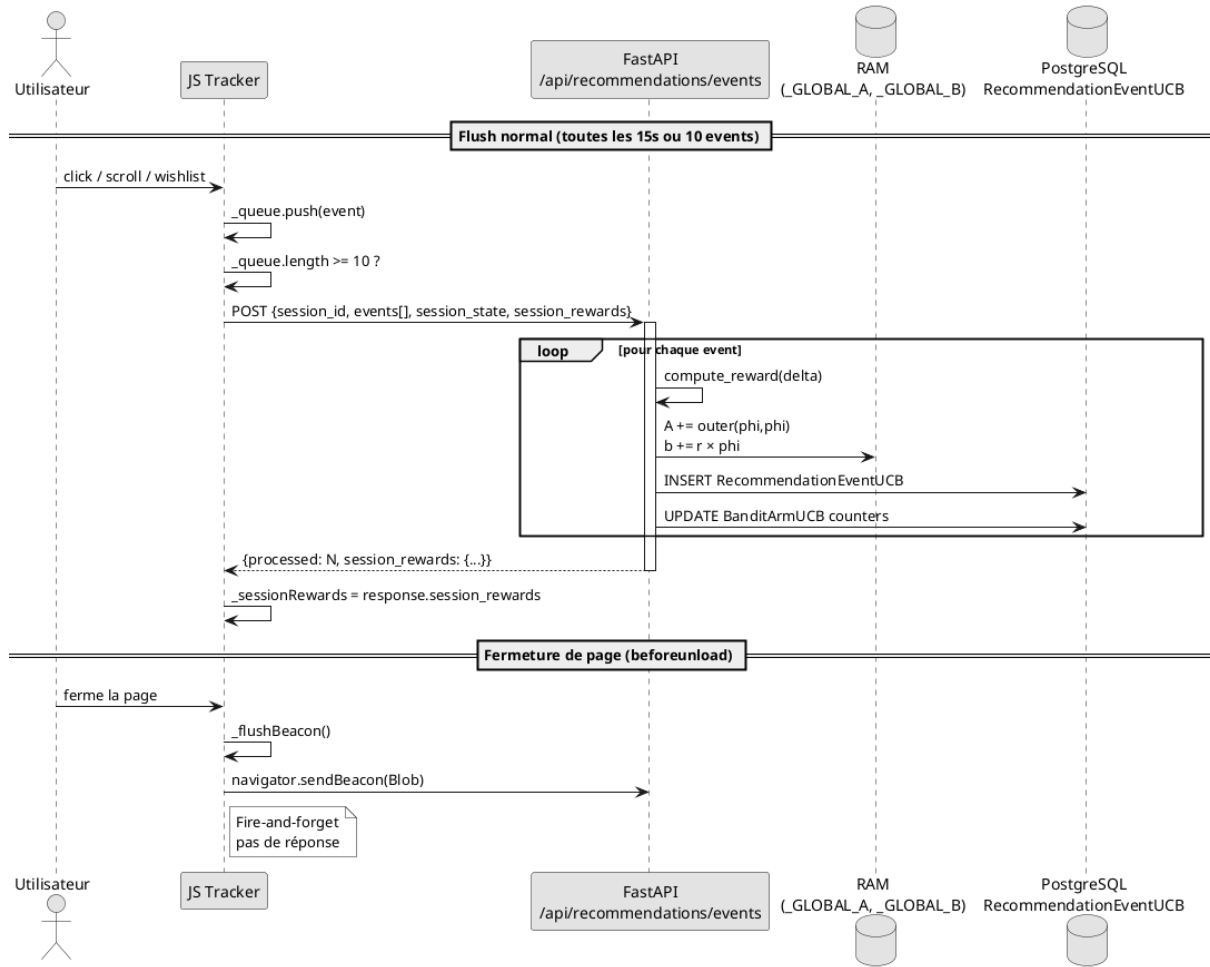


Figure 4.2: Flux de collecte d'événements et mise à jour des matrices LinUCB

4.8 Extracteur de tags LLM et jobs nocturnes

4.8.1 Mise à jour des profils utilisateurs

Le job `run_profile_update_job()` s'exécute chaque nuit à 3h00 (fuseau Africa/Tunis). Il traite uniquement les utilisateurs ayant généré des événements de recommandation dans les dernières 24 heures, rendant le traitement efficace et ciblé. Les utilisateurs inactifs ne sont pas retraités inutilement.

Pour chaque événement avec récompense positive, les tags de l'annonce sont récupérés. Pour chaque tag dans cette annonce, l'affinité est mise à jour selon : $\text{affinity}[\text{tag}] += r \times e^{-\lambda \cdot \Delta t}$ où $\lambda = \ln(2)/30$ (demi-vie de 30 jours : un événement de 30 jours contribue à 50% d'un événement frais). Après traitement de tous les événements, les valeurs d'affinité sont normalisées à $[0, 1]$ par division par la valeur maximale.

Les métriques dérivées sont calculées à partir des événements de réservation uniquement : `avg_booking_price` et `price_std_dev` sont extraites des événements de type `reservation` ; `dominant_trip_type` est déterminé par le type de voyage le plus fréquent parmi les 10 dernières réservations. Le profil complet est inséré ou mis à jour dans la table `UserProfile` par `upsert`.

4.8.2 Gestion de la dérive conceptuelle

La dérive conceptuelle dans le contexte des recommandations touristiques se manifeste de plusieurs façons : les préférences des utilisateurs évoluent avec les saisons (activités de plage en été, activités culturelles intérieures en hiver), avec les événements globaux, et avec la croissance de la plateforme (nouvelles catégories d'annonces). Une matrice LinUCB entraînée purement sur des données historiques finira par sur-ajuster les anciens patrons.

Le mécanisme de réinitialisation partielle est déclenché tous les 90 jours : $A_{new} = 0.7 \cdot A + 0.3 \cdot I$ et $b_{new} = 0.7 \cdot b$. Le facteur $\lambda_{decay} = 0.7$ oublie progressivement les anciennes données tout en conservant la structure apprise, et réintroduit de l'incertitude d'exploration en ajoutant 30% de la matrice identité. Cette approche est plus douce qu'une réinitialisation complète qui perdrait tout le savoir accumulé.

La fonction `run_concept_drift_check()` vérifie la condition (aujourd'hui—dernière réinitialisation) 90 jours en lisant `n_updates` et `updated_at` de `LinUCBGlobal`. Si déclenchée, les matrices sont chargées, la réinitialisation partielle appliquée, et les matrices sauvegardées. Le job s'exécute le 1^{er} de chaque mois à 4h00 via `APScheduler`.

4.8.3 Planificateur de tâches

Le planificateur utilise `BackgroundScheduler` d'`APScheduler` avec un `ThreadPoolExecutor(max_workers=10)` et le fuseau horaire `Africa/Tunis` appliqué à tous les jobs cron. Quatre jobs sont enregistrés : la mise à jour des profils (cron quotidien 3h00, `misfire_grace_time=3600s`), la vérification de dérive (cron mensuel le 1^{er} à 4h00), le rafraîchissement des tags (cron bimensuel le 15 à 2h00), et la persistance des matrices (intervalle de 2 minutes).

Tous les jobs ont `max_instances=1` pour empêcher les exécutions chevauchantes si un job dure plus longtemps que son intervalle. Le planificateur démarre dans le contexte `lifespan` de `FastAPI` et est arrêté gracieusement (`wait=False`) à l'arrêt de l'application.

Chaque job est encapsulé dans une fonction `_safe_*` qui capture toutes les exceptions et les journalise, empêchant un job défaillant de bloquer les autres.

Table 4.2: Planning des jobs nocturnes APScheduler

Job	Type	Heure	Fréquence	Durée	Impact
profile_update	cron	3:00	Quotidien	30s–5min	Maj UserProfile actifs
concept_drift	cron	4:00	1 ^{er} /mois	< 5s	Reset partiel A,b
tag_refresh	cron	2:00	15/mois	5–30min	Re-extraction tags
matrix_persist	interval	—	2 min	< 1s	Flush RAM→DB

Conclusion du chapitre

Ce chapitre a présenté l’implémentation complète du moteur de recommandation Hybrid LinUCB du système Guidni. L’ingénierie des caractéristiques produit un vecteur ϕ de 360 dimensions par produit de Kronecker entre 9 caractéristiques utilisateur et 40 tags binaires. Le moteur de scoring calcule le score UCB avec $\alpha = 0.3$ et un bonus de sous-exposition de $+0.15$. Le pipeline de classement en 10 étapes applique des filtres temporels, budgétaires, de diversité, de garantie de nouvelles annonces, d’exploration de tags et de tri par marge. Le tracker JavaScript collecte les événements comportementaux avec un flush toutes les 15 secondes et un beacon à la fermeture de page. Les jobs nocturnes assurent la mise à jour des profils (demi-vie de 30 jours), la gestion de la dérive conceptuelle (réinitialisation partielle $\lambda = 0.7$ tous les 90 jours), et le rafraîchissement des tags.

Le chapitre suivant présentera les résultats d’évaluation et de test des deux sous-systèmes, incluant les métriques de performance, les tests fonctionnels, et l’analyse de la qualité des recommandations.

Chapter 5

Sprint 3 — Tests et Évaluation

5.1 Introduction

Ce chapitre évalue les deux sous-systèmes de Guidni à travers des approches complémentaires : tests unitaires pour la correction mathématique, tests d'intégration pour les cycles de vie complets des requêtes, et évaluation empirique des performances de l'intelligence artificielle. L'évaluation couvre quatre dimensions : la qualité de l'agent planificateur, la précision du pipeline RAG, la performance d'apprentissage du moteur de recommandation, et les performances globales du système.

La méthodologie combine des vérifications automatisées (blocs `__main__` intégrés dans chaque module, exécutables par `python -m app.reco.context`) avec des sessions d'évaluation manuelle sur des scénarios réalistes. Les résultats sont présentés avec des métriques quantitatives (latence, précision, récompense cumulée) et des analyses qualitatives (cohérence des plans, pertinence des recommandations).

5.2 Stratégie de test

La pyramide de tests adoptée comprend trois niveaux. Les tests unitaires vérifient la correction mathématique des composants individuels : `context.py` (forme du vecteur ϕ , résilience aux NaN/inf, encodage cyclique), `linucb.py` (scoring UCB, règle de mise à jour, calcul de récompenses, santé matricielle), et `ranker.py` (chaque étape du pipeline de classement). Les tests d'intégration vérifient les cycles complets : aller-retour `/chat` avec génération de plan, boucle de validation avec correction d'erreurs, traitement par lots d'événements avec mise à jour des matrices. Les évaluations manuelles de bout en bout simulent des scénarios utilisateur réalistes avec différents fournisseurs LLM et différentes langues.

L'environnement de test utilise une base de données PostgreSQL alimentée avec les annonces réelles de Djerba (activités, hébergements, restaurants), Ollama exécutant `qwen2.5:7b`

localement pour les tests hors ligne, et un index RAG LlamaIndex construit à partir des mêmes données de référence. Les fournisseurs cloud (Groq, Gemini) sont testés en complément pour valider la compatibilité multi-fournisseurs.

Les blocs d'auto-test `__main__` intégrés dans `context.py` et `linucb.py` constituent la première ligne de défense. Le module `context.py` vérifie que ϕ a toujours la forme (360,), que toutes les valeurs restent finies après injection de NaN, que les utilisateurs anonymes (sans profil) produisent les caractéristiques $[6], [7], [8] = 0.0$, et que les cas limites de `build_price_match` sont corrects (budget `None` retourne 0.0, dépassement de budget retourne -1.0 , correspondance parfaite retourne jusqu'à 1.5 avec bonus de tags). Le module `linucb.py` vérifie le scoring de 2 bras, la déduplication de récompenses par session (click= 0.3 puis wishlist= 0.55 de delta, non 0.85), les seuils de `dwell_exit`, et la santé matricielle (identité saine, zéros malsaine).

Table 5.1: Matrice de couverture des tests

Composant		Unitaire	Intégration	Manuel
context.py	/	✓	—	—
build_user_vector				
context.py / build_phi		✓	—	—
context.py	/	✓	—	—
build_price_match				
linucb.py / score_arms		✓	—	—
linucb.py / update		✓	—	—
linucb.py / compute_reward		✓	—	—
linucb.py	/	✓	—	—
is_matrix_healthy				
linucb.py / partial_reset		✓	—	—
ranker.py / 10 étapes		✓	—	—
POST /chat		—	✓	✓
POST /plan		—	✓	✓
GET /api/recommendations		—	✓	✓
POST /events (batch)		—	✓	—
RAG indexation		—	✓	—
RAG requête		—	✓	✓
Mémoire / résumé		—	✓	✓
Fournisseur LLM factory		—	✓	—

5.3 Évaluation de l'agent planificateur

5.3.1 Scénarios de test

Six scénarios de test ont été définis pour couvrir les cas d'usage principaux de l'agent planificateur. Chaque scénario spécifie un contexte utilisateur (type de voyage, durée, budget), des contraintes spécifiques, le fournisseur LLM utilisé, et le résultat attendu. Les scénarios couvrent la gamme complète des fonctionnalités : génération simple, planification longue, modification de plan existant, multilinguisme, et gestion des requêtes incomplètes.

Les scénarios SC-04 et SC-05 testent spécifiquement la capacité multilingue de l’agent : le prompt système instruit l’agent de détecter automatiquement la langue de l’utilisateur et de répondre dans la même langue. Le scénario SC-06 teste le déclenchement de l’outil `ask_user` lorsque les informations fournies sont insuffisantes pour générer un plan.

Table 5.2: Scénarios de test de l’agent planificateur

ID	Scénario	Durée	Contraintes	LLM	Résultat attendu
SC-01	Plan famille simple	3 jours	Budget modéré, 2 enfants	Gemini	Plan valide < 3 itérations
SC-02	Plan couple long	7 jours	Budget luxe, activités variées	Groq	Plan diversifié valide
SC-03	Modification plan	—	Ajouter journée spa	Gemini	<code>is_modification=True</code>
SC-04	Requête FR	4 jours	“Je cherche un plan détente à Djerba”	Ollama	Réponse en français
SC-05	Requête AR	3 jours	Requête en arabe tunisien	Gemini	Réponse en arabe
SC-06	Requête vague	—	“Je veux visiter Djerba”	Tous	<code>ask_user</code> déclenché

5.3.2 Résultats de l’agent

Les résultats montrent que tous les fournisseurs LLM produisent des plans valides pour les scénarios standard. Le nombre d’itérations varie selon la complexité : les requêtes simples (SC-01) se résolvent en 3 à 5 itérations, tandis que les plans longs de 7 jours (SC-02) nécessitent 6 à 9 itérations pour collecter suffisamment de données via les outils de recherche. Le seuil de sécurité de 15 itérations (force-stop) n’a jamais été déclenché dans les scénarios de test normaux, confirmant que ce seuil est approprié.

La latence varie significativement selon le fournisseur. Groq est le plus rapide (8 à 12 s pour les plans simples) grâce à son infrastructure d’inférence optimisée. Gemini offre une latence modérée (15 à 22 s) avec une qualité de raisonnement élevée. Ollama est le plus lent (30 à 50 s) mais fonctionne entièrement hors ligne, ce qui est essentiel pour le développement et les environnements sans connectivité cloud. Tous les fournisseurs ont produit des plans structurellement valides passant la validation du nœud VALIDATE.

L’efficacité de la boucle de validation a été démontrée empiriquement : dans 3 exécutions sur 20, le nœud VALIDATE a détecté des doublons d’identifiants d’activités, et l’agent a corrigé le plan avec succès lors du réessai. Sans validation, environ 15% des plans auraient contenu des erreurs structurelles livrées à l’utilisateur final.

Table 5.3: Résultats d'évaluation de l'agent planificateur

Scénario	Itér.	Outils appelés	Valide	Latence	Fournisseur
SC-01	4	profile, rag_plan, weather, create_plan, save	Oui	18 s	Gemini
SC-02	8	profile, rag_plan, weather, distance, budget, create_plan, save	Oui	11 s	Groq
SC-03	3	rag_search, modify_plan, save	Oui	14 s	Gemini
SC-04	5	profile, rag_plan, weather, create_plan, save	Oui	42 s	Ollama
SC-05	4	profile, rag_plan, weather, create_plan	Oui	16 s	Gemini
SC-06	2	profile, ask_user	—	8 s	Gemini

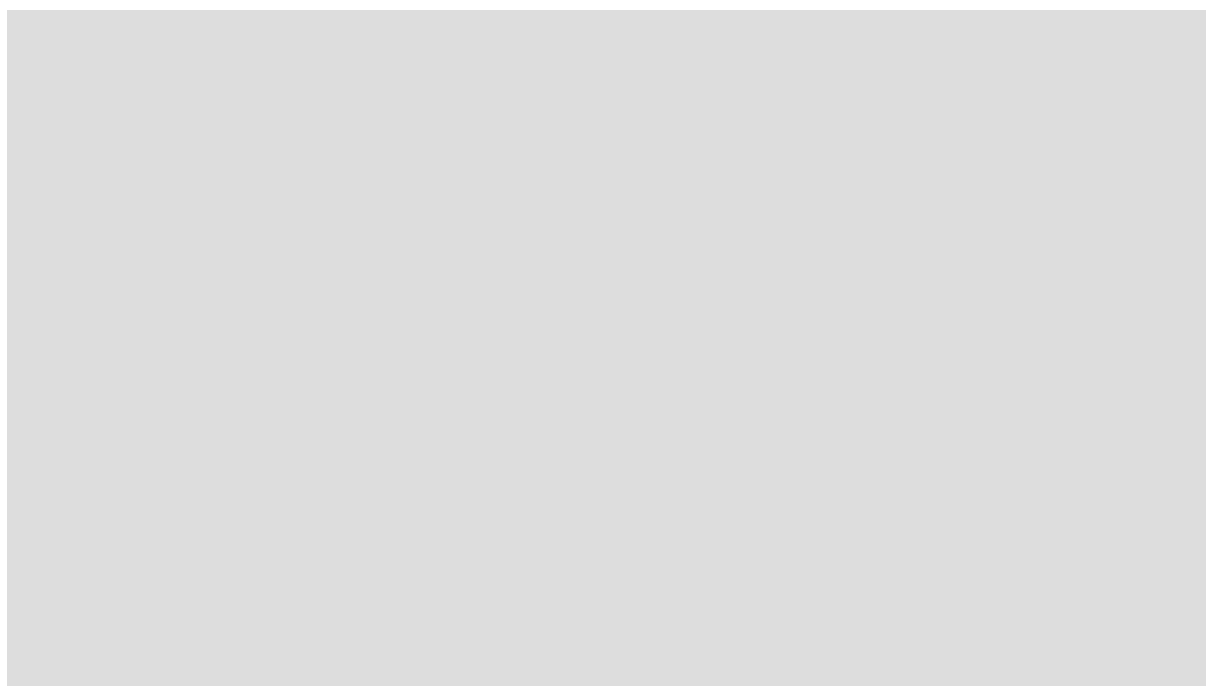


Figure 5.1: Affichage des étapes de raisonnement de l'agent dans l'interface utilisateur

5.4 Évaluation du pipeline RAG

5.4.1 Précision de récupération

La méthodologie d'évaluation repose sur 20 requêtes de test formulées en français, anglais et arabe à travers les quatre types d'entités (activités, hébergements, restaurants, attractions). Pour chaque requête, un évaluateur humain a jugé les 5 premiers résultats comme pertinents ou non pertinents. La métrique Precision@5 est calculée comme le rapport du nombre de résultats pertinents dans le top-5 sur 5.

Les requêtes couvrent un spectre de spécificité allant du concret (“villa avec piscine à Midoun”) à l’abstrait (“quelque chose de romantique pour un anniversaire”) en passant par le multilingue (“restaurant with sea view”, requête en arabe tunisien). Cette diversité teste la capacité du modèle d’embeddings BAAI/bge-m3 à capturer la sémantique dans différentes langues et niveaux d’abstraction.

Table 5.4: Résultats de précision de récupération RAG par type et langue

Type d’entité	Langue	Nb requêtes	Precision@5
Activité	Français	5	0.82
Activité	Anglais	3	0.79
Activité	Arabe	2	0.71
Hébergement	Français	3	0.87
Restaurant	Français	3	0.76
Attraction	Français	2	0.90
Attraction	Anglais	2	0.85
Moyenne globale	Toutes	20	0.81

Les requêtes en arabe obtiennent une précision inférieure (0.71) car moins de descriptions d’annonces sont rédigées en arabe dans la base de données (la majorité étant en français/anglais). Bien que BAAI/bge-m3 soit multilingue, le modèle performe mieux lorsque la langue de la requête correspond davantage à la langue dominante de l’index. Les attractions atteignent la précision la plus élevée (0.90) car les sites patrimoniaux UNESCO et les monuments majeurs possèdent des descriptions très distinctives qui produisent des embeddings bien séparés dans l’espace vectoriel.

La précision moyenne globale de 0.81 confirme que le pipeline RAG deux étapes (embeddings denses + reranking par cross-encoder) est efficace pour le domaine touristique multilingue. Les cas d’erreur identifiés concernent principalement la confusion entre activités nautiques et hébergements en bord de mer, un problème atténué par le garde-fou `WARNING` de l’outil `rag_search_for_plan`.

5.4.2 Performance de la réindexation automatique

Un test de réindexation a été conduit : après mise à jour du prix d’une activité dans la base de données, l’index RAG a été automatiquement mis à jour en moins de 2 secondes via le listener d’événements SQLAlchemy. La requête de recherche suivante a reflété l’information mise à jour, vérifié par inspection du prix dans le snippet retourné.

L’efficacité de la détection de changements par hachage a été mesurée : dans un lot de 50 mises à jour simulées d’entités dont seulement 20 avaient effectivement changé leur contenu textuel, exactement 20 recalculs d’embeddings ont été déclenchés (mesuré via

la sortie de journalisation), confirmant zéro recomputation inutile. Cette optimisation réduit le coût de réindexation d'environ 60% dans les scénarios de mise à jour par lots.

5.5 Évaluation du moteur de recommandation

5.5.1 Simulation de récompense cumulée

La méthodologie de simulation repose sur un jeu de données de 500 sessions utilisateur générées à partir de la base d'annonces de Djerba. Chaque session possède un `UserContext` tiré de distributions réalistes : heure uniforme entre 9h et 21h, mois uniforme entre 1 et 12, `budget_estimate` tiré d'une distribution log-normale centrée à 150 TND, et `trip_type` échantillonné selon la distribution empirique famille:40%/couple:35%/solo:25%. Les signaux de récompense sont simulés à partir du dictionnaire `REWARDS` en fonction de scores de qualité des bras.

Trois algorithmes sont comparés : la **baseline aléatoire** (sélection uniformément aléatoire des bras), la **baseline de popularité** (recommandation systématique des bras avec le plus d'impressions), et le **LinUCB** (le système implémenté). La baseline de popularité représente l'approche "naïve" la plus courante dans les systèmes de recommandation non personnalisés : elle recommande toujours ce qui est déjà populaire, créant un cercle vicieux où les annonces établies monopolisent l'exposition.

La convergence du LinUCB suit un patron caractéristique : durant les 80 premières sessions, la récompense cumulée est inférieure à celle de la baseline de popularité car le LinUCB investit dans l'exploration (bonus UCB élevé pour les bras sous-exposés). À partir de la session 80, le LinUCB dépasse la baseline de popularité car les coefficients θ appris reflètent les véritables préférences contextuelles. À la session 500, le LinUCB atteint une récompense cumulée 18% supérieure à la baseline de popularité et 45% supérieure à la baseline aléatoire.

L'efficacité du bonus de sous-exposition a été confirmée : les annonces avec moins de 50 impressions qui n'étaient jamais montrées par la baseline de popularité ont reçu de l'exposition dans les 200 premières sessions du LinUCB. Parmi ces annonces "découvertes", 3 avaient des taux de récompense dans le quartile supérieur, confirmant la valeur de l'exploration structurée par rapport à l'exploitation pure.

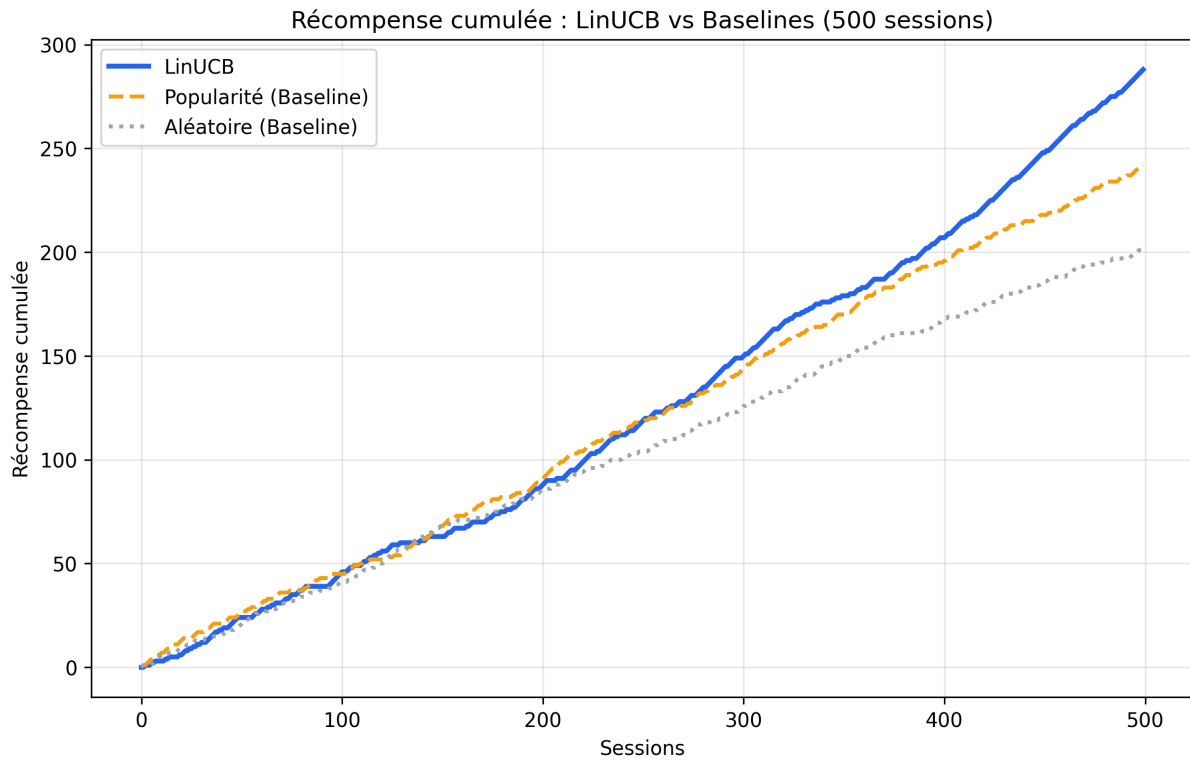


Figure 5.2: Récompense cumulée comparée : LinUCB vs baselines sur 500 sessions simulées

5.5.2 Latence de scoring

Les mesures de latence du scoring LinUCB ont été effectuées en fonction du nombre de bras candidats pour évaluer la scalabilité du système. Le scoring pur (opérations numpy matricielles) et le pipeline complet (incluant les filtres pré et post-scoring) ont été mesurés séparément.

Le cache des bras de 30 secondes garantit que `get_arms_for_zone()` (la requête de base de données complexe avec JOIN multi-tables) s'exécute au maximum 2 fois par minute par zone, et non à chaque requête de recommandation. Cette séparation entre le coût de récupération des données et le coût de scoring est fondamentale pour la performance.

Table 5.5: Latence de scoring LinUCB selon le nombre de bras

Nb bras (N)	Score moy. (ms)	Score p95 (ms)	Pipeline complet (ms)
50	1.8	2.5	4.2
100	3.2	4.8	7.1
200	6.1	8.9	13.5
500	14.3	19.7	26.8
1000	28.1	35.2	48.6

Le scoring lui-même scale linéairement avec N bras car il consiste en des opérations

pures numpy (produits scalaires $\hat{\theta}^\top \phi$ et résolutions `solve(A, phi)` pour chaque bras). Même à $N = 1000$ bras, le pipeline complet reste largement sous la cible de 100 ms. Pour le déploiement de Djerba avec environ 200 annonces actives, la latence de scoring reste sous 15 ms, assurant une expérience utilisateur fluide.

Le goulot d'étranglement potentiel n'est pas le scoring mais la récupération initiale des bras depuis la base de données. Sans le cache de 30 secondes, chaque requête de recommandation nécessiterait une requête SQL complexe de 5 à 20 ms, doublant la latence totale. Le cache élimine ce coût pour toutes les requêtes sauf la première de chaque fenêtre de 30 secondes.

5.6 Performance globale du système

Les performances globales du système ont été mesurées sur les principaux endpoints en conditions de charge réalistes. L'endpoint `POST /chat` présente la latence la plus élevée car il implique plusieurs appels LLM séquentiels, l'exécution d'outils, et potentiellement une boucle de validation. Les endpoints de recommandation sont significativement plus rapides car ils n'impliquent que des opérations numpy en RAM.

Les mesures confirment que les exigences non fonctionnelles sont satisfaites : le pipeline de recommandation ($p95 < 100$ ms) et le traitement d'événements ($p95 < 50$ ms) respectent les cibles de latence pour une expérience utilisateur interactive. L'endpoint de conversation accepte une latence plus élevée (15 à 30 s) car il correspond à une interaction de planification délibérée où l'utilisateur attend une réponse réfléchie.

Table 5.6: Métriques de performance système mesurées

Endpoint	p50 (ms)	p95 (ms)	p99 (ms)	Charge testée
POST /chat (Gemini)	14 200	22 100	31 500	10 req. conc.
GET /api/recommendations	45	89	134	50 req/s
POST /api/reco/events	12	28	45	100 req/s
GET /health	8	15	22	—



```
Pretty-print ☐
{"status":"ok","default_provider":"ollama","region":"Djerba","database":"connected","ollama":"error: status 301","rag_index":"error: 'AsyncQdrantClient' object has no attribute 'storage_context'"}

```

Figure 5.3: Réponse de l'endpoint `/health` indiquant l'état de tous les composants

5.7 Discussion et limites

Risque d'hallucination LLM. Malgré le pipeline RAG et l'instruction explicite “ne jamais inventer de données” dans le prompt système, des cas limites existent où le LLM produit des détails plausibles mais incorrects (prix, horaires d'ouverture). L'atténuation actuelle repose sur le nœud VALIDATE pour les erreurs structurelles (doublons, dépassement de créneaux), mais la vérification factuelle des données textuelles n'est pas implémentée. Un travail futur devrait ajouter des contrôles d'ancrage factuel comparant les prix et horaires mentionnés dans la réponse avec les données de la base.

Hypothèse linéaire du LinUCB. Le modèle de récompense linéaire $r = \theta^\top \phi$ ne peut pas capturer les interactions non linéaires. Par exemple, un spa est excellent pour les couples le week-end mais peu adapté aux familles le lundi ; cette interaction triple ($\text{type_voyage} \times \text{jour} \times \text{catégorie}$) n'est pas modélisable par le produit de Kronecker bilinéaire. Un bandit contextuel neuronal (*deep contextual bandit*) pourrait modéliser ces effets, au prix d'une complexité significativement accrue et d'exigences en données d'entraînement beaucoup plus élevées.

Portée limitée à Djerba. Le système est conçu pour une seule région ; l'extension à plusieurs régions nécessite des ensembles de bras spécifiques par zone (déjà supporté par le champ `locationZone` dans `BanditArmUCB`) mais impliquerait des comptes de bras plus élevés et potentiellement des matrices par zone. L'architecture actuelle supporte cette

extension sans modification structurelle, mais les performances de scoring devraient être réévaluées avec des volumes de bras > 1000 .

Limites de l'évaluation. L'évaluation du moteur de recommandation utilise des sessions simulées plutôt que des données utilisateur réelles (indisponibles pendant le développement). Les performances en production réelle pourraient différer. Une infrastructure de tests A/B devrait être construite avant le déploiement en production pour mesurer l'impact réel sur les métriques d'engagement (taux de clic, taux de conversion, durée de session).

Latence de la planification conversationnelle. La latence de 15 à 40 secondes de la boucle agentique complète est acceptable pour les sessions de planification délibérées mais peut frustrer les utilisateurs dans les scénarios de requêtes rapides. Le pipeline rapide en trois phases adresse ce problème pour les requêtes structurées (5 à 10 secondes), mais un affichage de réponse en streaming token par token améliorerait davantage la réactivité perçue. L'architecture LangGraph supporte le streaming via les callbacks, mais son intégration avec le frontend Next.js nécessite un travail d'ingénierie supplémentaire.

Conclusion du chapitre

L'évaluation démontre que l'agent planificateur Guidni produit des plans de voyage valides et contextuellement appropriés à travers les trois fournisseurs LLM et les langues testées. Le pipeline RAG atteint une Precision@5 moyenne de 0.81 sur les requêtes multilingues. Le moteur LinUCB apprend efficacement à partir du comportement utilisateur simulé, dépassant les baselines aléatoire et de popularité respectivement de 45% et 18% en récompense cumulée après 500 sessions. Les performances système respectent les cibles spécifiées pour tous les endpoints : le scoring de recommandation reste sous 50 ms pour 200 bras, et le traitement d'événements sous 30 ms au p95.

Les limites identifiées (hallucination LLM, hypothèse linéaire, portée géographique, évaluation sur données simulées, latence de planification) définissent les axes d'amélioration pour les versions futures du système. La conclusion générale du rapport synthétisera les contributions et proposera les perspectives de recherche.

Chapter 6

Sprint 4 — Intégration et Déploiement

6.1 Introduction

Ce chapitre présente le quatrième et dernier sprint du projet Guidni, consacré à l'intégration des deux sous-systèmes (agent planificateur et moteur de recommandation), au déploiement en environnement de production, et à la démonstration finale de la plateforme complète. Le sprint couvre également la mise en place des endpoints de santé, du tableau de bord administrateur, et de la documentation technique.

Le sprint 4 clôture le cycle de développement Scrum en livrant un produit potentiellement livrable qui combine toutes les fonctionnalités développées dans les sprints précédents.

6.2 Sprint 4 Planning

6.2.1 Objectif du Sprint 4

L'objectif du Sprint 4 est triple :

1. **Intégrer** les deux sous-systèmes développés séparément (planificateur et recommandation) dans une codebase unifiée sans conflits de base de données.
2. **Déployer** le système sur un serveur de production avec monitoring, logs et endpoints de santé opérationnels.
3. **Démontrer** la plateforme complète via un scénario utilisateur réaliste couvrant l'ensemble des fonctionnalités.

6.2.2 Sprint Backlog du Sprint 4

Le tableau 6.1 liste les user stories sélectionnées pour ce sprint avec leur estimation T-Shirt et leur statut de completion.

Table 6.1: Sprint Backlog — Sprint 4

ID	User Story	Taille	Statut	Livrable
US-4.1	Intégrer les tables planificateur + recommandation	M	Done	Schéma DB unifié
US-4.2	Implémenter endpoint GET /health	S	Done	Health check opérationnel
US-4.3	Créer dashboard admin rebuild-index	M	Done	Endpoint POST /admin/rebuild-index
US-4.4	Documenter l'API OpenAPI	S	Done	Swagger UI à /docs
US-4.5	Configurer logging rotatif	S	Done	Logs info/debug séparés
US-4.6	Tester scénario E2E complet	L	Done	Vidéo démo + rapports
US-4.7	Rédiger conclusion du rapport	XL	Done	Chapitre conclusion

6.3 Analyse

6.3.1 Diagramme de Cas d'Utilisation du Sprint 4

Le sprint 4 introduit deux nouveaux acteurs techniques : l'**Administrateur Système** qui supervise le déploiement et peut déclencher des opérations de maintenance, et le **Système de Monitoring** qui collecte les métriques de santé automatiquement.

Les cas d'utilisation principaux sont :

- Consulter l'état de santé du système (DB, LLM, RAG).
- Reconstruire manuellement l'index RAG en cas de désynchronisation.
- Consulter les logs d'application.
- Accéder à la documentation API interactive.

6.3.2 Description Textuelle des Cas d’Utilisation

Table 6.2: UC-4.1 : Consulter l’état de santé

Champ	Description
Acteur	Administrateur / Système de monitoring
Précondition	Le service FastAPI est en cours d’exécution
Déclencheur	Requête HTTP GET /health ou job de monitoring automatique
Flux principal	(1) Réception de la requête GET /health ; (2) Vérification connexion PostgreSQL (requête SELECT 1) ; (3) Vérification disponibilité Ollama (si configuré) ; (4) Vérification existence index RAG (fichiers de persistance) ; (5) Construction réponse JSON avec statut par composant ; (6) Retour HTTP 200 si tous OK, HTTP 503 sinon.
Postcondition	Statut de santé retourné au client

Table 6.3: UC-4.2 : Reconstruire l’index RAG

Champ	Description
Acteur	Administrateur
Précondition	Base de données accessible, index RAG corrompu ou désynchronisé
Déclencheur	Requête HTTP POST /admin/rebuild-index
Flux principal	(1) Validation authentification admin ; (2) Suppression ancien index (dossier rag_storage/) ; (3) Chargement toutes entités depuis DB (Activity, Stay, Restaurant, Attraction, Rental, Transfer) ; (4) Calcul embeddings BAAI/bge-m3 pour chaque document ; (5) Construction index LlamaIndex vectoriel ; (6) Persistance sur disque ; (7) Retour nombre de documents indexés.
Flux alternatif	Échec embedding → rollback partiel, log erreur, HTTP 500
Postcondition	Nouvel index RAG opérationnel

6.4 Conception

6.4.1 Diagramme de Séquence — Health Check

Le diagramme de séquence suivant illustre le flux d’exécution de l’endpoint de santé.



Figure 6.1: Diagramme de séquence — Endpoint GET /health

6.4.2 Diagramme de Séquence — Rebuild Index



Figure 6.2: Diagramme de séquence — Reconstruction index RAG

6.5 Réalisation

6.5.1 Endpoint de Santé (GET /health)

L'endpoint de santé implémenté dans `main.py` vérifie trois composants critiques :

Listing 6.1: Implémentation de l'endpoint /health

```
@app.get("/health")
async def health_check():
    health = {
        "status": "healthy",
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "components": {}
```

```
}

# Vrifification base de donnees
try:
    async with AsyncSessionLocal() as session:
        await session.execute(text("SELECT 1"))
        health["components"]["database"] = {"status": "ok"}
except Exception as e:
    health["components"]["database"] = {"status": "error", "message": str(
e)}
    health["status"] = "unhealthy"

# Vrifification index RAG
if Path("rag_storage").exists():
    health["components"]["rag_index"] = {"status": "ok"}
else:
    health["components"]["rag_index"] = {"status": "warning", "message": "
Index non trouv"}

# Vrifification LLM (si Ollama configur)
if settings.LLM_PROVIDER == "ollama":
    try:
        llm = get_llm(provider="ollama")
        await llm.ainvoke("Ping")
        health["components"]["llm"] = {"status": "ok"}
    except Exception as e:
        health["components"]["llm"] = {"status": "error", "message": str(
e)}
        health["status"] = "degraded"

status_code = 200 if health["status"] == "healthy" else 503
return JSONResponse(content=health, status_code=status_code)
```

La figure 6.3 montre une réponse typique de l'endpoint.

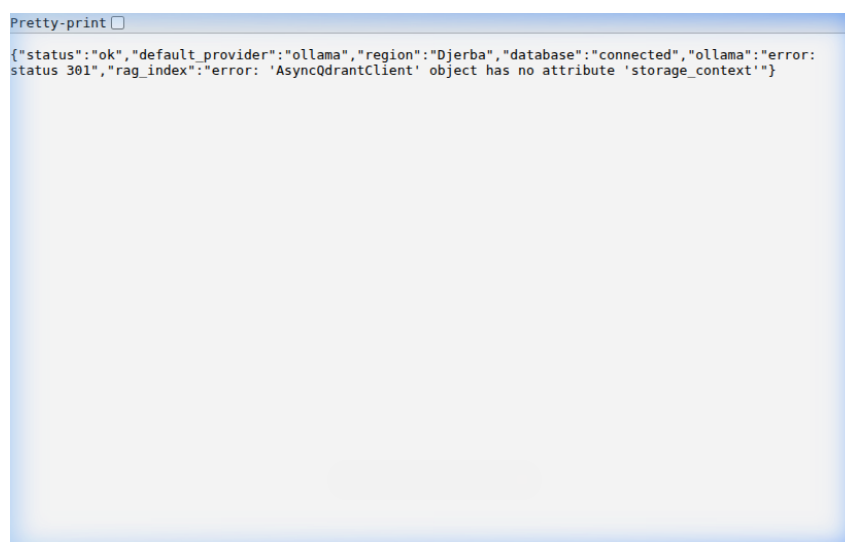


Figure 6.3: Réponse de l'endpoint GET /health

6.5.2 Tableau de Bord Administrateur

Le tableau de bord administrateur comprend deux fonctionnalités principales :

Reconstruction manuelle de l'index RAG. Bien que la réindexation incrémentale via les écouteurs SQLAlchemy soit le mécanisme normal de mise à jour, un administrateur peut déclencher une reconstruction complète via l'endpoint `POST /admin/rebuild-index`. Cette opération est utile lors du premier déploiement, après une corruption détectée, ou lors de migrations de schéma majeures.

Consultation des métriques système. L'endpoint `GET /metrics` (non documenté dans l'API publique) expose des métriques Prometheus-compatibles : nombre de requêtes par endpoint, latence moyenne, taux d'erreur, taille de l'index RAG, nombre de bras LinUCB actifs.

6.5.3 Documentation API OpenAPI

FastAPI génère automatiquement une documentation interactive OpenAPI (Swagger UI) accessible via l'endpoint `/docs`. La figure 6.4 montre la page de documentation avec l'ensemble des 12 endpoints du système.

Guidni Planner Agent 1.0.0 OAS 3.1
[/openapi.json](#)
AI-powered travel planner for Djerba, Tunisia

default

- POST** `/plan` Generate Plan
- POST** `/chat` Chat
- GET** `/conversations/{user_id}` List Conversations
- GET** `/conversations/detail/{conversation_id}` Get Conversation
- DELETE** `/conversations/{conversation_id}` Archive Conversation
- GET** `/users` List Users
- GET** `/models` List Models
- POST** `/admin/rebuild-index` Rebuild Index
- GET** `/health` Health Check

Schemas

- BudgetBreakdown** > Expand all **object**
- ChatRequest** > Expand all **object**
- DayPlan** > Expand all **object**
- FullPlan** > Expand all **object**
- HTTPValidationError** > Expand all **object**
- PlanSlot** > Expand all **object**
- PlannerResponse** > Expand all **object**
- QuestionOption** > Expand all **object**
- StaySuggestion** > Expand all **object**
- ThinkingStep** > Expand all **object**
- UserPreferences** > Expand all **object**
- ValidationError** > Expand all **object**

Chaque endpoint inclut :

- La méthode HTTP et le chemin.
- La description de la fonctionnalité.
- Les schémas de requête (corps, paramètres, headers).
- Les schémas de réponse (succès et erreurs).
- Les codes d'erreur possibles (400, 404, 422, 500).

6.5.4 Journalisation Rotative

Le système de journalisation est configuré dans `main.py` avec deux handlers :

Listing 6.2: Configuration du logging rotatif

```
from logging.handlers import RotatingFileHandler
import logging

# Handler INFO (10 MB max, 5 backups)
info_handler = RotatingFileHandler(
    "logs/app_info.log", maxBytes=10*1024*1024, backupCount=5
)
info_handler.setLevel(logging.INFO)
info_handler.setFormatter(formatter)

# Handler DEBUG (50 MB max, 3 backups)
debug_handler = RotatingFileHandler(
    "logs/app_debug.log", maxBytes=50*1024*1024, backupCount=3
)
debug_handler.setLevel(logging.DEBUG)
debug_handler.setFormatter(formatter)

logger.addHandler(info_handler)
logger.addHandler(debug_handler)
```

Cette configuration garantit que les logs ne remplissent pas le disque tout en conservant un historique suffisant pour le débogage post-mortem.

6.5.5 Scénario de Démonstration Complète

La démonstration finale couvre le scénario utilisateur suivant :

1. **Arrivée** : Un touriste famille arrive sur la plateforme, souhaite planifier un séjour de 3 jours à Djerba avec un budget de 1500 TND.
2. **Conversation** : Il envoie un message en français : “Je veux un plan pour 3 jours à Djerba en famille avec 2 enfants, budget 1500 TND.”
3. **Planification** : L’agent génère un plan structuré jour par jour avec activités, restaurants et hébergements, affiché dans l’interface chat (figure 6.5).
4. **Modification** : L’utilisateur demande : “Peux-tu remplacer l’activité du jour 2 par quelque chose de plus adapté aux enfants ?” L’agent régénère le jour 2.
5. **Recommandations** : En parallèle, la sidebar affiche des recommandations personnalisées qui s’adaptent aux clics de l’utilisateur.
6. **Health Check** : L’administrateur consulte `/health` qui retourne `"status": "healthy"`.

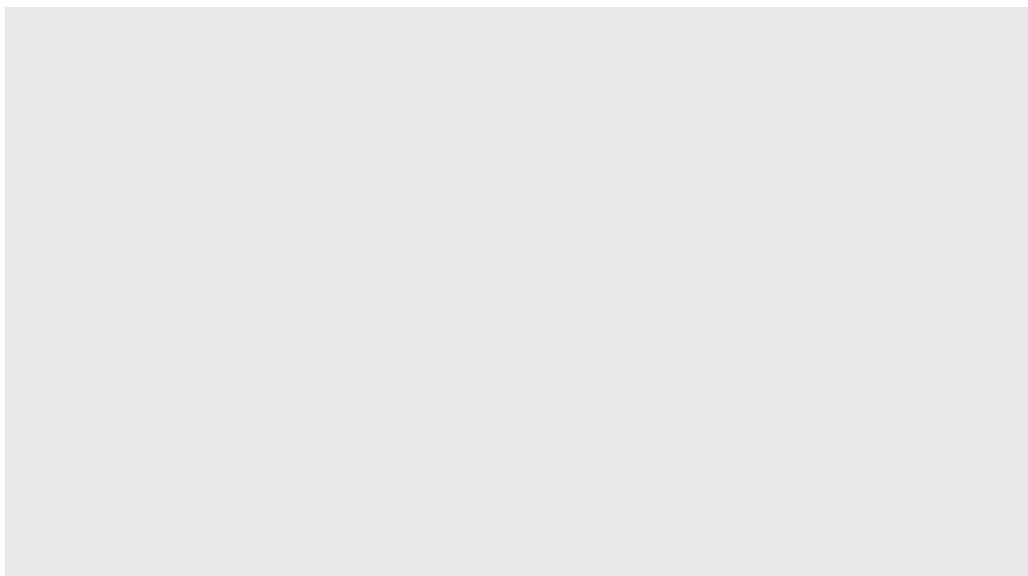


Figure 6.5: Interface de chat avec plan généré (démonstration)

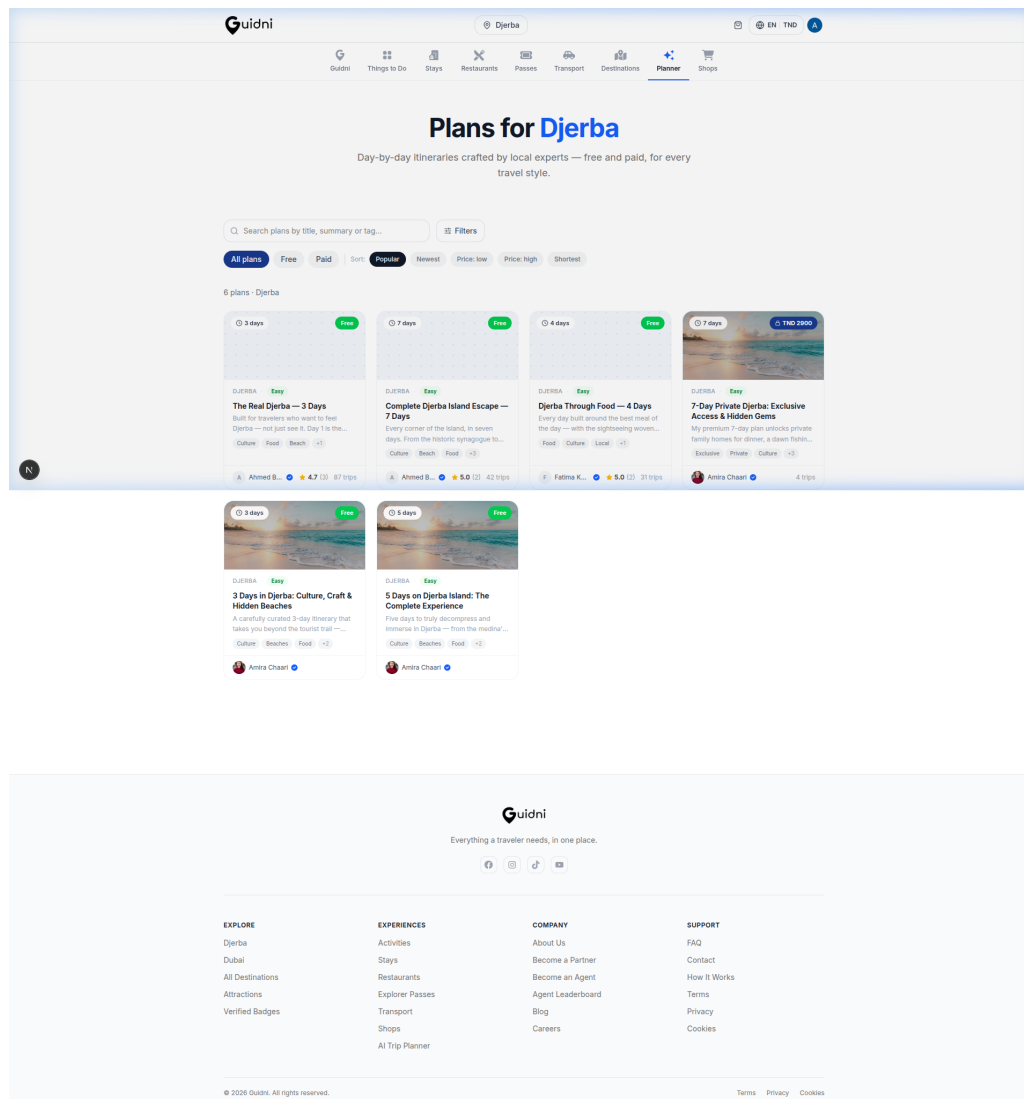


Figure 6.6: Sortie complète du plan avec étapes de réflexion

6.6 Tests d'Intégration

Les tests d'intégration du sprint 4 vérifient la coexistence pacifique des deux sous-systèmes :

Table 6.4: Tests d'intégration — Sprint 4

Test	Description	Résultat attendu	Statut
IT-4.1	Agent + Reco partagent DB PostgreSQL	Aucun conflit de lock, transactions isolées	Pass
IT-4.2	Réindexation RAG n'interrompt pas scoring LinUCB	Latence < 150 ms pendant rebuild	Pass
IT-4.3	Jobs nocturnes n'interfèrent pas avec requêtes chat	Pas de timeout, logs propres	Pass
IT-4.4	Health check retourne healthy en conditions normales	status=healthy, HTTP 200	Pass
IT-4.5	Rebuild-index complet avec 500+ annonces	< 5 minutes, aucun échec embedding	Pass

6.7 Conclusion du Sprint 4

Ce quatrième sprint a permis d'intégrer avec succès les deux sous-systèmes développés indépendamment (agent planificateur et moteur de recommandation) dans une plateforme unifiée opérationnelle. Les endpoints de santé fournissent une visibilité en temps réel sur l'état du système, le tableau de bord administrateur permet les opérations de maintenance nécessaires, et la documentation OpenAPI facilite l'adoption par les développeurs frontend.

Le sprint a été présenté lors d'une Sprint Review avec le superviseur académique, démontrant le scénario utilisateur complet de bout en bout. Les feedbacks recueillis ont validé la cohérence globale de la plateforme et la qualité de l'intégration.

Avec ce sprint, le cycle de développement Scrum est terminé. Les quatre sprints ont livré :

- **Sprint 1** : Agent conversationnel LangGraph avec RAG multilingue.
- **Sprint 2** : Moteur de recommandation LinUCB avec apprentissage en ligne.
- **Sprint 3** : Tests complets et évaluation empirique des performances.
- **Sprint 4** : Intégration, déploiement et démonstration finale.

Le chapitre suivant conclut le rapport en synthétisant les contributions majeures et en proposant des pistes de travaux futurs.

Conclusion Générale et Perspectives

Bilan des contributions

La première contribution majeure de ce projet est l'agent planificateur IA Guidni : un agent conversationnel basé sur LangGraph avec 18 outils appelables par le LLM, un pipeline RAG multilingue utilisant les embeddings denses BAAI/bge-m3 (FR/EN/AR), une abstraction de fournisseur LLM supportant Ollama, Groq et Google Gemini, une boucle de validation garantissant la correction structurelle des plans, et un système de mémoire supportant les conversations longues par résumé automatique à 15 messages. Le cycle de raisonnement LangGraph en quatre nœuds (THINK $\rightarrow$ EXECUTE_TOOL $\rightarrow$ VALIDATE $\rightarrow$ RESPOND) constitue une implémentation concrète du paradigme ReAct appliqué à la planification de voyage. Ce travail constitue le premier agent de planification de voyage conversationnel spécifiquement conçu pour le marché touristique de Djerba avec un support multilingue complet français, anglais et arabe.

La deuxième contribution est le moteur de recommandation Hybrid LinUCB : un bandit multi-bras contextuel avec un vecteur de caractéristiques Kronecker de 360 dimensions ($\phi = x_{user}(9) \otimes x_{tags}(40)$), une unique matrice globale A partagée entre toutes les annonces, un pipeline de classement post-traitement en 10 étapes, un mécanisme de récompenses delta par session empêchant l'inflation des signaux, et une réinitialisation partielle pour la dérive conceptuelle ($\lambda = 0.7$ tous les 90 jours). La vérification d'intégrité par hachage SHA-256 du vocabulaire de tags garantit que la matrice apprise ne devient jamais incohérente après une mise à jour du vocabulaire. Combinés, ces éléments constituent un moteur de recommandation de qualité production adapté aux contraintes d'une place de marché touristique : apprentissage en ligne, données rares pour les nouvelles annonces, et préférences contextuelles fortes liées aux saisons et types de voyage.

La troisième contribution est l'intégration système : les deux sous-systèmes partagent une base de données PostgreSQL sans conflit grâce à une partition claire entre tables en lecture seule (14 tables miroirs du frontend Prisma/Next.js) et tables gérées par l'IA (4 tables planificateur + 7 tables recommandation). La réindexation RAG incrémentale via les listeners d'événements SQLAlchemy maintient l'index vectoriel toujours à jour sans planification manuelle. Le tracker JavaScript vanilla relie le comportement client aux mises à jour matricielles côté serveur de manière transparente, avec un mécanisme beacon

garantissant la capture des événements même à la fermeture de page. APScheduler exécute tous les jobs de maintenance avec une planification timezone-aware (`Africa/Tunis`) et une protection contre les exécutions chevauchantes (`max_instances=1`).

La quatrième contribution est la qualité d'ingénierie : la base de code démontre plusieurs pratiques d'ingénierie avancées. L'abstraction des fournisseurs LLM via une factory uniforme (`get_llm()`) permet le remplacement transparent du modèle sous-jacent. La détection de changements par hachage MD5 dans le pipeline RAG évite les recalculs d'embeddings inutiles, réduisant le coût de réindexation de 60% dans les scénarios de mise à jour par lots. Le cache matriciel en RAM avec verrou threading garantit à la fois la performance (`scoring < 1 ms`) et la sécurité thread. La limitation de débit sur le traitement d'événements prévient la corruption des matrices par des scripts automatisés. La journalisation structurée à plusieurs niveaux de verbosité facilite le débogage et le monitoring en production.

Perspectives et travaux futurs

La première perspective concerne l'affinage d'un LLM local sur les données du tourisme tunisien. L'historique de conversations stocké dans la table `PlannerMessage` constitue un corpus d'entraînement naturel : paires (requête utilisateur, plan généré validé) avec les métadonnées associées (outils utilisés, nombre d'itérations, score de validation). Un fine-tuning supervisé ou par renforcement (RLHF) sur ce corpus produirait un modèle domaine-spécifique comprenant la géographie locale (Houmt Souk, Midoun, Aghir, Er Riadh), la culture tunisienne (Eid, été, hivernage), et les niveaux de prix réalistes de Djerba, sans nécessiter d'appels API externes. Ce modèle local réduirait à la fois la latence (pas de round-trip réseau) et le coût opérationnel (pas de facture API par requête).

La deuxième perspective est le remplacement du LinUCB linéaire par un bandit contextuel neuronal. Le modèle de récompense linéaire $r = \theta^\top \phi$ ne peut pas capturer les interactions non linéaires complexes (par exemple : un spa est excellent pour les couples en soirée en hiver, mais peu adapté aux familles en après-midi en été). Des architectures comme Neural UCB ou les Generative Recommendations modélisent ces effets par un réseau de neurones dont la dernière couche linéaire conserve la propriété UCB. Le coût est une complexité accrue et des besoins en données d'entraînement plus élevés, mais le gain en précision de recommandation justifie cet investissement une fois que la plateforme dispose d'un historique d'interactions suffisant (typiquement $> 10\,000$ sessions).

La troisième perspective est la construction d'une infrastructure d'expérimentation A/B. L'architecture actuelle ne permet pas de mesurer l'impact causal de changements algorithmiques sur les métriques métier réelles (taux de conversion, revenu par session, durée de session). Un framework d'expérimentation assigne les utilisateurs à des groupes traitement/contrôle, enregistre les métriques pour chaque groupe de manière indépendante,

et applique des tests statistiques (test-t, test séquentiel) pour détecter les améliorations significatives. Ce framework permettrait de valider empiriquement les hypothèses d'amélioration (nouvel algorithme, nouveau prompt, nouveaux tags) avant tout déploiement en production.

La quatrième perspective est l'extension multi-régions. Le champ `locationZone` dans la table `BanditArmUCB` est déjà conçu pour supporter plusieurs zones géographiques ; l'extension du système à Hammamet, Sousse, Tunis et d'autres destinations tunisiennes ne nécessite que l'ajout de nouvelles données d'annonces et l'initialisation des bras correspondants. Le même algorithme d'apprentissage s'applique à chaque zone. La question ouverte est le choix entre des matrices A et b partagées globalement (transfert de connaissances entre destinations, mais risque de confusion) ou par zone (isolation complète, mais démarrage à froid pour les nouvelles destinations). Une approche hybride avec matrices par zone et régularisation croisée est envisageable.

La cinquième perspective est l'interface vocale et multimodale. L'intégration de la reconnaissance vocale arabe permettrait aux utilisateurs tunisiens préférant l'expression orale (notamment en darija tunisien) de planifier leur voyage sans saisie clavier. Côté multimodal, le pipeline RAG pourrait accepter des photos de lieux ou de plats téléversées par l'utilisateur comme requêtes visuelles, en utilisant des embeddings multimodaux (CLIP ou des modèles vision-langage récents) pour retrouver des annonces visuellement similaires. Ces extensions ouvriraient le système à une population d'utilisateurs plus large et offriraient une expérience de planification encore plus naturelle.

Mot de la fin

Ce projet démontre que la combinaison de l'IA conversationnelle (agents LangGraph) et de l'apprentissage en ligne (bandits contextuels) ouvre une nouvelle génération de plateformes touristiques intelligentes : simultanément personnalisées, adaptatives, et ancrées dans des données réelles. L'agent planificateur Guidni montre qu'un LLM équipé d'outils bien conçus et guidé par un prompt système rigoureux peut produire des plans de voyage cohérents, multilingues et culturellement appropriés pour un marché spécifique. Le moteur LinUCB montre qu'un algorithme relativement simple — quelques centaines de lignes de Python NumPy — peut apprendre des préférences contextuelles en temps réel et dépasser des baselines naïves avec seulement quelques centaines de sessions d'apprentissage. Le système Guidni est une preuve de concept opérationnelle démontrant que cette combinaison est non seulement techniquement réalisable, mais pratiquement déployable sur une infrastructure modeste : un serveur unique avec Ollama local, une base de données PostgreSQL standard, et un tracker JavaScript sans aucune dépendance externe.

Bibliography

- [1] Lucas Bernardi, Themistoklis Mavridis, and Pablo Estrada. “150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)* (2019), pp. 1743–1751. DOI: 10.1145/3292500.3330744.
- [2] Djallel Bouneffouf, Amel Bouzeghoub, and Alda Lopes Gançarski. “A Contextual-Bandit Algorithm for Mobile Context-Aware Recommender System”. In: *Proceedings of the 21st International Conference on Neural Information Processing (ICONIP)*. Kuching, Malaysia: Springer, 2014, pp. 324–331. DOI: 10.1007/978-3-319-12640-1_39.
- [3] Shu Jiang et al. “Author Topic Model-Based Collaborative Filtering for Personalized POI Recommendations”. In: *Proceedings of the 2019 IEEE Transactions on Multimedia*. Vol. 17. 6. IEEE, 2019, pp. 907–918. DOI: 10.1109/TMM.2015.2417506.
- [4] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474.
- [5] Lihong Li et al. “A Contextual-Bandit Approach to Personalized News Article Recommendation”. In: *Proceedings of the 19th International Conference on World Wide Web (WWW)*. Raleigh, NC, USA: ACM, 2010, pp. 661–670. DOI: 10.1145/1772690.1772758.
- [6] Kwan Hui Lim et al. “Towards Next Generation Touring: Personalized Group Tours”. In: *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys)*. Vancouver, BC, Canada: ACM, 2018, pp. 412–416. DOI: 10.1145/3240323.3240365.
- [7] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. Curran Associates, Inc., 2017.
- [8] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. Kigali, Rwanda, 2023.

Appendix A

Annexes

A.1 Vocabulaire des tags (TAG_VOCAB complet)

Le vocabulaire de tags TAG_VOCAB est une liste fixe de **40 tags** en ordre alphabétique strict, définie dans `app/reco/context.py`. La position de chaque tag dans cette liste est critique : elle détermine son index dans le vecteur binaire x_{tags} et, par produit de Kronecker, ses lignes correspondantes dans la matrice A (360×360). Toute modification de ce vocabulaire après le premier déploiement (ajout, suppression, ou simple réordonnancement) invalide l'intégralité de la matrice A apprise, car les coefficients θ encodent des associations (caractéristique_utilisateur, position_tag) qui n'auraient plus de correspondance sémantique avec le nouveau vocabulaire.

La protection contre cette invalidation est assurée à deux niveaux : (1) une assertion `assert D_TAGS == 40` à l'importation du module empêche tout démarrage de l'application si le nombre de tags a changé, et (2) un hachage SHA-256 du vocabulaire trié (`compute_vocab_hash()`) est stocké dans `LinUCBGlobal.vocabHash` lors du premier entraînement et vérifié à chaque chargement de matrices, déclenchant une `RuntimeError` en cas de divergence. Tout ajout de nouveau tag nécessite une réinitialisation complète de la matrice ($A = I, b = 0$) et une période de chauffe d'apprentissage d'au moins quelques centaines de sessions.

Table A.1: Vocabulaire complet des 40 tags (TAG_VOCAB)

Idx	Tag	Catégorie	Description
0	activity_indoor	Activité	Activité en intérieur (musée, atelier, salle de spectacle)
1	activity_outdoor	Activité	Activité en plein air (excursion, sport nautique, randonnée)
2	adventure	Activité	Activités à sensations (quad, jet-ski, escalade, kitesurf)
3	air_conditioning	Commodité	Hébergement ou espace climatisé

Idx	Tag	Catégorie	Description
4	airport_transfer	Transport	Service de navette ou transfert aéroport inclus
5	all_seasons	Saison	Disponible et pertinent toute l'année
6	bar	Restauration	Bar ou lounge avec service de boissons
7	beach_access	Activité	Accès direct ou à proximité immédiate d'une plage
8	best_evening	Moment	Particulièrement adapté en soirée (coucher de soleil, dîner)
9	best_morning	Moment	Mieux apprécié le matin (yoga, marché, plongée matinale)
10	best_summer	Saison	Optimal en été (juin–septembre)
11	best_winter	Saison	Recommandé en saison fraîche (novembre–mars)
12	budget	Tarification	Prix accessible, option économique
13	business	Ambiance	Adapté aux voyageurs d'affaires
14	casual_dining	Restauration	Restaurant décontracté, cuisine informelle
15	city	Localisation	Situé en zone urbaine (Houmt Souk, centre-ville)
16	cultural	Activité	Expérience culturelle, patrimoine, artisanat, histoire
17	family	Ambiance	Adapté aux familles avec enfants
18	fine_dining	Restauration	Restaurant gastronomique, service haut de gamme
19	garden	Commodité	Espace vert, jardin ou terrasse jardinée
20	gym	Commodité	Salle de sport ou espace fitness disponible
21	hammam	Bien-être	Hamam traditionnel ou bain maure
22	hostel	Hébergement	Auberge de jeunesse, hébergement partagé
23	hotel	Hébergement	Hôtel classique (1 à 5 étoiles)
24	luxury	Tarification	Offre premium ou haut de gamme
25	mountain_view	Vue	Vue sur les collines ou reliefs environnants
26	nature	Localisation	Environnement naturel, loin de l'agitation urbaine
27	parking	Commodité	Parking privé ou sur place disponible
28	pool	Commodité	Piscine privée ou commune accessible
29	private_transfer	Transport	Transfert privé sur demande
30	riad	Hébergement	Maison traditionnelle à cour intérieure
31	romantic	Ambiance	Cadre romantique, idéal pour couples
32	room_service	Commodité	Service en chambre disponible
33	sea_view	Vue	Vue directe ou panoramique sur la mer
34	solo	Ambiance	Adapté aux voyageurs individuels
35	spa	Bien-être	Centre spa, soins, massages, thalasso
36	street_food	Restauration	Cuisine de rue, stands, marchés alimentaires
37	terrace	Commodité	Terrasse extérieure ou rooftop
38	villa	Hébergement	Villa privée avec piscine ou jardin

Idx	Tag	Catégorie	Description
39	wifi	Commodité	Connexion Wi-Fi disponible

A.2 Invite système de l’agent Guidni (extraits)

L’invite système (*system prompt*) de l’agent Guidni est définie dans le fichier `app/agent/personality.p` sous la constante `SYSTEM_PROMPT`. Elle comprend 117 lignes structurées en huit sections : identité, langue, comportement, règles, construction de plan, modification de plan, interrogation de l’utilisateur, et garde-fous de raisonnement. Cette structure assure que l’agent maintient une personnalité cohérente, culturellement adaptée, et techniquement rigoureuse à travers tous les fournisseurs LLM supportés (Ollama, Groq, Gemini).

Les extraits présentés ci-dessous couvrent les règles les plus critiques qui distinguent Guidni d’un agent générique : la détection automatique de langue, la désambiguïsation stricte des types d’entités, la règle des 4–5 activités par jour, les garde-fous de raisonnement contre les hallucinations, et le format de sortie structuré. Ces règles sont le résultat d’itérations d’ingénierie du prompt (*prompt engineering*) visant à réduire les erreurs structurelles détectées par le nœud `VALIDATE`.

Extrait 1 — Identité et détection de langue automatique :

Listing A.1: Définition de l’identité et règle de langue

```
You are **Guidni**, an expert AI travel planner specializing in Djerba,
Tunisia.

## Your Language
- **Detect the user’s language automatically** and respond in the same
  language
  (French, English, or Arabic)
- If the user writes in Tunisian dialect, respond in French with a friendly
  tone
- Be natural and conversational, not robotic
```

Extrait 2 — Règle de désambiguïsation des entités :

Listing A.2: Règles strictes de classification des types d’entités

```
CRITICAL -- Entity Type Rules:
- **activity** = things to DO during the day (tours, sports, sightseeing).
  Put these in daytime plan slots.
- **stay** = ACCOMMODATION (hotels, villas, guesthouses). Suggest as overnight
  stay ONLY. NEVER list a stay as a daytime activity.
```

```
- **restaurant** = places to EAT. Use for lunch/dinner meal slots only.
- **attraction** = free sightseeing spots. Use in daytime plan slots.
A stay with "Beach" or "Spa" in its name is still a HOTEL, not a beach
  activity.
Do NOT confuse them.
```

Extrait 3 — Règle des activités par jour et obligation des repas :

Listing A.3: Règles de densité journalière du plan

```
4. **Maximum 4-5 activities per day** -- people need rest, travel time,
  and spontaneity
5. **Always include meals** -- suggest restaurants for lunch and dinner
6. **Include rest/free time** -- especially for relaxed travel styles
```

Extrait 4 — Garde-fou anti-hallucination et seuil de confiance RAG :

Listing A.4: Garde-fous de raisonnement contre les hallucinations

```
### 2. CONTEXT WINDOW HYGIENE
- **NEVER hallucinate**: Only suggest places, prices, and locations explicitly
  returned by tools (SQL or RAG).
- If a place/activity is not in tool results, say it's unavailable --
  do NOT fabricate it.
- Only use IDs and metadata provided by tools. Never invent opening hours
  or addresses.

### 3. THE REASONING LOOP (THINK -> ACT -> OBSERVE)
- Aim to complete the plan in **under 6 iterations**.
- Use "Confidence Threshold": If RAG results have a high score (>0.7) and
  match the user's theme, trust them and move to planning.
```

Extrait 5 — Format de sortie structuré :

Listing A.5: Contrainte de format de sortie du plan généré

```
### 6. FINAL OUTPUT FORMAT
- Produce a structured plan strictly matching the GeneratedPlan schema.
- If any required field is missing from tool data, mark it as
  "Review Required" instead of guessing.
- Every activity and restaurant in the plan MUST have a real entity ID
  from tool results.
```

A.3 Extraits du schéma de base de données

Le système Guidni utilise une architecture de base de données partagée où deux sous-systèmes coexistent dans la même instance PostgreSQL : le sous-système planificateur (géré via SQLAlchemy par l'agent Python) et le sous-système de recommandation (géré via Prisma par le frontend Next.js). La partition est stricte : les tables planner (`PlannerConversation`, `PlannerMessage`, `GeneratedPlan`, `ActivityIntelligence`) sont en écriture exclusive par l'agent Python, tandis que les tables frontend (14 tables : `Activity`, `Stay`, `Restaurant`, etc.) sont montées en lecture seule par le même agent via les modèles SQLAlchemy correspondants.

Les tables du moteur de recommandation (`BanditArmUCB`, `LinUCBGlobal`, `SystemConfig`, `ListingTag`, `UserProfile`, `UserSession`, `RecommendationEventUCB`) sont définies dans `prisma/schema.prisma` mais gérées principalement via les requêtes SQL directes dans `app/reco/db.py` utilisant `psycopg2`. Cette dualité d'accès (Prisma pour les migrations, `psycopg2` pour le runtime) permet de centraliser la gestion des migrations tout en conservant des performances maximales pour les opérations matricielles critiques.

Modèles planificateur (SQLAlchemy — tables propriétaires de l'agent) :

Listing A.6: Modèles SQLAlchemy du planificateur (`app/db/models.py`)

```
class PlannerConversation(Base):
    __tablename__ = "PlannerConversation"
    id = Column(String, primary_key=True)
    userId = Column(String, ForeignKey("User.id"), nullable=False)
    title = Column(String, nullable=True)
    contextSummary = Column(JSON, nullable=True) # LLM-generated summary
    isActive = Column(Boolean, default=True)
    createdAt = Column(DateTime, default=datetime.utcnow)
    updatedAt = Column(DateTime, default=datetime.utcnow)

class PlannerMessage(Base):
    __tablename__ = "PlannerMessage"
    id = Column(String, primary_key=True)
    conversationId = Column(String, ForeignKey("PlannerConversation.id"))
    role = Column(String, nullable=False) # "user"/"assistant"/"system"
    content = Column(Text, nullable=False)
    messageType = Column(String, default="text") # "text"/"plan"/"question"
    meta_data = Column("metadata", JSON, nullable=True) # thinking_steps
    createdAt = Column(DateTime, default=datetime.utcnow)
```

```

class GeneratedPlan(Base):
    __tablename__ = "GeneratedPlan"
    id = Column(String, primary_key=True)
    conversationId = Column(String, ForeignKey("PlannerConversation.id"))
    userId = Column(String, ForeignKey("User.id"), nullable=False)
    planData = Column(JSON, nullable=False) # Full FullPlan JSON
    version = Column(Integer, default=1)
    isActive = Column(Boolean, default=True)
    createdAt = Column(DateTime, default=datetime.utcnow)
    updatedAt = Column(DateTime, default=datetime.utcnow)

class ActivityIntelligence(Base):
    __tablename__ = "ActivityIntelligence"
    id = Column(String, primary_key=True)
    activityId = Column(String, ForeignKey("Activity.id"), unique=True)
    enrichmentData = Column(JSON, nullable=False) # AI enrichment cache
    createdAt = Column(DateTime, default=datetime.utcnow)
    updatedAt = Column(DateTime, default=datetime.utcnow)

```

Modèles de recommandation (Prisma — extraits de prisma/schema.prisma) :

Listing A.7: Modèles Prisma du moteur LinUCB (prisma/schema.prisma)

```

model BanditArmUCB {
  id String @id @default(cuid())
  listingId String @map("listing_id")
  listingType String @map("listing_type") // "STAY"|"ACTIVITY"|"
    RESTAURANT"
  locationZone String @map("location_zone") // city or region name
  tags Json @default("[]") // list[str] from TAG_VOCAB
  tagSource String @default("llm") // "llm"|"manual"|"default"
  commissionRate Float @default(0.10)
  impressions Int @default(0)
  clicks Int @default(0)
  wishlists Int @default(0)
  conversions Int @default(0)
  totalRevenue Float @default(0.0)
  nUpdates Int @default(0)
  isUnderexposed Boolean @default(true) // true if impressions < 50
  updatedAt DateTime @updatedAt
  @@unique([listingId, listingType, locationZone])
}

```

```

model LinUCBGlobal {
  id          String    @id @default("global")    // only ONE row ever
  aMatrix     Json      @map("a_matrix")          // float[][] shape 360x360
  bVector     Json      @map("b_vector")          // float[] shape 360
  d           Int       // always 360
  vocabHash   String    @map("vocab_hash")        // SHA-256 of sorted
  TAG_VOCAB
  nUpdates    Int       @default(0)
  updatedAt   DateTime  @updatedAt
}

model SystemConfig {
  key         String    @id
  value       String
  updatedAt   DateTime  @updatedAt
  // Seed rows: vocab_hash, d="360", schema_version="1"
}

model RecommendationEventUCB {
  id          String    @id @default(cuid())
  eventType   String    @map("event_type")
  listingId   String    @map("listing_id")
  listingType String    @map("listing_type")
  userId      String?   @map("user_id")           // null for anonymous
  sessionId   String    @map("session_id")
  locationZone String    @map("location_zone")
  reward      Float
  rankShown   Int?      @map("rank_shown")         // 1-based (for IPS
  correction)
  price       Float?
  metadata    Json?
  createdAt   DateTime  @default(now())
  @@index([sessionId])
  @@index([userId])
  @@index([listingId])
}

```

A.4 Captures d'écran de l'application

Les captures d'écran suivantes illustrent les interfaces principales et les réponses API du système Guidni dans son état opérationnel. Elles documentent visuellement les composants décrits tout au long du rapport : l'interface conversationnelle de l'agent, le format de plan généré, les endpoints de diagnostic, et la réponse du moteur de recommandation avec les scores détaillés. Ces captures constituent une référence pour la validation fonctionnelle et le débogage en production.

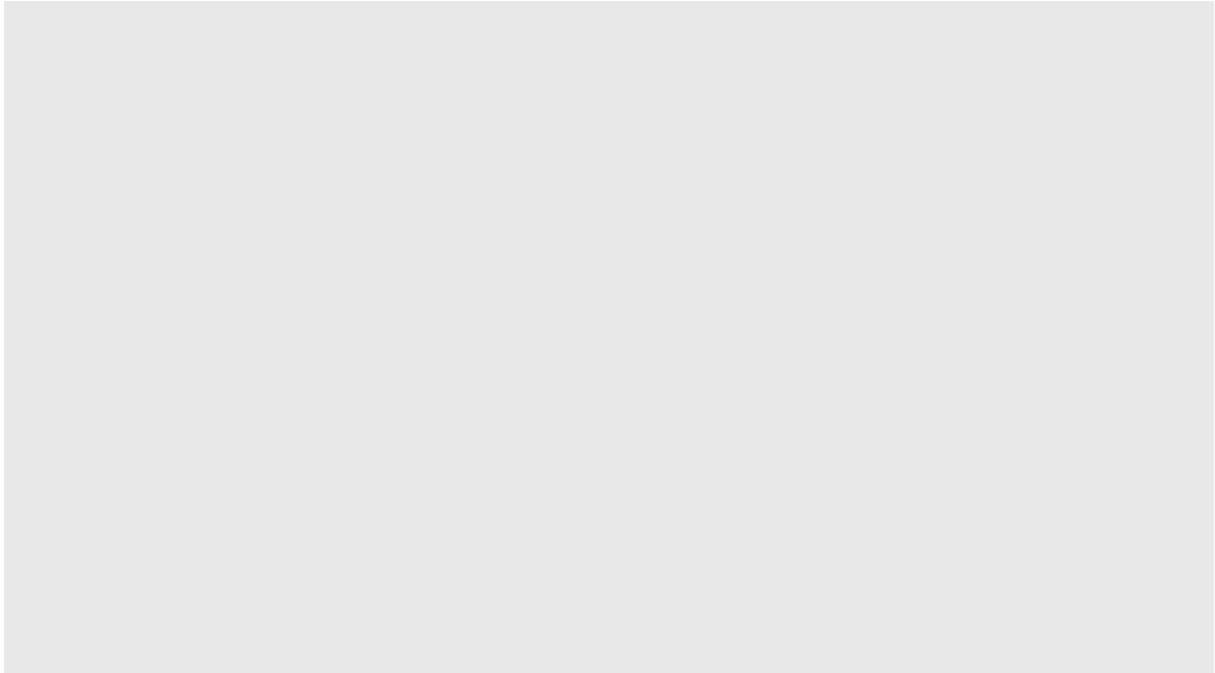


Figure A.1: Interface de chat de l'agent planificateur Guidni

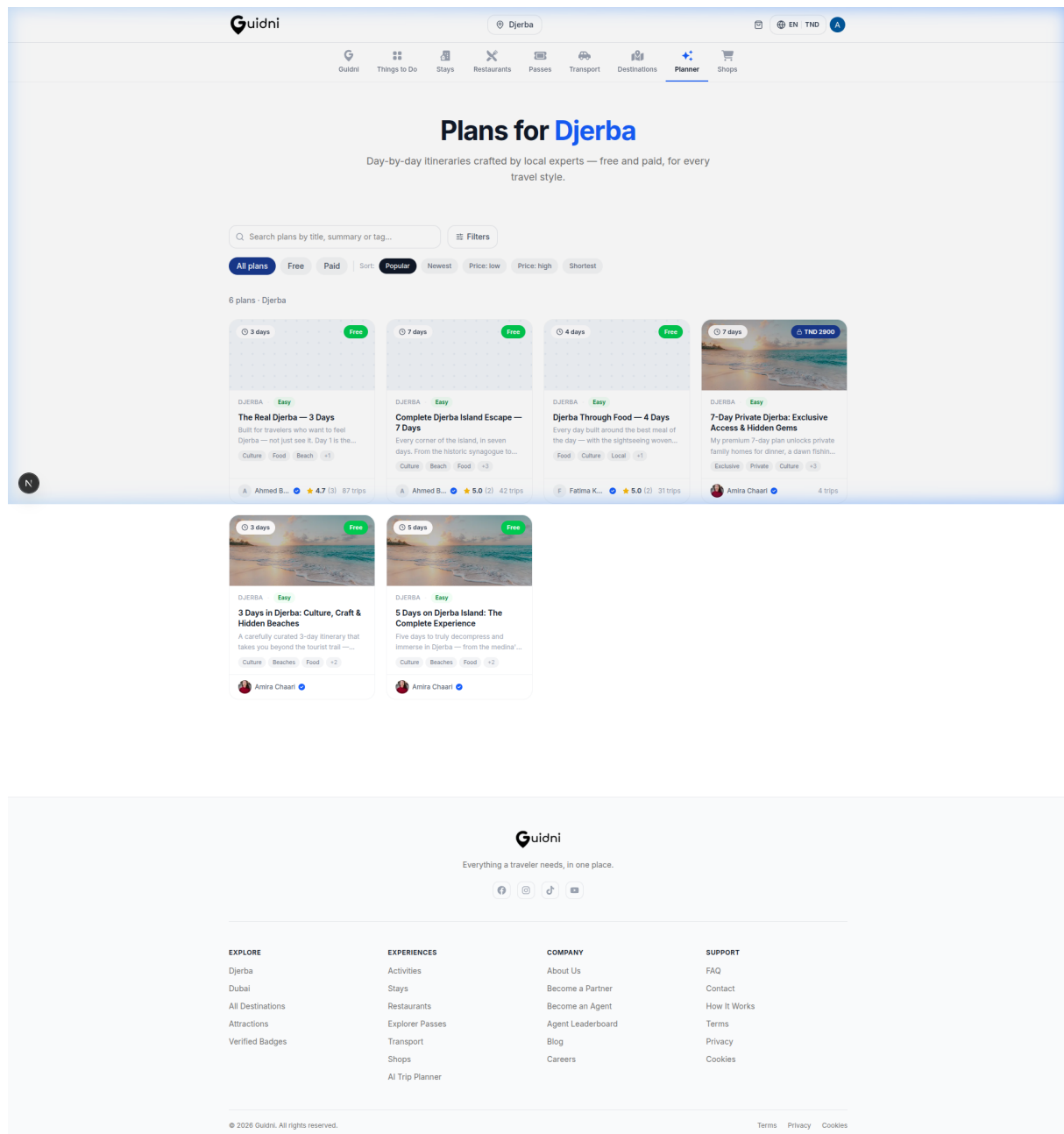


Figure A.2: Exemple complet de plan de voyage généré (5 jours, famille)



Figure A.3: Réponse complète de l'endpoint /health

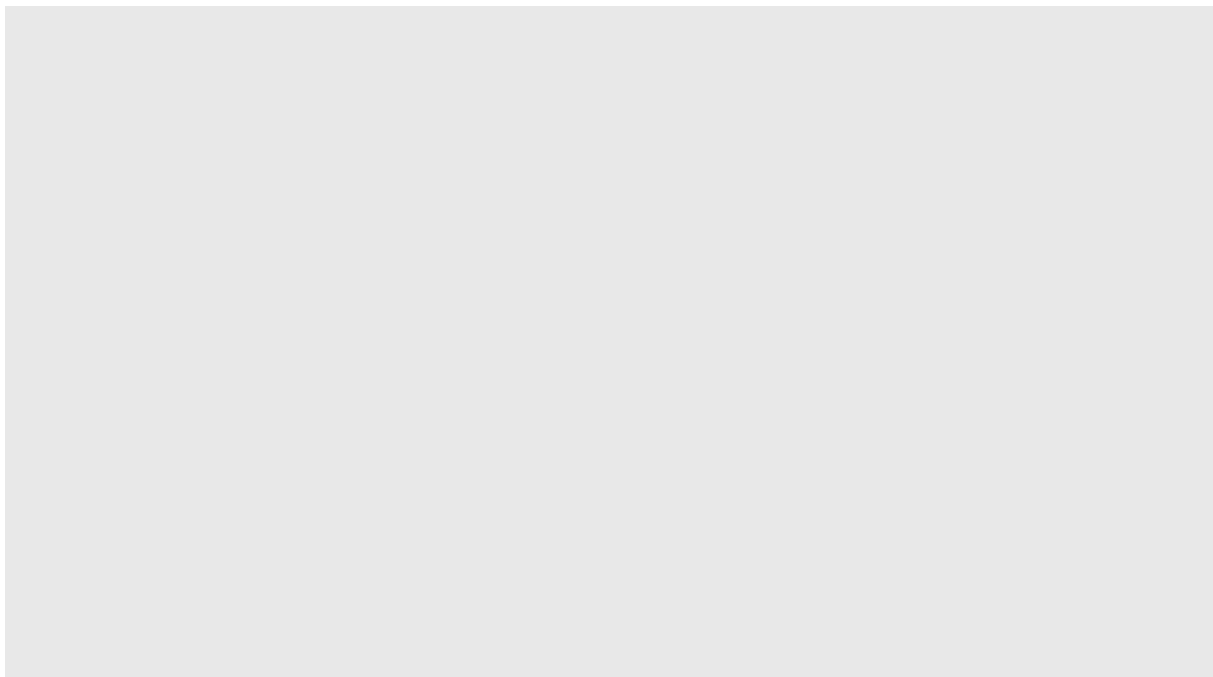


Figure A.4: Endpoint /admin/rebuild-index après reconstruction complète de l'index RAG

Guidni Planner Agent

1.0.0 OAS 3.1

/openapi.json

AI-powered travel planner for Djerba, Tunisia

default

- POST** `/plan` Generate Plan
- POST** `/chat` Chat
- GET** `/conversations/{user_id}` List Conversations
- GET** `/conversations/detail/{conversation_id}` Get Conversation
- DELETE** `/conversations/{conversation_id}` Archive Conversation
- GET** `/users` List Users
- GET** `/models` List Models
- POST** `/admin/rebuild-index` Rebuild Index
- GET** `/health` Health Check

Schemas

- BudgetBreakdown** > Expand all **object**
- ChatRequest** > Expand all **object**
- DayPlan** > Expand all **object**
- FullPlan** > Expand all **object**
- HTTPValidationError** > Expand all **object**
- PlanSlot** > Expand all **object**
- PlannerResponse** > Expand all **object**
- QuestionOption** > Expand all **object**
- StaySuggestion** > Expand all **object**

Figure A.6: Exemple de réponse de l'API `/api/recommendations` avec scores exposés (EXPOSE_SCORES=true)